MOST IMPORTANT FIRST | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB

# Google study path (clean full code) - HARD only

53 Hard questions in 30 groups (the same code-template groups as the full book, only the Hard questions). Each group shares ONE code template, shown on the page before its questions.

| 0 | 0 | 53 |
|---|---|---|
| EASY | MEDIUM | HARD |

**Sections (tap one to jump to it)**


1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance — 2 questions | page 6
2. Sweep over events in time order — 3 questions | page 20
3. Tree recursion, Tier 1: one function that visits a node, asks its children, a... — 2 questions | page 40
4. A heap of what is available: pop the best, put back the changed one — 1 question | page 55
5. Recurrence on a position or a range (memo or table) — 3 questions | page 64
6. Best-first search with a heap: Dijkstra and its minimax twin — 4 questions | page 83
7. Backtracking: choose, recurse, undo — 1 question | page 110
8. Union-find: merge the pairs, then ask who is together — 1 question | page 119
9. Binary search on a yes/no condition — 3 questions | page 128
10. Sets and counters: remember the first collection, check the second agai... — 1 question | page 148
11. bisect on a sorted list: find a position, count a range, keep it sorted — 2 questions | page 156
12. Sorted intervals: merge while they touch, then read the gaps or search — 2 questions | page 171
13. Trie: nested dicts, one character per level — 1 question | page 186
14. A dict for lookup plus an ordered container for who is next (lazy deletion) — 1 question | page 197
15. Rolling state: the next state comes only from the previous one — 1 question | page 209
16. Walk two strings at once with two indices — 1 question | page 216
17. Simulate a machine: a class with state and one branch per kind of event — 1 question | page 224
18. Digits and characters: peel one at a time and carry — 2 questions | page 232

## Sections, continued (tap one to jump to it)


| | | |
|---|---|---|
| **19. Look at the input once and keep only a small summary (and count your...** | 2 questions | page 245 |
| **20. Grid DP: each cell from its neighbours above and to the left** | 2 questions | page 258 |
| **21. Counting pairs by looking up each partner in a Counter or a bucket** | 1 question | page 273 |
| **22. Tree DP: build a parent order once, then add up from the leaves** | 2 questions | page 280 |
| **23. Prefix sums, Tier 2: running total plus a dict of earlier totals** | 1 question | page 293 |
| **24. Group by a canonical key** | 1 question | page 301 |
| **25. K-way merge: a heap of the next item from each source** | 2 questions | page 311 |
| **26. Random pick without repeats: swap the chosen item with the last one an...** | 1 question | page 324 |
| **27. Build and serialize: write a structure out, read it back with the same walk** | 2 questions | page 332 |
| **28. Games: win or lose with a cache, and guessing with feedback** | 4 questions | page 347 |
| **29. Sliding window: grow on the right, shrink on the left while it is too bad** | 1 question | page 370 |
| **30. Split into pieces, fix each piece, join (and pack words into lines)** | 2 questions | page 378 |


- The master contents lists every main topic and question; each row is a link.
- Inside a topic, the bar at the top of each page moves Prev / Contents / Next; the link at the bottom returns here.
- Questions run Easy, then Medium, then Hard inside each main topic, newest date first.

Go to Contents >

# Contents

**53 questions | tap a topic, a main topic or a question to jump to it**


**START HERE: topics in the order of importance, the most important question first inside each**

**1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (2)** Go >

- H 1. Infection Spread on a Grid (Cellular Automaton) — PracHub 2026-06-14
- H 2. Shortest Distance From All Buildings — ColorStack 2025-09-22

**2. Sweep over events in time order (3)** Go >

- H 3. Count Overlapping Rectangle Updates on a Grid — PracHub 2026-07-10
- H 4. Compute city skyline outline — PracHub 2025-12-08
- H 5. Compute Turnstile Crossing Times — PracHub 2026-05-04

**3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upw...** Go >

- H 6. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — Tree PracHub 2026-09-16
- H 7. Delete an Entire Binary Tree — ColorStack 2025-09-29

**4. A heap of what is available: pop the best, put back the changed one (1)** Go >

- H 8. Huffman Tree Construction — 1Point3Acres 2026-06-01

**5. Recurrence on a position or a range (memo or table) (3)** Go >

- H 9. Edit distance — Exponent no date
- H 10. Count Perfect Wake Numbers up to N — 1Point3Acres 2026-05-07
- H 11. Word Break: All Sentences — ColorStack 2026-01-28

**6. Best-first search with a heap: Dijkstra and its minimax twin (4)** Go >

- H 12. Minimize maximum height along a grid path — PracHub 2026-01-04
- H 13. Find minimum threshold enabling grid path — PracHub 2026-02-11
- H 14. Minimize travel time with optional meeting point — PracHub 2026-01-06
- H 15. Compute minimax grid path and network delay — PracHub 2026-01-22

**7. Backtracking: choose, recurse, undo (1)** Go >

- H 16. Matrix Flower Placement with House Adjacency — 1Point3Acres 2026-05-31

**8. Union-find: merge the pairs, then ask who is together (1)** Go >

- H 17. Friend Graph with Timeline (Friend / Unfriend Events) — 1Point3Acres 2026-09-29

**9. Binary search on a yes/no condition (3)** Go >

- H 18. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — 1Point3Acres 2026-10-09
- H 19. Find the median of two sorted arrays. — Exponent no date
- H 20. Find longest duplicated substring — PracHub 2025-12-08

**10. Sets and counters: remember the first collection, check the second against it (1)** Go >

- H 21. Smallest Missing Positive Integer in Linear Time and Constant Extra Space — Space PracHub 2026-09-09

# Contents

**53 questions | tap a topic, a main topic or a question to jump to it**


**11. bisect on a sorted list: find a position, count a range, keep it sorted (2)** Go >

H 22. Find Maximum Envelope Nesting — **PracHub 2026-03-18**
H 23. Count of Smaller Numbers After Self — **HackTheRounds no date**

**12. Sorted intervals: merge while they touch, then read the gaps or search (2)** Go >

H 24. Find safe travel intervals between planet influences — **PracHub 2025-11-22**
H 25. Implement a function that processes a stream of interval data. — **Exponent no date**

**13. Trie: nested dicts, one character per level (1)** Go >

H 26. Implement Longest-Match Text Replacement — **PracHub 2026-01-20**

**14. A dict for lookup plus an ordered container for who is next (lazy deletion) (1)** Go >

H 27. Implement LRU Cache. — **Exponent no date**

**15. Rolling state: the next state comes only from the previous one (1)** Go >

H 28. Constrained Dice Roll Sequences — **ColorStack 2025-10-06**

**16. Walk two strings at once with two indices (1)** Go >

H 29. Find Dictionary Matches for a Jammed String — **PracHub 2026-04-09**

**17. Simulate a machine: a class with state and one branch per kind of event (1)** Go >

H 30. Simulate In-Place Cellular State Updates — **PracHub 2026-05-04**

**18. Digits and characters: peel one at a time and carry (2)** Go >

H 31. Check Digits and Combine Ranges — **PracHub 2026-03-29**
H 32. Permutation Sequence — **ColorStack 2025-10-17**

**19. Look at the input once and keep only a small summary (and count your calls) (2)** Go >

H 33. Find All Bad Pairs Using runTest Oracle — **1Point3Acres 2025-11-27**
H 34. Find maximum value in huge text file — **PracHub 2025-12-08**

**20. Grid DP: each cell from its neighbours above and to the left (2)** Go >

H 35. Find the Longest Straight Run of One Color — **PracHub 2026-08-23**
H 36. Count same-color squares in a character grid — **PracHub 2025-11-22**

**21. Counting pairs by looking up each partner in a Counter or a bucket (1)** Go >

H 37. Maximum-Area Rectangle from Planar Points — **1Point3Acres 2026-09-29**

**22. Tree DP: build a parent order once, then add up from the leaves (2)** Go >

H 38. Compute distance-sum from every tree node — **PracHub 2026-01-22**
H 39. Tree Distance Sum (Re-root DP) — **1Point3Acres 2026-01-19**

# Contents

**53 questions | tap a topic, a main topic or a question to jump to it**


**23. Prefix sums, Tier 2: running total plus a dict of earlier totals (1)** Go >

H 40. Given an integer array nums and an integer k, return true if nums has a sub... **a subarray of at least two el... Exponent no date**

**24. Group by a canonical key (1)** Go >

H 41. Given two strings and two allowed operations, swapping positions of lett... **of letters and converting all occ... Exponent no date**

**25. K-way merge: a heap of the next item from each source (2)** Go >

H 42. Merge k sorted linked lists. **Exponent no date**

H 43. Return top-k subset sums in descending order **PracHub 2026-01-07**

**26. Random pick without repeats: swap the chosen item with the last one and pop (1)** Go >

H 44. Shuffle array using random integer API **PracHub 2025-12-08**

**27. Build and serialize: write a structure out, read it back with the same walk (2)** Go >

H 45. Implement a decode and encode function of a general N-ary tree. **Exponent no date**

H 46. Serialize and Deserialize a Graph **ColorStack 2025-09-24**

**28. Games: win or lose with a cache, and guessing with feedback (4)** Go >

H 47. Wordle-Style 5-Letter Minimum-Guess Strategy **1Point3Acres 2025-12-15**

H 48. Determine winner in stack-merging game **PracHub 2026-01-06**

H 49. Find secret word with match-count feedback **PracHub 2026-02-11**

H 50. Babylon Tile-Stack Merge Game **1Point3Acres 2025-12-29**

**29. Sliding window: grow on the right, shrink on the left while it is too bad (1)** Go >

H 51. Necklace Cut into Two Halves with Equal D/R Counts **1Point3Acres 2026-05-03**

**30. Split into pieces, fix each piece, join (and pack words into lines) (2)** Go >

H 52. Implement a function to justify a list of words within a given width. **Exponent no date**

H 53. Convert Valid Snake Case to Lower Camel Case **PracHub 2026-04-16**

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 1 OF 30 |

# Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 1. Infection Spread on a Grid (Cellular Automaton) — PracHub 2026-06-14

H 2. Shortest Distance From All Buildings — ColorStack 2025-09-22

CODE TEMPLATE | GROUP 1 OF 30 | 2 QUESTIONS, ONE PATTERN

# Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

## The one idea these questions share

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def frontier_search(starts, neighbors, usable=lambda v: True):
    dist = {s: 0 for s in starts}          # doubles as the seen set
    frontier = deque(starts)               # popleft = BFS, pop = DFS
    while frontier:
        u = frontier.popleft()
        # CHANGE 1: act on u here (repaint it, count it, stop at the goal)
        for v in neighbors(u):             # CHANGE 2: grid steps or adj[u]
            if v not in dist and usable(v):    # CHANGE 3: colour, wall, blocked
                dist[v] = dist[u] + 1
                frontier.append(v)
    return dist
```

## 2. Helper functions that usually get written next to it

```
def grid_steps(rows, cols):
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc)
    return step


def rebuild_path(parent, target):         # walk parent[] back from the target
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| 1 | Infection Spread on a Grid (Cellular Automaton) | starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds) |
| 2 | Shortest Distance From All Buildings | run it once per building (one start each); add each run into a total table and count how many buildings reached each cell |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 1 of 53 | topic 1 of 30: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 1 of 2**

Template fit: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds) | asked 128 days ago

**HARD | PracHub | Technical Screen | 1 of 53**

# Infection Spread on a Grid (Cellular Automaton)

**POSTED 2026-06-14 | 128 DAYS AGO | LAST 6 MONTHS**

**STUDY: TYPE** **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 75/288**

## ABOUT THIS QUESTION

**Topic:** Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

**Optimal answer from:** the interview site's own solution

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

**LeetCode match:** LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-06-14 (updated 2026-06-24)

**Original:** https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

## QUESTION

A grid holds '.' for a healthy cell, 'I' for an infected cell and any other character (such as '#') for a barrier that is never infected. Every step, all infected cells infect their healthy neighbours (up, down, left, right) at the same time. infection_times(grid) returns the step at which each infected cell was infected (0 for the cells infected at the start); spread_steps(grid) returns the number of steps until nothing new gets infected.

### Function signature

def infection_times(grid: list) -> dict
def spread_steps(grid: list) -> int

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `['I..', '...', '..#']` |
| **Output** | `3` |
| **Why** | `The infection travels from the corner: (0,1) and (1,0) at step 1, then three cells at step 2, and so on. The barrier at the bottom right is never infected.` |

**Example 2**

| | |
|---|---|
| **Input** | `['I...']` |
| **Output** | `{(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3): 3}` |
| **Why** | `A one-row grid: one cell per step.` |

**Example 3**

| | |
|---|---|
| **Input** | `['I#.', 'I#.']` |
| **Output** | `0` |
| **Why** | `Both cells next to the barrier are already infected and the barrier blocks the right column, so nothing can spread: 0 steps.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    steps = 0
    while True:
        nxt = copy(grid)
        for r, c in all_cells(grid):
            if grid[r][c] == HEALTHY:
                if sick_neighbour(grid, r, c):
                    nxt[r][c] = INFECTED
        if nxt == grid:
            return steps
        grid = nxt
        steps += 1
```

| TIME | SPACE |
|---|---|
| O(steps * m * n) | O(m * n) |

### WHY IT IS SLOW

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

## OPTIMAL SOLUTION · Frontier BFS (only recently infected cells spread)

| TIME | SPACE |
|---|---|
| O(m * n) | O(m * n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

## FOLLOW-UP 2 Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

## FOLLOW-UP 3 Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Infection Spread on a Grid (Cellular
# Automaton)
```

```
#
# A grid holds '.' for a healthy cell, 'I' for an infected
# cell and any other character (such as '#') for a barrier
# that is never infected. Every step, all infected cells
# infect their healthy neighbours (up, down, left, right)
# at the same time. infection_times(grid) returns the step
# at which each infected cell was infected (0 for the
# cells infected at the start); spread_steps(grid) returns
# the number of steps until nothing new gets infected.
#
# Signature:
#   def infection_times(grid: list) -> dict
#   def spread_steps(grid: list) -> int
#
# Examples:
# ['I..', '...', '..#']  ->  3
# ['I...']  ->  {(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3):
#     3}
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a recovery delay: an infected cell recovers after
# d steps and cannot spread afterwards.
# 2. Why can you stop early?
# 3. Make the spread probabilistic.
#
# INTERVIEW NOTES
# A synchronous cellular automaton is a BFS in disguise:
# the step number is the BFS level. Only cells next to
# cells that just changed can change next, so keep a
# frontier instead of rescanning the grid. Always compute
# the next state from the old state (not in place) so all
# cells update together.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```python
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def find_infected(grid):
    # the cells that start infected ('I')
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "I":
                cells.append((r, c))
    return cells


# ---- main question: main function ----
# Idea: only the cells infected in the last step can infect new
# cells, so keep a frontier. 'I' = infected at time 0, '.' =
# healthy, '#' = blocked. Returns the step at which each cell
# gets infected.
# Complexity: Time O(m * n), Space O(m * n)
def infection_times(grid):
```

```
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = find_infected(grid)
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        next_frontier = []
        for r, c in frontier:
            for dr, dc in STEPS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    if grid[nr][nc] == "." and (nr, nc) not in when:
                        when[(nr, nc)] = step       # every healthy neighbour is infected next step
                        next_frontier.append((nr, nc))
        frontier = next_frontier
    return when


# Complexity: Time O(m * n), Space O(m * n)
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
```

```
    last = 0
    for step in when.values():
        if step > last:
            last = step
    return last


# ---- main question: tests ----
def test_main_question():
    assert spread_steps(["I..", "...", "..."]) == 4
    wall = ["I#.", "I#."]                           # a wall splits the grid: the far cell is never infected
    assert spread_steps(wall) == 0
    assert spread_steps(["..."]) == 0


# ========================================================
# FOLLOW-UP 1: Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards
# ========================================================
# ---- follow-up 1: function ----
# Answer: a cell infects every healthy neighbour on the very next
# step, so recovery does not change how long spreading takes; it
# only changes what the grid looks like at time t.
# Complexity: Time O(m * n), Space O(m * n)
def grid_at(grid, t, recovery=None):
    when = infection_times(grid)
    result = []
    for r in range(len(grid)):
        line = ""
        for c in range(len(grid[0])):
            start = when.get((r, c))
            if start is None or start > t:
                line += grid[r][c]                  # not infected yet
            elif recovery and t >= start + recovery:
                line += "R"                         # recovered
            else:
                line += "I"
        result.append(line)
    return result
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    g = ["I..", "...", "..."]
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    assert grid_at(g, 1, recovery=1) == ["RI.", "I..", "..."]
```

```
    assert grid_at(["I#.", "I#."], 9)[0] == "I#."


# ============================================================
# FOLLOW-UP 2: Why can you stop early?
# ============================================================
# Answer: when a step infects nobody new the frontier is empty and
# nothing can change again, so the loop ends by itself.


# ============================================================
# FOLLOW-UP 3: Make the spread probabilistic
# ============================================================
# Answer: infect each healthy neighbour only with probability p
# (random.random() < p) and repeat the whole simulation many times
# to estimate the average spread time.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(grid: list[str]) -> list[str]:
```

### Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

### Test cases

| Input | Expected |
|---|---|
| `(['HHH', 'HIH', 'HHH'],)` | `['HIH', 'III', 'HIH']` |
| `(['HHH'],)` | `['HHH']` |

**STUDY PATH: question 2 of 53 | topic 1 of 30: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 2 of 2**

Template fit: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell | asked 393 days ago | frequency Single report

HARD | IN APP LC 317 | ColorStack | technical interview | 2 of 53

# Shortest Distance From All Buildings

ASKED 2025-09-22 | 393 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 202/288

## ABOUT THIS QUESTION

**Topic:** Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

**Optimal answer from:** your app's interview approach (LeetCode #317)

**LeetCode match:** LC 317 Shortest Distance from All Buildings

Frequency: Single report | Last asked: 2025-09-22 | Roles: SWE Intern

## QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

**Function signature**

```
def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...
```

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

### EXAMPLES

| Example 1 | |
|---|---|
| Input | `grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]` |
| Output | `(1, 2)` |
| Why | `Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.` |

| Example 2 | |
|---|---|
| Input | `grid = [[1, 0, 1]]` |
| Output | `(0, 1)` |
| Why | `The only empty cell is distance 1 from each house; total 2.` |

| Example 3 | |
|---|---|
| Input | `grid = [[1, 2, 1]]` |
| Output | `(-1, -1)` |
| Why | `There is no empty cell, so no valid location exists.` |

### CONSTRAINTS

## QUESTION

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
from collections import deque
def brute_school(grid):
    rows, cols = len(grid), len(grid[0])
    houses = sum(
        v == 1 for row in grid for v in row
    )
    best = (float("inf"), -1, -1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 0:
                continue
            dist, queue, total, found = (
                {(r, c): 0},
                deque([(r, c)]),
                0,
                0,
            )
            while queue:
                x, y = queue.popleft()
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < rows
                        and 0 <= ny < cols
                        and (nx, ny)
                        not in dist
                        and grid[nx][ny] != 2
                    ):
                        dist[(nx, ny)] = (
                            dist[(x, y)] + 1
                        )
                        if grid[nx][ny] == 1:
                            total, found = (
                                total
                                + dist[
                                    (nx, ny)
                                ],
                                found + 1,
                            )
                        else:
                            queue.append(
                                (nx, ny)
                            )
            if (
                found == houses
                and (total, r, c) < best
            ):
                best = (total, r, c)
    return (
        (best[1], best[2])
        if best[1] >= 0
        else
            (-1, -1)
    )
```

| TIME | SPACE |
| --- | --- |
| O(cells * R * C) | O(R * C) |

WHY IT IS SLOW

## BRUTE FORCE

Each empty cell runs its own full search for the houses.

## OPTIMAL SOLUTION · One BFS per house

| TIME | SPACE |
|---|---|
| O(houses * R * C) | O(R * C) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Minimize the average distance instead of the total.

With a fixed number of houses the average is total / houses, so the best cell is the same; compute it once at the end to report the average.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Distance From All Buildings
#
# You are given a 2D grid where each cell is: 0 = empty
# land (a school can be built here), 1 = a house, 2 = a
# tree (an obstacle that cannot be entered or crossed).
# You want to build one school on an empty cell so that
# the total travel distance from every house to the school
# is minimized. Distance is measured by 4-directional
# steps (up/down/left/right); trees block movement. A
# valid school location must be reachable from every
# house. Return the coordinates (row, col) of the empty
# cell with the minimum total distance to all houses. If
# several cells tie on total distance, return the one that
# comes first in row-major order (smallest row, then
# smallest column). If no empty cell can reach all houses,
# return (-1, -1) . Function signature: def
# shortest_distance_from_all_buildings(grid:
# list[list[int]]) -> tuple[int, int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Minimize the average distance instead of the total.
#
# INTERVIEW NOTES
# Search from the houses, not from every empty cell: one
# BFS per house adds that house's distance to every empty
# cell it reaches and counts how many houses reached the
# cell. A cell is valid only when all houses reached it.
# Choose the smallest total, ties in row-major order.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

from collections import deque


# ======================================================
```

```python
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def bfs_from_house(grid, house, total, reached):
    # walk outward from one house over empty cells (0); add the distance to
```

```python
    # `total` and count one more house in `reached` for every cell it reaches
    rows, cols = len(grid), len(grid[0])
    seen = {house}
    queue = deque([(house[0], house[1], 0)])
    while queue:
        r, c, d = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] != 0 or (nr, nc) in seen:
                continue
            seen.add((nr, nc))
            total[nr][nc] += d + 1
            reached[nr][nc] += 1
            queue.append((nr, nc, d + 1))


def school_totals(grid):
    # for every empty cell: the sum of the distances to all houses it can reach,
    # how many houses reach it, and the number of houses
    rows, cols = len(grid), len(grid[0])
    total = []
    reached = []
    for r in range(rows):
        total.append([0] * cols)
        reached.append([0] * cols)
    houses = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                houses += 1
                bfs_from_house(grid, (r, c), total, reached)
    return total, reached, houses


# ---- main question: main function ----
# Idea: one BFS per house adds its distances to the empty cells it
# reaches; a valid cell is one that every house reached. Pick the valid
# cell with the smallest total; (-1, -1) if there is none.
# Complexity: Time O(houses * m * n), Space O(m * n)
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best = None
    spot = (-1, -1)
    for r in range(len(grid)):
```

```python
        for c in range(len(grid[0])):
            if grid[r][c] == 0 and reached[r][c] == houses:
                if best is None or total[r][c] < best:
                    best = total[r][c]
                    spot = (r, c)
    return spot


# ---- main question: tests ----
def test_main_question():
    assert shortest_distance_from_all_buildings(
        [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]) == (1, 2)
    assert shortest_distance_from_all_buildings([[1, 0, 1]]) == (0, 1)
```

```
    assert shortest_distance_from_all_buildings([[1, 2, 1]]) == (-1, -1)


# ======================================================
# FOLLOW-UP 1: Minimize the average distance instead of the total
# ======================================================
# ---- follow-up 1: function ----
# Answer: with a fixed number of houses the average is total / houses,
# so the best cell is the same; divide once at the end to report it.
# Complexity: Time O(houses * m * n), Space O(m * n)
def best_school_average(grid):
    spot = shortest_distance_from_all_buildings(grid)
    if spot == (-1, -1):
        return spot, None
    total, reached, houses = school_totals(grid)
    return spot, total[spot[0]][spot[1]] / houses


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_school_average([[1, 0, 1]]) == ((0, 1), 1.0)
    assert best_school_average([[1, 2, 1]]) == ((-1, -1), None)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 2 OF 30 |

# Sweep over events in time order

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**3**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Sweep over events in time order | 3 questions | tap a title to jump to it**

**READ FIRST: the code template for these 3 questions (tap here) >**


| | | |
|---|---|---|
| H | 3. Count Overlapping Rectangle Updates on a Grid | PracHub 2026-07-10 |
| H | 4. Compute city skyline outline | PracHub 2025-12-08 |
| H | 5. Compute Turnstile Crossing Times | PracHub 2026-05-04 |

CODE TEMPLATE | GROUP 2 OF 30 | 3 QUESTIONS, ONE PATTERN

# Sweep over events in time order

## The one idea these questions share

Turn every item into events (a start adds one, an end removes one, an arrival joins a queue), sort the events by time, and walk them keeping a small state: a counter, a queue, a heap or a running total. What you record at each event is the only thing that changes.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def max_overlap(intervals):
    events = sorted([(s, 1) for s, e in intervals] + [(e, -1) for s, e in intervals])
    cur = best = 0
    for t, delta in events:                  # CHANGE 1: what an event does to your state
        cur += delta
        best = max(best, cur)                # CHANGE 2: what you record
    return best                              # ends sort before starts at the same time (-1 < 1)
```

## 2. Helper functions that usually get written next to it

```
import heapq
from collections import deque


def two_queue_clock(arrivals):               # who is served when two queues share one server
    order = sorted(range(len(arrivals)), key=lambda i: arrivals[i])
    queue, t, i, served = deque(), 0, 0, []
    while i < len(order) or queue:
        if not queue:
            t = max(t, arrivals[order[i]])
        while i < len(order) and arrivals[order[i]] <= t:
            queue.append(order[i]); i += 1
        served.append((queue.popleft(), t))
        t += 1
    return served
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **3** | Count Overlapping Rectangle Updates on a Grid | the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes |
| **4** | Compute city skyline outline | events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes |
| **5** | Compute Turnstile Crossing Times | sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 3 of 53 | topic 2 of 30: Sweep over events in time order | 1 of 3**

Template fit: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes | asked 102 days ago

HARD | IN APP LC 2536 | PracHub | Technical Screen | 3 of 53

# Count Overlapping Rectangle Updates on a Grid

POSTED 2026-07-10 | 102 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 172/288

## ABOUT THIS QUESTION

**Topic:** Sweep over events in time order

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #2536 Increment Submatrices by One

**LeetCode match:** LC 2536 Increment Submatrices by One

Posted on PracHub: 2026-07-10 (updated 2026-07-18)

**Original:** https://prachub.com/coding-questions/count-overlapping-rectangle-updates-on-a-grid

## QUESTION

### Count Overlapping Rectangle Updates on a Grid Implement

`range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

### Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

### Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

EXAMPLES

| Example 1 | |
|---|---|
| Input | `3, [(0, 0, 1, 1), (1, 1, 2, 2)]` |
| Output | `[[1, 1, 0], [1, 2, 1], [0, 1, 1]]` |
| Why | `The cell (1, 1) is covered by both rectangles, so it is counted twice.` |

| Example 2 | |
|---|---|
| Input | `2, []` |
| Output | `[[0, 0], [0, 0]]` |
| Why | `No rectangles: all zeros.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def range_add_counts(n, rectangles):
    grid = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rectangles:
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                grid[r][c] += 1
    return grid
```

| TIME | SPACE |
|---|---|
| O(R * area) | O(n^2) |

**WHY IT IS SLOW**

O(R * area): 100,000 rectangles of area up to 250,000 is billions of cell updates. A 2D difference array touches four corners per rectangle and one prefix-sum pass rebuilds the counts, O(R + n^2).

## OPTIMAL SOLUTION · 2D difference array and prefix sums

| TIME | SPACE |
|---|---|
| O(R + n^2) | O(n^2) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + n^2).

## FOLLOW-UP 2 The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. **Cost:** O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

## FOLLOW-UP 3 Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Overlapping Rectangle Updates on a
# Grid
#
# # Count Overlapping Rectangle Updates on a Grid
# Implement range_add_counts(n, rectangles). Start with an
# n x n matrix of zeroes. Each rectangle is represented as
# [row1, col1, row2, col2] with inclusive, zero-based
# coordinates. For every rectangle, add one to each matrix
# cell inside that rectangle. Return the final matrix,
```

```
# where each cell therefore contains the number of input
# rectangles covering it. ## Example For n = 3 and
# rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]], return: text
# [ [1, 1, 0], [1, 2, 1], [0, 1, 1] ] ## Constraints - 1
# <= n <= 500 - 0 <= len(rectangles) <= 100_000 - 0 <=
# row1 <= row2 < n - 0 <= col1 <= col2 < n
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many cells are covered by at least k rectangles?
# 2. The grid is huge (10^9 x 10^9) but there are few
# rectangles.
# 3. Rectangles are added and the count of a cell is
# queried online.
#
# INTERVIEW NOTES
# Adding 1 to every cell of up to 100,000 rectangles one
# cell at a time is too slow. Record only the corners of
# each rectangle (+1 at the top-left, -1 just beyond the
# right and bottom edges, +1 at the bottom-right corner
# beyond), and rebuild the counts with one pass of 2D
# prefix sums. Each rectangle then costs O(1) instead of
# its area.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# An n x n grid; each rectangle [r1, c1, r2, c2] (corners inclusive) adds 1
# to every cell inside. Return the final count of every cell.
```

```
# Idea: a 2D DIFFERENCE array. For each rectangle mark only its 4 corners:
# +1 at (r1, c1), -1 at (r1, c2 + 1), -1 at (r2 + 1, c1), +1 at
# (r2 + 1, c2 + 1). Then two prefix-sum passes (along every row, then down
# every column) spread the marks over the cells. Cost O(R + n^2) for R
# rectangles instead of O(R * n^2).
# Complexity: Time O(n^2 + R) for R rectangles, Space O(n^2)
def range_add_counts(n, rects):
    d = []
    for i in range(n + 1):
        d.append([0] * (n + 1))
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    for r in range(n + 1):                          # prefix sums along every row ...
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):                          # ... then down every column
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    counts = []
    for r in range(n):
        counts.append(d[r][:n])
    return counts


# ---- main question: tests ----
```

```python
RECTS = [[0, 0, 1, 1], [1, 1, 2, 2]]


def test_main_question():
    assert range_add_counts(3, RECTS) == [[1, 1, 0], [1, 2, 1], [0, 1, 1]]


# ========================================================
# FOLLOW-UP 1: How many cells are covered by at least k rectangles?
# ========================================================
# ---- follow-up 1: function ----
# Answer: build the count table with the difference array and count the
# cells whose value is at least k. O(R + n^2).
# Complexity: Time O(n^2 + R), Space O(n^2)
def cells_at_least(n, rects, k):
    total = 0
    for row in range_add_counts(n, rects):
        for value in row:
            if value >= k:
                total += 1
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert cells_at_least(3, RECTS, 2) == 1
    assert cells_at_least(3, RECTS, 1) == 7


# ========================================================
# FOLLOW-UP 2: The grid is huge (10^9 x 10^9) but there are few rectangles
# ========================================================
# ---- follow-up 2: function ----
# Answer: coordinate compression. Only the corner rows and columns matter, so
# keep just those (at most 2R of each) and run the same difference array on
# that small grid; each compressed cell stands for a whole BLOCK of real cells,
# so add its area (block height * block width) when its count is at least k.
# Complexity: Time O(R log R + R^2), Space O(R^2) (independent of the grid size)
def cells_at_least_compressed(rects, k):
    row_set = set()
    column_set = set()
    for r1, c1, r2, c2 in rects:
        row_set.add(r1)
        row_set.add(r2 + 1)
        column_set.add(c1)
        column_set.add(c2 + 1)
    rows = sorted(row_set)
    cols = sorted(column_set)
    row_index = {}
    for i in range(len(rows)):
        row_index[rows[i]] = i
    col_index = {}
    for j in range(len(cols)):
        col_index[cols[j]] = j
    d = []
    for i in range(len(rows) + 1):
        d.append([0] * (len(cols) + 1))
    for r1, c1, r2, c2 in rects:
        a = row_index[r1]
        b = col_index[c1]
        e = row_index[r2 + 1]
        f = col_index[c2 + 1]
```

```
        d[a][b] += 1
        d[a][f] -= 1
        d[e][b] -= 1
        d[e][f] += 1
    total = 0
    column_sums = [0] * (len(cols) + 1)             # the prefix sums down each column
    for i in range(len(rows) - 1):
        running = 0                                 # the prefix sum along this row
        for j in range(len(cols) - 1):
            column_sums[j] += d[i][j]
            running += column_sums[j]
            if running >= k:
                total += (rows[i + 1] - rows[i]) * (cols[j + 1] - cols[j])
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert cells_at_least_compressed(RECTS, 2) == 1          # the same answer as the plain grid
    assert cells_at_least_compressed(RECTS, 1) == 7
    assert cells_at_least_compressed([[0, 0, 10 ** 9, 10 ** 9], [5, 5, 10, 10]], 2) == 36


# ========================================================
# FOLLOW-UP 3: Rectangles are added and the count of a cell is queried online
# ========================================================
# ---- follow-up 3: function ----
# Answer: a 2D Fenwick tree (the difference-array idea as a binary indexed
# tree): add a rectangle by changing its 4 corners and read a cell as a prefix
# sum, both in O(log^2 n).
# Complexity: Time add_rect and get are O(log^2 n), Space O(n^2)
class Fenwick2D:
    def __init__(self, n):
        self.n = n
        self.tree = []
        for i in range(n + 2):
            self.tree.append([0] * (n + 2))

    def add_corner(self, r, c, value):
        i = r + 1
        while i <= self.n + 1:
            j = c + 1
            while j <= self.n + 1:
                self.tree[i][j] += value
                j += j & -j                          # j & -j is the lowest set bit of j
            i += i & -i

    def add_rect(self, r1, c1, r2, c2):
        self.add_corner(r1, c1, 1)
        self.add_corner(r1, c2 + 1, -1)
        self.add_corner(r2 + 1, c1, -1)
        self.add_corner(r2 + 1, c2 + 1, 1)

    def get(self, r, c):
        total = 0
        i = r + 1
        while i > 0:
            j = c + 1
            while j > 0:
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = Fenwick2D(3)
    for rect in RECTS:
        grid.add_rect(rect[0], rect[1], rect[2], rect[3])
    expected = range_add_counts(3, RECTS)
    for r in range(3):
        for c in range(3):
            assert grid.get(r, c) == expected[r][c]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def range_add_counts(n: int, rectangles: list[list[int]]) -> list[list[int]]:
```

### Sample call

```
range_add_counts(1, [])
```

### Test cases

| Input | Expected |
|---|---|
| (1, []) | [[0]] |
| (1, [[0, 0, 0, 0]]) | [[1]] |

**STUDY PATH: question 4 of 53 | topic 2 of 30: Sweep over events in time order | 2 of 3**

Template fit: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes | asked 316 days ago

HARD | IN APP LC 218 | PracHub | Technical Screen | 4 of 53

# Compute city skyline outline

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK ALL TIME 29.8 30 DAYS 0 PRIORITY 145/288

## ABOUT THIS QUESTION

**Topic:** Sweep over events in time order

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #218 The Skyline Problem

**LeetCode match:** LC 218 The Skyline Problem

Posted on PracHub: 2025-12-08 (updated 2026-06-26)

**Original:** https://prachub.com/coding-questions/compute-city-skyline-outline

## QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),
- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

**Input**

- An integer **`n`** — the number of buildings.
- A list of **`n`** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

**Output**

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

**Example**

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):
- From x = 2 to 3, the highest building has height 10.

## QUESTION

- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]` |
| Output | `[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]` |
| Why | `Buildings are [left, right, height]; the outline lists the points where the height changes.` |

| Example 2 | |
|---|---|
| Input | `[[0, 2, 3]]` |
| Output | `[[0, 3], [2, 0]]` |
| Why | `One building: a point at its left edge and one at its right edge, where the height returns to 0.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(buildings):
    xs = set()
    for L, R, H in buildings:
        xs |= {L, R}
    out, prev = [], 0
    for x in sorted(xs):
        h = 0
        for L, R, H in buildings:
            if L <= x < R:
                h = max(h, H)
        if h != prev:
            out.append([x, h])
            prev = h
    return out
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

## OPTIMAL SOLUTION · Sweep line with a max-heap and lazy deletion

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Divide and conquer instead of a heap.**

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

**FOLLOW-UP 2 Coordinates up to 10^9 or buildings of a very large count.**

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

**FOLLOW-UP 3 Output the area under the skyline.**

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute city skyline outline
#
# You are given a list of rectangular buildings in a 2D
# city skyline. Each building is represented by three
# integers [L, R, H]: - L: the x-coordinate of the left
# edge of the building, - R: the x-coordinate of the right
# edge of the building (L < R), - H: the height of the
# building. All buildings stand on a flat horizontal line
# (y = 0) and extend vertically up to height H. Buildings
# may overlap in the x-dimension. Your task is to compute
# the *skyline* formed by these buildings, as seen from a
# distance. The skyline should be represented as a list of
# **key points** [x, height] that describe the outer
# contour of the union of all buildings. A **key point**
# is a point where the height of the skyline changes. The
# output list must satisfy all of the following: 1. The
# list is sorted by increasing x. 2. No two consecutive
# points have the same height (i.e., the height must
# change at each key point). 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer instead of a heap.
# 2. Coordinates up to 10^9 or buildings of a very large
# count.
# 3. Output the area under the skyline.
#
# INTERVIEW NOTES
# The skyline height at x is the tallest building covering
# x, and it can only change at a building's left or right
# edge. So sweep over those x positions in order, keep the
# active buildings in a max-heap by height, and report a
# key point whenever the maximum changes. The details that
# matter are the event order at the same x and removing
# finished buildings lazily.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
import heapq
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Buildings are [left, right, height]. Return the skyline: the "key points"
# [x, height] where the height of the outline changes.
# Idea: a sweep line from left to right with a max-heap of the buildings
# that are active (entries (-height, right edge), push negatives to get a
# max-heap). Start events carry -height so that, at the same x, taller starts
# come first and starts come before ends; end events carry 0. Buildings that
# have ended are removed LAZILY: only when they reach the top of the heap. After
# each event, the top of the heap is the current height; write a key point
# only when it differs from the last one. The ground (height 0) is always in the heap.
# Complexity: Time O(n log n), Space O(n)
def skyline(buildings):
    events = []
    for left, right, height in buildings:
        events.append((left, -height, right))        # a start
        events.append((right, 0, 0))                 # an end
    events.sort()
    points = []
    heap = [(0, float("inf"))]                       # the ground, which never ends
    for x, minus_height, right in events:
        while heap[0][1] <= x:
            heapq.heappop(heap)                      # a building that ended at or before x
        if minus_height != 0:
            heapq.heappush(heap, (minus_height, right))
        top = -heap[0][0]
        if not points or points[len(points) - 1][1] != top:
            points.append([x, top])
    return points


# ---- main question: tests ----
BUILDINGS = [[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]
OUTLINE = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]


def test_main_question():
    assert skyline(BUILDINGS) == OUTLINE
    assert skyline([[0, 2, 3], [2, 5, 3]]) == [[0, 3], [5, 0]]
    assert skyline([]) == []
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [[1, 3], [2, 0]]
```

```
# ========================================================
# FOLLOW-UP 1: Divide and conquer instead of a heap
# ========================================================
# ---- follow-up 1: helper functions ----
def merge_skylines(a, b):
    # sweep two skylines with the current height of each; the height of the
    # merged skyline is the bigger of the two
    merged = []
    i = 0
    j = 0
    height_a = 0
    height_b = 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b) or a[i][0] <= b[j][0])
        if take_a:
            x = a[i][0]
            height_a = a[i][1]
```

```
            i += 1
        else:
            x = b[j][0]
            height_b = b[j][1]
            j += 1
        if i < len(a) and a[i][0] == x:
            continue                                # both skylines change at x: wait for the second
        if j < len(b) and b[j][0] == x:
            continue
        top = height_a
        if height_b > top:
            top = height_b
        if not merged or merged[len(merged) - 1][1] != top:
            merged.append([x, top])
    return merged


# ---- follow-up 1: function ----
# Answer: build the skyline of each half of the buildings and merge the
# two by sweeping both lists with the current height of each. T(n) = 2 T(n/2)
# + O(n) = O(n log n), the same as the heap.
# Complexity: Time O(n log n), Space O(n)
def skyline_dc(buildings):
    if not buildings:
        return []
    if len(buildings) == 1:
        left, right, height = buildings[0]
        return [[left, height], [right, 0]]
```

```
    middle = len(buildings) // 2
    return merge_skylines(skyline_dc(buildings[:middle]), skyline_dc(buildings[middle:]))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert skyline_dc(BUILDINGS) == OUTLINE
    assert skyline_dc([]) == []


# ========================================================
# FOLLOW-UP 2: Coordinates up to 10^9 or buildings of a very large count
# ========================================================
# Answer: the sweep only touches the building edges, never the coordinates
# between them, so 10^9 coordinates cost nothing. For a huge number of buildings,
# split them across machines (divide and conquer) and merge the skylines.


# ========================================================
# FOLLOW-UP 3: Output the area under the skyline
# ========================================================
# ---- follow-up 3: function ----
# Answer: add up height * width over every pair of neighbouring key
# points: one extra pass over the output.
# Complexity: Time O(p) for p skyline points, Space O(1)
def area_under(points):
    area = 0
    for i in range(len(points) - 1):
        x, height = points[i]
        area += height * (points[i + 1][0] - x)
    return area


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert area_under(OUTLINE) == 212               # 10*1 + 15*4 + 12*5 + 10*5 + 8*4
```

```
# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

### Test cases

| Input | Expected |
|---|---|
| (3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]]) | [[2, 10], [3, 15], [7, 12], [12, 0]] |
| (1, [[0, 2, 3]]) | [[0, 3], [2, 0]] |

**STUDY PATH: question 5 of 53 | topic 2 of 30: Sweep over events in time order | 3 of 3**

Template fit: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second | asked 169 days ago

**HARD** | **PracHub** | **Technical Screen** | **5 of 53**

# Compute Turnstile Crossing Times

**POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 200/288**

## ABOUT THIS QUESTION

**Topic:** Sweep over events in time order

**Optimal answer from:** the interview site's own solution

**LeetCode match:** LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-10)

**Original:** https://prachub.com/coding-questions/compute-turnstile-crossing-times

## QUESTION

A turnstile lets one person through per second. events[i] = (arrival time, direction) for person i, with direction 'enter' or 'exit'. When people in both directions wait in the same second: if the turnstile was not used in the previous second, people exiting go first; otherwise the direction used in the previous second goes first again. People of the same direction go in order of arrival (person id). Return the second at which each person crosses.

### Function signature

def turnstile_times(events: list) -> list

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `[(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]` |
| **Output** | `[0, 1, 2, 3]` |
| **Why** | `The idle turnstile lets the exit (person 0) go first at second 0, then the two entering people follow at 1 and 2; person 3 arrives alone at 3.` |

| Example 2 | |
|---|---|
| **Input** | `[(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]` |
| **Output** | `[2, 0, 1, 3]` |
| **Why** | `Idle turnstile: the exits (persons 1 and 2) go first at seconds 0 and 1, then the entering people (persons 0 and 3) at seconds 2 and 3.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        who = pick_by_priority(prev, enter_q,
                               exit_q)
        if who is not None:
            passed[who] = t
            prev = direction_of(who)
        t += 1
    return passed
```

| TIME | SPACE |
|---|---|
| **O(T + n)** where T is the largest timestamp | **O(n)** |

**WHY IT IS SLOW**

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

## OPTIMAL SOLUTION · Event-driven simulation with two queues

| TIME | SPACE |
|---|---|
| **O(n log n)** | **O(n)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

## FOLLOW-UP 2 Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

## FOLLOW-UP 3 Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute Turnstile Crossing Times
#
# A turnstile lets one person through per second.
# events[i] = (arrival time, direction) for person i, with
```

```
# direction 'enter' or 'exit'. When people in both
# directions wait in the same second: if the turnstile was
# not used in the previous second, people exiting go
# first; otherwise the direction used in the previous
# second goes first again. People of the same direction go
# in order of arrival (person id). Return the second at
# which each person crosses.
#
# Signature:
#   def turnstile_times(events: list) -> list
#
# Examples:
# [(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]
#     ->  [0, 1, 2, 3]
# [(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]
#     ->  [2, 0, 1, 3]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority when the turnstile was idle is different
# in this version of the question.
# 2. Timestamps are up to 10^9.
# 3. Two turnstiles or a capacity of two people per
# second.
#
# INTERVIEW NOTES
# A scheduling rule that looks at 'what happened last
# second' is a state machine: write the state down
# (previous direction: enter, exit, none) before coding.
# If time values can be huge, simulate events, not
# seconds: jump the clock to the next arrival when both
# queues are empty, and remember an idle gap resets the
# state.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# events[i] = (arrival time, "enter" or "exit"). One person passes the
# turnstile per second. If it was used in the second before, the same
# direction keeps going when both lines wait; if it was idle, `idle_first`
# wins (exit, by default). Return the second each person goes through.
# Idea: an event-driven simulation with two queues. Sort the arrivals once;
# when both queues are empty jump straight to the next arrival time, and if
# the jump leaves at least one idle second, reset the previous direction to
# "none". Then everyone who has arrived joins a line, and one person goes.
# Complexity: Time O(n log n) (the sort of arrivals; the simulation is O(n)), Space O(n)
def turnstile_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()                                    # by arrival time, then by number
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
```

```
    t = 0
    next_person = 0
    previous = None
    last_used = -10
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]           # nobody is waiting: jump to the next arrival
            if t > last_used + 1:
                previous = None                        # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)    # everyone who has arrived joins a line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                        # both lines: keep the direction used last second
            else:
                want = idle_first                      # after an idle gap the tie-break decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
        result[person] = t
        previous = want
        last_used = t
        t += 1
    return result


# ---- main question: tests ----
def test_main_question():
    both = [(0, "enter"), (0, "exit")]
    assert turnstile_times(both) == [1, 0]                      # idle tie: the exit goes first
    assert turnstile_times(both, "enter") == [0, 1]             # a version where enter wins the idle tie
    assert turnstile_times([(0, "enter"), (0, "enter"), (1, "exit")]) == [0, 1, 2]     # the direction in u
se keeps going
    assert turnstile_times([(0, "enter"), (10, "enter"), (10, "exit")]) == [0, 11, 10]  # an idle second re
sets the state
    assert turnstile_times([(10 ** 9, "enter"), (10 ** 9, "exit")]) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []


# =======================================================
# FOLLOW-UP 1: The priority when the turnstile was idle is different in this version of the question
# =======================================================
# Answer: keep the same loop and make the idle-state winner a parameter
# (`idle_first`). Only the line that picks the direction changes, which is why
# you should write the state machine first.


# =======================================================
# FOLLOW-UP 2: Timestamps are up to 10^9
# =======================================================
# Answer: no problem: the clock jumps over empty stretches, one loop step per person.


# =======================================================
# FOLLOW-UP 3: Two turnstiles or a capacity of two people per second
# =======================================================
# Answer: let two people go through per second: take up to two from the
# queues in each step (same direction rule), or keep one "previous direction"
# per turnstile.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(time: list[int], direction: list[int]) -> list[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

### Test cases

| Input | Expected |
|---|---|
| [<br>[<br>0,<br>0,<br>1,<br>5<br>],<br>[<br>0,<br>1,<br>1,<br>0<br>]<br>] | [<br>2,<br>0,<br>1,<br>5<br>] |
| [<br>[<br>0,<br>0,<br>0<br>],<br>[<br>0,<br>1,<br>0<br>]<br>] | [<br>1,<br>0,<br>2<br>] |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 3 OF 30 |

# Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 6. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree **PracHub 2026-09-16**

H 7. Delete an Entire Binary Tree **ColorStack 2025-09-29**

CODE TEMPLATE | GROUP 3 OF 30 | 2 QUESTIONS, ONE PATTERN

# Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

## The one idea these questions share

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def solve(root):
    best = 0

    def dfs(node, depth=0):
        nonlocal best
        if node is None:
            return 0                          # CHANGE 1: what an empty tree reports
        left = dfs(node.left, depth + 1)
        right = dfs(node.right, depth + 1)
        best = max(best, left + right)        # CHANGE 2: update the answer at this node
        return 1 + max(left, right)           # CHANGE 3: what this node reports upward

    dfs(root)
    return best
```

## 2. Helper functions that usually get written next to it

```
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def levels(root):                             # BFS by level: height, right-side view, zigzag
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c)
    return out


def preorder(root):                           # iterative: push right first, then left
    stack = [root] if root else []
    while stack:
        n = stack.pop()
        yield n.val
        stack.extend(c for c in (n.right, n.left) if c)
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **6** | Repeatedly Delete Leaves Matching a Target Value From a Binary Tree | set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value |
| **7** | Delete an Entire Binary Tree | post-order: free the children first, then the node itself |

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 6 of 53 | topic 3 of 30: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 1 of 2**

Template fit: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value | asked 34 days ago

HARD | IN APP LC 1325 | PracHub | Technical Screen | 6 of 53

# Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 10.6 | 30 DAYS 0 | PRIORITY 188/288

## ABOUT THIS QUESTION

**Topic:** Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #1325 Delete Leaves With a Given Value

**LeetCode match:** LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

**Original:** https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

## QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

### Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

### Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

### Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

### Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

### Examples

## QUESTION

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `tree = [1, 2, 3, 2, null, 2, 4], target = 2 and tree = [1, 3, 3, 3, 2], target = 3` |
| **Output** | `values(remove_leaves(tree([1, 2, 3, 2, None, 2, 4]), 2)[0]) -> [1, null, 3, null, 4]`<br>`values(remove_leaves(tree([1, 3, 3, 3, 2]), 3)[0]) -> [1, 3, null, null, 2]` |
| **Why** | `remove_leaves returns (the new root, how many leaves were removed). Deleting a leaf can turn its parent into a leaf, which is deleted too if it also equals the target: in the first tree the 2s go and the root's left branch disappears entirely.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

| TIME | SPACE |
|---|---|
| O(n * h) | O(h) |

WHY IT IS SLOW

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

## OPTIMAL SOLUTION · One post-order pass

| TIME | SPACE |
|---|---|
| O(n) | O(h) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?**

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

**FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.**

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

**FOLLOW-UP 3 Return how many nodes were deleted.**

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list with None for a missing child: [1, 2, 3, 2, None, 2, 4]
    if not values:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


def tree_to_list(root):
    # the same encoding, with the trailing Nones removed
    result = []
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            result.append(None)
        else:
            result.append(node.val)
            queue.append(node.left)
            queue.append(node.right)
    while result and result[-1] is None:
        result.pop()
```

```
    return result


def remove_leaves(root, target):
    # post-order: handle both children first, then look at the node itself.
    # Returns (the new root, how many nodes were deleted) -- follow-up 3.
    deleted = 0

    def visit(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = visit(node.left)
        node.right = visit(node.right)
```

```
        if node.left is None and node.right is None and node.val == target:
            deleted += 1                          # a leaf with the target value disappears
            return None
        return node

    return visit(root), deleted


# ---- main question: main function ----
# Idea: one post-order pass. Process both children first, then the node:
# if it has become a leaf and its value is the target, it is removed
# (its parent then sees None). That is exactly how a deleted leaf can
# turn its parent into a new leaf.
# Complexity: Time O(n), Space O(n) (the recursion is O(h))
def remove_target_leaves(tree, target):
    root, deleted = remove_leaves(build_tree(tree), target)
    return tree_to_list(root)


# ---- main question: tests ----
def test_main_question():
    assert remove_target_leaves([1, 2, 3, 2, None, 2, 4], 2) == [1, None, 3, None, 4]
    assert remove_target_leaves([1, 3, 3, 3, 2], 3) == [1, 3, None, None, 2]
    assert remove_target_leaves([1, 2, None, 2, None, 2], 2) == [1]
    assert remove_target_leaves([5], 5) == []
    assert remove_target_leaves([], 5) == []


# ========================================================
# FOLLOW-UP 1: The tree is a 100,000-node chain. How do you avoid a stack overflow?
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: collect the nodes in pre-order with your own stack and remember
# each node's parent; then walk that list BACKWARDS (children before
# parents) and unlink every node that is a matching leaf. That keeps the
# post-order guarantee without recursion.
# Complexity: Time O(n), Space O(n)
def remove_leaves_iterative(root, target):
    if root is None:
        return None, 0
    parent = {root: None}
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    for node in reversed(order):                  # children are handled before their parents
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = TreeNode(5)
    node = chain
    for i in range(99999):
        node.left = TreeNode(5)
        node = node.left
    new_root, deleted = remove_leaves_iterative(chain, 5)
    assert new_root is None and deleted == 100000
    root, deleted = remove_leaves_iterative(build_tree([1, 2, 3, 2, None, 2, 4]), 2)
    assert tree_to_list(root) == [1, None, 3, None, 4] and deleted == 3
```

```
# ============================================================
# FOLLOW-UP 2: Delete any node whose whole subtree sums to zero, or whose value matches a predicate
# ============================================================
# Answer: the same post-order pass: after both children are handled,
# decide with the new rule (the sum of the node and its remaining
# children, or a yes/no function of the value) instead of "is it a
# leaf with the target value".


# ============================================================
# FOLLOW-UP 3: Return how many nodes were deleted
# ============================================================
# Answer: remove_leaves above already returns the count next to the new root.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    root, deleted = remove_leaves(build_tree([1, 2, 3, 2, None, 2, 4]), 2)
    assert deleted == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]:
```

### Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

### Test cases

| Input | Expected |
|---|---|
| `([5, 7, 7, 7, 3, None, 7], 7)` | `[5, 7, None, None, 3]` |
| `([3, 3, 1], 3)` | `[3, None, 1]` |

**STUDY PATH: question 7 of 53 | topic 3 of 30: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 2 of 2**

Template fit: post-order: free the children first, then the node itself | asked 386 days ago | frequency Single report

HARD | IN APP LC 145 | ColorStack | technical interview | 7 of 53

# Delete an Entire Binary Tree

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 204/288

## ABOUT THIS QUESTION

**Topic:** Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

**Optimal answer from:** your app's interview approach (LeetCode #145)

**LeetCode match:** LC 145 Binary Tree Postorder Traversal (free every node)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

## QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

**Function signature**

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

### EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `root = 1 / \ 2 3` |
| **Output** | `[2, 3, 1]` |
| **Why** | `Post-order frees the left child, then the right child, then the root.` |

| Example 2 | |
|---|---|
| **Input** | `root = None` |
| **Output** | `[]` |
| **Why** | `An empty tree frees nothing.` |

## QUESTION

| Example 3 | |
|---|---|
| **Input** | `root = 1 / 2 / 3` |
| **Output** | `[3, 2, 1]` |
| **Why** | `A left-leaning chain frees deepest-first.` |

CONSTRAINTS

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_post_order(root):
    out, stack = [], [root] if root else []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return out[::-1]
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

WHY IT IS SLOW

It stores every node twice (the work stack and the output).

## OPTIMAL SOLUTION · Recursive post-order

| TIME | SPACE |
|---|---|
| O(n) | O(h) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Do it with O(1) extra space.

Rotate instead of recursing: while the root has a left child, rotate it up (the left child becomes the root and the old root becomes its right child). When the root has no left child, free it and move to its right child. Every node is freed once and no stack is used; the order is no longer post-order.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Delete an Entire Binary Tree
#
# You are given the root of a binary tree (not a BST);
# each node has only left and right child pointers. You
```

```
# must "free" every node so that after the call the tree
# is empty. Because a node must not be freed before its
# children, the correct freeing order is a post-order
# traversal. Implement delete_tree(root) that frees all
# nodes and returns the list of node values in the exact
# order they are freed (post-order: left subtree, right
# subtree, then node). Function signature: from typing
# import Optional class TreeNode: def __init__(self, val:
# int = 0, left: "Optional[TreeNode]" = None, right:
# "Optional[TreeNode]" = None) -> None: self.val = val
# self.left = left self.right = right def
# delete_tree(root: Optional[TreeNode]) -> list[int]:
# """Free every node and return the values in the order
# freed (post-order)."""  ... Follow-up (as reported):
# Recursive DFS uses O(h) stack space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with O(1) extra space.
#
# INTERVIEW NOTES
# A node may only be freed after its children, which is
# exactly post-order: left subtree, right subtree, then
# the node. Recursion uses O(h) stack; two stacks or an
# explicit stack avoid recursion. A constant-space version
# rotates left children up until the root has no left
# child, frees the root, and moves right.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
```

```
        self.left = left
        self.right = right


def build_tree(shape):
    # shape is None or (value, left shape, right shape)
    if shape is None:
        return None
    value, left, right = shape
    return TreeNode(value, build_tree(left), build_tree(right))


# ---- main question: main function ----
# Idea: a node may only be freed AFTER its children, so free the left
# subtree, then the right subtree, then the node itself (post-order).
# "Freeing" a node clears its two pointers; the values are returned in
# the order they were freed.
# Complexity: Time O(n), Space O(h) for the recursion (plus the O(n) list of freed values)
def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
```

```
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = None
        node.right = None

    free(root)
    return freed


# ---- main question: tests ----
def test_main_question():
    assert delete_tree(build_tree((1, (2, None, None), (3, None, None)))) == [2, 3, 1]
    assert delete_tree(None) == []
    assert delete_tree(build_tree((1, (2, (3, None, None), None), None))) == [3, 2, 1]


# ========================================================
# FOLLOW-UP 1: Do it with O(1) extra space
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: rotate instead of recursing. While the root has a left child,
# rotate it up (the left child becomes the root and the old root becomes
# its right child). When the root has no left child, free it and move to
# its right child. No stack is used (the order of freeing is no longer
# post-order, but every node is freed once).
# Complexity: Time O(n), Space O(1) extra (plus the O(n) list of freed values)
def delete_tree_constant_space(root):
    freed = []
    while root:
        if root.left:
            left = root.left
            root.left = left.right
            left.right = root
            root = left
        else:
            following = root.right
            freed.append(root.val)
            root.right = None
            root = following
    return freed


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = build_tree((1, (2, (4, None, None), (5, None, None)), (3, None, None)))
    assert sorted(delete_tree_constant_space(tree)) == [1, 2, 3, 4, 5]
    assert delete_tree_constant_space(None) == []


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 4 OF 30 |

# A heap of what is available: pop the best, put back the changed one

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**A heap of what is available: pop the best, put back the changed one | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 8. Huffman Tree Construction 1Point3Acres 2026-06-01

CODE TEMPLATE | GROUP 4 OF 30 | 1 QUESTIONS, ONE PATTERN

# A heap of what is available: pop the best, put back the changed one

## The one idea these questions share

The heap holds the things you can choose from right now: machines and when they are free, rooms, lockers, the biggest remaining count, the two smallest weights. Each step pops the best choice, uses it, and pushes it back changed (or a combined item).

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def run_jobs(jobs, workers):
    free = [0] * workers                        # CHANGE 1: what is available (times, ids, counts)
    heapq.heapify(free)
    finish = 0
    for ready, duration in sorted(jobs):
        start = max(ready, heapq.heappop(free))          # CHANGE 2: take the best available
        heapq.heappush(free, start + duration)           # CHANGE 3: put it back, changed
        finish = max(finish, start + duration)
    return finish
```

## 2. Helper functions that usually get written next to it

```
def pop_two_push_sum(weights):              # the Huffman step, also the generic "combine two best"
    heap = list(weights)
    heapq.heapify(heap)
    total = 0
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, a + b)
        total += a + b
    return total


class MaxHeap:                              # heapq only has a min-heap: negate on the way in and out
    def __init__(self):
        self.h = []

    def push(self, x):
        heapq.heappush(self.h, -x)

    def pop(self):
        return -heapq.heappop(self.h)
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **8** | Huffman Tree Construction | pop the two smallest weights, push their sum as a new node; repeat until one node remains |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 8 of 53 | topic 4 of 30: A heap of what is available: pop the best, put back the changed one | 1 of 1**

Template fit: pop the two smallest weights, push their sum as a new node; repeat until one node remains | asked 141 days ago | frequency Low

**HARD | 1Point3Acres | onsite-coding | 8 of 53**

# Huffman Tree Construction

**ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 249/288**

## ABOUT THIS QUESTION

**Topic:** A heap of what is available: pop the best, put back the changed one

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

## QUESTION

Given symbol frequencies, build the Huffman code: repeatedly merge the two least frequent items into one until a single tree is left; the path from the root gives each symbol its bit string. Return the code of every symbol. To make the answer unique, the codes are assigned canonically: shorter codes first, then alphabetical order.

**Function signature**

def huffman_codes(freq: dict) -> dict

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `{'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}` |
| **Output** | `{'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a': '1110', 'b': '1111'}` |
| **Why** | `The frequent symbol f gets a 1-bit code; the rare ones get longer codes.` |

| Example 2 | |
|---|---|
| **Input** | `{'x': 3, 'y': 8}` |
| **Output** | `{'x': '0', 'y': '1'}` |
| **Why** | `Two symbols always get one bit each.` |

| Example 3 | |
|---|---|
| **Input** | `{'z': 10}` |
| **Output** | `{'z': '0'}` |
| **Why** | `A single symbol still needs a code of one bit.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

## OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

## FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

## FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

## FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Huffman Tree Construction
#
```

```
# Given symbol frequencies, build the Huffman code:
# repeatedly merge the two least frequent items into one
# until a single tree is left; the path from the root
# gives each symbol its bit string. Return the code of
# every symbol. To make the answer unique, the codes are
# assigned canonically: shorter codes first, then
# alphabetical order.
#
# Signature:
#   def huffman_codes(freq: dict) -> dict
#
# Examples:
# {'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}  ->
#     {'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a':
#     '1110', 'b': '1111'}
# {'x': 3, 'y': 8}  ->  {'x': '0', 'y': '1'}
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def huffman_lengths(freq):
    # the code length of every symbol = its depth in the Huffman tree.
    # Idea: put every symbol in a min-heap by frequency; repeatedly take the
    # TWO lightest nodes and merge them under a new node whose weight is the
    # sum; every symbol below the merged node gets one level deeper.
    if not freq:
        return {}
    symbols = sorted(freq)
    if len(symbols) == 1:
        return {symbols[0]: 1}                      # a lone symbol still needs one bit
    heap = []
    for i in range(len(symbols)):
        heap.append((freq[symbols[i]], i, [symbols[i]]))   # (weight, tie-break, the symbols below)
    heapq.heapify(heap)
    depth = {}
    for symbol in symbols:
        depth[symbol] = 0
    tie = len(heap)
```

```
    while len(heap) > 1:
        weight1, t1, below1 = heapq.heappop(heap)
        weight2, t2, below2 = heapq.heappop(heap)
        for symbol in below1 + below2:
            depth[symbol] += 1
        heapq.heappush(heap, (weight1 + weight2, tie, below1 + below2))
        tie += 1
    return depth


def to_binary(number, length):
    # the number written in binary with exactly `length` digits
    digits = ""
    for i in range(length):
        digits = str(number % 2) + digits
        number = number // 2
    return digits


def canonical_codes(lengths):
    # CANONICAL codes (follow-up 2 and 4): sort the symbols by (length,
    # symbol) and give them consecutive binary numbers, shifting left when
    # the length grows. A decoder can rebuild all codes from the lengths alone
```

```
    # (this is how DEFLATE stores its tables).
    ordered = []
    for symbol in lengths:
        ordered.append((lengths[symbol], symbol))
    ordered.sort()
    code = 0
    previous_length = 0
    codes = {}
    for length, symbol in ordered:
        code = code * 2 ** (length - previous_length)   # a longer length shifts the code left
        codes[symbol] = to_binary(code, length)
        code += 1
        previous_length = length
    return codes


# ---- main question: main function ----
# Idea: the lengths come from the min-heap merging above; the actual
# codes are then assigned in the canonical way.
# Complexity: Time O(n log n) for n symbols, Space O(n)
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# ---- main question: tests ----
FREQ = {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}


def decode(bits, codes):
    # read bits until they match a code (the codes are prefix-free)
    symbol_of = {}
    for symbol in codes:
        symbol_of[codes[symbol]] = symbol
    text = ""
    current = ""
    for bit in bits:
        current += bit
        if current in symbol_of:
            text += symbol_of[current]
            current = ""
    return text
```

```
def total_cost(freq, lengths):
    cost = 0
    for symbol in freq:
        cost += freq[symbol] * lengths[symbol]
    return cost


def test_main_question():
    lengths = huffman_lengths(FREQ)
    assert lengths["f"] == 1                        # the most frequent symbol is next to the root
    assert total_cost(FREQ, lengths) == 224         # the cost of the optimal tree
    codes = huffman_codes(FREQ)
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)    # prefix-free: no code starts another code
    bits = ""
    for ch in "abfadc":
        bits += codes[ch]
    assert decode(bits, codes) == "abfadc"
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}


# ============================================================
# FOLLOW-UP 1: Why is the greedy choice optimal?
# ============================================================
# Answer: the two LIGHTEST symbols can always be siblings at the deepest
# level of some optimal tree (swapping a heavier symbol with a lighter one
# at a shallower level never makes the cost bigger). Merging them leaves a
# smaller problem of the same kind, so repeating the choice is optimal.


# ============================================================
# FOLLOW-UP 2: How do you make the code canonical, and why would you?
# ============================================================


# ============================================================
# FOLLOW-UP 4: Make the codes canonical so only the code lengths need to be stored
# ============================================================
# Answer: canonical_codes above: keep only each symbol's code LENGTH, sort
# the symbols by (length, symbol) and give consecutive binary numbers. The
# decoder needs just the lengths instead of the whole tree.


# ============================================================
# FOLLOW-UP 3: Time complexity if symbols arrive already sorted?
```

```
# ============================================================
# ---- follow-up 3: function ----
# Answer: use TWO queues: one holds the sorted leaves, the other the merged
# nodes (which are created in non-decreasing weight order). Always take the
# smaller front of the two queues. That is O(n) after the sort.
# Complexity: Time O(n) merges after the sort (O(n^2) here because the symbols below each merge are updated
 one by one), Space O(n)
def lengths_two_queues(sorted_items):
    leaves = deque()
    depth = {}
    for symbol, weight in sorted_items:
        leaves.append((weight, [symbol]))
        depth[symbol] = 0
    merged = deque()
```

```
    def take_smaller():
        if not merged or (leaves and leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        weight1, below1 = take_smaller()
        weight2, below2 = take_smaller()
        for symbol in below1 + below2:
            depth[symbol] += 1
        merged.append((weight1 + weight2, below1 + below2))
    return depth


# ---- follow-up 3: tests ----
def test_follow_up_3():
    items = sorted(FREQ.items(), key=lambda pair: pair[1])      # the symbols ordered by weight
    assert total_cost(FREQ, lengths_two_queues(items)) == 224


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 5 OF 30 |

# Recurrence on a position or a range (memo or table)

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**3**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Recurrence on a position or a range (memo or table) | 3 questions | tap a title to jump to it**

**READ FIRST: the code template for these 3 questions (tap here) >**


| | | |
|---|---|---|
| H | 9. Edit distance | Exponent no date |
| H | 10. Count Perfect Wake Numbers up to N | 1Point3Acres 2026-05-07 |
| H | 11. Word Break: All Sentences | ColorStack 2026-01-28 |

CODE TEMPLATE | GROUP 5 OF 30 | 3 QUESTIONS, ONE PATTERN

# Recurrence on a position or a range (memo or table)

## The one idea these questions share

Define best(i): the answer for everything from position i onward (or best(i, j) for a range, or a table over two sequences). Write what an empty or finished state returns, then combine the answers of the states you can jump to. Cache it, or fill a table in the right order.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def solve(seq, valid):
    n = len(seq)

    @lru_cache(maxsize=None)
    def best(i):                          # CHANGE 1: i is a position, or (i, j) for a range
        if i == n:
            return 1                      # CHANGE 2: what a finished state returns
        total = 0
        for j in range(i + 1, n + 1):     # CHANGE 3: which later states can you jump to
            if valid(seq[i:j]):           # CHANGE 4: can this piece be used
                total += best(j)          # CHANGE 5: combine: sum, max, min, or build lists
        return total

    return best(0)
```

## 2. Helper functions that usually get written next to it

```
def edit_distance(a, b):                    # the two-row table most alignment questions use
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(prev[j - 1] if ca == cb else 1 + min(prev[j - 1], prev[j], cur[j - 1]))
        prev = cur
    return prev[-1]
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **9** | Edit distance | table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours |
| **10** | Count Perfect Wake Numbers up to N | digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag |
| **11** | Word Break: All Sentences | best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end) |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 9 of 53 | topic 5 of 30: Recurrence on a position or a range (memo or table) | 1 of 3**

Template fit: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours | no date

HARD | IN APP LC 127, 72 | Exponent | 9 of 53

# Edit distance

ASKED (ANY CO.) ~3 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 46 | 30 DAYS 0 | PRIORITY 90/288

## ABOUT THIS QUESTION

**Topic:** Recurrence on a position or a range (memo or table)

**Optimal answer from:** your app's interview approach (LeetCode #127)

**LeetCode match:** LC 127 Word Ladder (statement) / LC 72 Edit Distance (title)

Last asked (any company): 3 months ago, about 2026-07 (Exponent shows only a rough age) | +612 interviews

**Original:** https://www.aced.io/questions/3891/edit-distance

## QUESTION

Given two words a and b, return the edit distance: the smallest number of single-letter operations (insert a letter, delete a letter, or replace a letter) that turn a into b.

**Function signature**

def edit_distance(a: str, b: str) -> (int, list)

**(the distance, the whole DP**

table)

EXAMPLES

| Example 1 | |
|---|---|
| Input | `a = 'horse', b = 'ros'` |
| Output | `edit_distance('horse', 'ros')[0] -> 3` |
| Why | `horse -> rorse (replace h) -> rose (delete r) -> ros (delete e): 3 operations.` |

| Example 2 | |
|---|---|
| Input | `a = 'intention', b = 'execution'` |
| Output | `edit_distance('intention', 'execution')[0] -> 5` |
| Why | `intention -> execution takes 5 operations.` |

| Example 3 | |
|---|---|
| Input | `a = '', b = 'abc'` |
| Output | `edit_distance('', 'abc')[0] -> 3` |
| Why | `From the empty word, insert every letter.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def dist(i, j):
    if i == 0:
        return j
    if j == 0:
        return i
    if a[i - 1] == b[j - 1]:
        return dist(i - 1, j - 1)
    replace = dist(i - 1, j - 1)
    delete = dist(i - 1, j)
    insert = dist(i, j - 1)
    return 1 + min(replace, delete, insert)
```

| TIME | SPACE |
|---|---|
| O(3^(m + n)) | O(m + n) |

**WHY IT IS SLOW**

Three calls per step and the same (i, j) pair is solved many times: about 3^(m + n). A table of all (i, j) values is O(m * n), with a rolling row for O(min(m, n)) memory.

## OPTIMAL SOLUTION · DP table with a rolling row

| TIME | SPACE |
|---|---|
| O(m * n) | O(min(m, n)) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the actual operations.

Keep the full table and walk back from dp[m][n]: if the characters match step diagonally; otherwise take the neighbour whose value is one smaller and record replace, delete or insert. O(m * n) memory is needed for the walk back.

## FOLLOW-UP 2 Are the strings exactly one edit apart (LeetCode 161)?

No table needed: if the lengths differ by more than one answer false; scan to the first mismatch and compare the remainders (equal lengths: skip one char in each; otherwise skip one in the longer). O(n), O(1).

## FOLLOW-UP 3 Operations have different costs, or only a small edit distance is of interest (at most k).

Weighted costs just change the numbers added in the recurrence. For 'at most k' only compute a band of width 2k + 1 around the diagonal: O(k * n).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Edit distance
#
# Given two words a and b, return the edit distance: the
# smallest number of single-letter operations (insert a
# letter, delete a letter, or replace a letter) that turn
# a into b.
#
# Signature:
```

```
#   def edit_distance(a: str, b: str) -> (int, list)       # (the distance, the whole DP table)
#
# Examples:
# a = 'horse', b = 'ros'  ->  edit_distance('horse',
#     'ros')[0]  ->  3
# a = 'intention', b = 'execution'  ->
#     edit_distance('intention', 'execution')[0]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual operations.
# 2. Are the strings exactly one edit apart (LeetCode
# 161)?
# 3. Operations have different costs, or only a small edit
# distance is of interest (at most k).
#
# INTERVIEW NOTES
# Compare the last characters. If they match, nothing is
# lost: the cost is the cost of the shorter prefixes. If
# not, one of three operations was used, so take the
# cheapest of replace, delete and insert, each followed by
# the cost of a smaller problem. The same smaller problems
# repeat, so tabulate them in a grid dp[i][j] = cost of
# turning the first i characters of a into the first j of
# b; each cell needs only its left, top and top-left
# neighbours.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def smallest(x, y, z):
    result = x
    if y < result:
        result = y
    if z < result:
        result = z
    return result


# ---- main question: main function ----
# Idea: dp[i][j] = the fewest edits that turn the first i letters of a
# into the first j letters of b. With equal last letters it is
# dp[i-1][j-1]; otherwise 1 + the best of replace (dp[i-1][j-1]),
# delete (dp[i-1][j]) and insert (dp[i][j-1]). The first row and column
# are plain deletes / inserts. Returns (the distance, the whole table).
# Complexity: Time O(m * n), Space O(m * n)
def edit_distance(a, b):
    m = len(a)
    n = len(b)
    dp = []
    for i in range(m + 1):
        dp.append([0] * (n + 1))
    for i in range(m + 1):
        dp[i][0] = i                              # delete i letters
```

```
    for j in range(n + 1):
        dp[0][j] = j                                # insert j letters
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]         # same letter: no edit
            else:
                dp[i][j] = 1 + smallest(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
    return dp[m][n], dp


# RELATED: are the words at most k edits apart?
# Complexity: Time O(m * n), Space O(m * n)
def within_k(a, b, k):
    if abs(len(a) - len(b)) > k:
        return False                                # lengths alone need more than k edits
    return edit_distance(a, b)[0] <= k
```

```
# RELATED: the word-ladder version: change one letter at a time and every
# word must be in the list; BFS over the words
# Complexity: Time O(N * L^2 * 26) for N words of length L, Space O(N * L)
def ladder_length(begin, end, words):
    words = set(words)
    if end not in words:
        return 0
    queue = deque([(begin, 1)])
    seen = {begin}
    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                following = word[:i] + c + word[i + 1:]
                if following in words and following not in seen:
                    seen.add(following)
                    queue.append((following, steps + 1))
    return 0


# ---- main question: tests ----
def test_main_question():
    assert edit_distance("horse", "ros")[0] == 3
    assert edit_distance("", "abc")[0] == 3
    assert edit_distance("same", "same")[0] == 0
    assert within_k("kitten", "sitting", 3)
    assert not within_k("kitten", "sitting", 2)
    words = ["hot", "dot", "dog", "lot", "log", "cog"]
    assert ladder_length("hit", "cog", words + ["hit"]) == 5
    assert ladder_length("hit", "cog", ["hot", "dot"]) == 0


# ========================================================
# FOLLOW-UP 1: Return the actual operations
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the whole table and walk back from dp[m][n]: if the
# letters match, step diagonally; otherwise take the neighbour that is one
# smaller and write down the replace, delete or insert. The walk back needs
# the whole table: O(m * n) memory.
# Complexity: Time O(m * n), Space O(m * n)
def edit_operations(a, b):
```

```
    distance, dp = edit_distance(a, b)
    i = len(a)
    j = len(b)
    operations = []
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            operations.append("replace " + a[i - 1] + " with " + b[j - 1])
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            operations.append("delete " + a[i - 1])
            i -= 1
        else:
            operations.append("insert " + b[j - 1])
            j -= 1
    operations.reverse()
    return operations


# ---- follow-up 1: tests ----
def test_follow_up_1():
    operations = edit_operations("horse", "ros")
    assert len(operations) == 3
    assert edit_operations("same", "same") == []


# ========================================================
# FOLLOW-UP 2: Are the strings exactly one edit apart (LeetCode 161)?
# ========================================================
# ---- follow-up 2: function ----
# Answer: no table needed. If the lengths differ by more than one, say
# no. Walk to the first place where the letters differ and compare the rest:
# equal lengths: skip one letter in each; otherwise skip one in the longer
# one. O(n) time, O(1) space.
# Complexity: Time O(n), Space O(n)
def one_edit_apart(a, b):
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) > len(b):
        a, b = b, a                                # a is now the shorter (or equal) one
    for i in range(len(a)):
        if a[i] != b[i]:
            if len(a) == len(b):
                return a[i + 1:] == b[i + 1:]      # one replaced letter
            return a[i:] == b[i + 1:]              # one extra letter in b
    return len(a) != len(b)                        # no difference: only an extra letter at the end counts


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert one_edit_apart("cat", "cut")
    assert one_edit_apart("cat", "cats")
    assert not one_edit_apart("cat", "cat")
    assert not one_edit_apart("cat", "dog")


# ========================================================
# FOLLOW-UP 3: Operations have different costs, or only a small edit distance is of interest (at most k)
# ========================================================
# Answer: different costs: use the cost of each operation instead of 1 in
# the formula. At most k: only cells within k of the diagonal can matter,
# so fill a band of width 2k + 1: O(n * k).
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## OFFICIAL PAGE TEXT

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples

```
beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog", "cog"]
output: 4 # hit -> hot -> dot -> dog -> cog

beginWord = 'word'
endWord = 'word'
wordList = ['word', 'ward']
output: 0

beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog"]
output: None  # because no 'cog' in list
```

Our solution tackles the problem by first constructing a graph that represents all possible transformations between words in the list, where an edge exists between two words if they differ by exactly one character (i.e., they are "one edit distance" apart). This is done by iterating over the list of words and comparing each pair to determine if they meet the one edit distance condition. If they do, an edge is added between them in the graph. After constructing the graph, we use a breadth-first search (BFS) approach to find the shortest path from the start_word to the end_word. We initiate a queue that starts with the start_word and its associated edit distance (initially 0). As we traverse the graph, for each word, we check if it's the end_word. If so, we return the current edit distance incremented by one. If not, we add the word's neighbors to the queue for further exploration, ensuring that we don't revisit previously seen words. The BFS guarantees that the first time we reach the end_word, it is via the shortest path, and we can stop the search early. Time Complexity: The time complexity of the solution is O(n^2 * m), where n is the number of words and m is the length of each word. This complexity arises from the double loop that compares each word to every other word (O(n^2)), and for each comparison, we check if the words differ by exactly one character (O(m)). Space Complexity: The space complexity is O(n^2), where n is the number of words. This space is required to store the graph, where each word can potentially have edges to all other words. Additionally, the BFS queue and the visited set require O(n) space in the worst case.In this video, Thomas, Google SWE, answers the Edit Distance coding interview question.No answers contributed by the community yet.

**STUDY PATH: question 10 of 53 | topic 5 of 30: Recurrence on a position or a range (memo or table) | 2 of 3**

Template fit: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag | asked 166 days ago | frequency Single report

**HARD | 1Point3Acres | phone-screen, onsite-coding | 10 of 53**

# Count Perfect Wake Numbers up to N

**ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 247/288**

## ABOUT THIS QUESTION

**Topic:** Recurrence on a position or a range (memo or table)

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

## QUESTION

A number is perfect if its digits are all different, it contains no 0, and no digit is a valley (a digit strictly smaller than both of its neighbours). Part 1: decide whether a number is perfect. Part 2: count the perfect numbers from 1 to N.

### Function signature

def is_perfect(n: int) -> bool

def count_up_to(N: int) -> int

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `196` |
| **Output** | `true` |
| **Why** | `Digits 1, 9, 6 are different, no zero, and 9 is a peak, not a valley.` |

| Example 2 | |
|---|---|
| **Input** | `1546` |
| **Output** | `false` |
| **Why** | `The 4 in 1546 sits between 5 and 6, both larger: a valley.` |

| Example 3 | |
|---|---|
| **Input** | `99` |
| **Output** | `81` |
| **Why** | `Every number 1-9 and every two-digit number with different non-zero digits: 9 + 72 = 81.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

| TIME | SPACE |
|---|---|
| O(N * d) | O(1) |

**WHY IT IS SLOW**

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

## OPTIMAL SOLUTION · Part 1: test a single number

| TIME | SPACE |
|---|---|
| O(d) | O(d) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

| TIME | SPACE |
|---|---|
| O(d * 2^9 * 9 * 9) | O(2^9 * 9) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

## FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

## FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

### FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

### FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

### FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Perfect Wake Numbers up to N
#
# A number is perfect if its digits are all different, it
# contains no 0, and no digit is a valley (a digit
# strictly smaller than both of its neighbours). Part 1:
# decide whether a number is perfect. Part 2: count the
# perfect numbers from 1 to N.
#
# Signature:
#   def is_perfect(n: int) -> bool
#   def count_up_to(N: int) -> int
#
# Examples:
# 196  ->  true
# 1546  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the DP only need a 'falling' flag instead of
# the full history?
# 2. How do you count the numbers in a range [A, B]?
# 3. What is the closed form without a DP?
# 4. Is there a closed form for k-digit numbers?
# 5. Count perfect numbers in a range [A, B].
#
# INTERVIEW NOTES
# Distinct digits with no zero and no valley means the
# digits go up and then down (they can only change
# direction once). That structure is both a closed form
# for each length and a state for a digit DP: position,
# which digits are used, the last digit, and whether the
# digits have started falling. The tight flag handles the
# bound N.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def digits_of(n):
```

```
    # the digits of n as a list of numbers: 1203 -> [1, 2, 0, 3]
    result = []
```

```
    for ch in str(n):
        result.append(int(ch))
    return result


def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


def with_k_digits(k):
    # how many perfect numbers have exactly k digits (closed form, follow-up 4):
    # pick k different digits out of 1..9; the biggest is the peak and every
    # other digit goes left or right of it in sorted order: choose(9, k) * 2^(k-1)
    if k < 1:
        return 0
    return choose(9, k) * 2 ** (k - 1)


# ---- main question: main functions ----
# Idea: a number is perfect when it has no zero, no repeated digit, and
# no "valley": no digit smaller than both of its neighbours.
# Complexity: Time O(d) for d digits, Space O(d)
def is_perfect(n):
    d = digits_of(n)
    seen = set()
    for digit in d:
        if digit == 0 or digit in seen:
            return False
        seen.add(digit)
    for i in range(1, len(d) - 1):
        if d[i - 1] > d[i] and d[i] < d[i + 1]:
            return False
    return True


# PART 2: how many perfect numbers are there from 1 to n?
# Idea (digit DP): numbers with fewer digits come from the closed form.
# For numbers with the SAME number of digits, build them digit by digit
# from the left, remembering: which digits are used (a bitmask), the last
```

```
# digit, whether we already went DOWN (`falling`: going up after that
# would be a valley), and whether we are still equal to the limit
# (`tight`: then the next digit may not exceed n's digit).
# Complexity: Time O(d * 2^10 * 10 * 10) states (a fixed amount for decimal digits), Space the same
def count_up_to(n):
    if n < 1:
        return 0
    digits = digits_of(n)
    size = len(digits)
    total = 0
    for k in range(1, size):
        total += with_k_digits(k)
    memo = {}

    def count_from(position, used, last, falling, tight):
        if position == size:
            return 1
        key = (position, used, last, falling, tight)
        if key in memo:
            return memo[key]
        limit = 9
```

```
        if tight:
            limit = digits[position]
        count = 0
        for d in range(1, limit + 1):
            if (used >> d) % 2 == 1:
                continue                            # digit already used
            now_falling = falling
            if position > 0:
                if d > last and falling:
                    continue                        # going up after going down: a valley
                if d < last:
                    now_falling = True
            count += count_from(position + 1, used + (1 << d), d, now_falling, tight and d == digits[positi
on])
        memo[key] = count
        return count

    return total + count_from(0, 0, 0, False, True)


# ---- main question: tests ----
def test_main_question():
    for good in (196, 23, 12463):
        assert is_perfect(good)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    for limit in (1, 9, 10, 99, 100, 500, 2000):         # the DP agrees with testing every number
        slow = 0
        for n in range(1, limit + 1):
            if is_perfect(n):
                slow += 1
        assert count_up_to(limit) == slow


# =====================================================
# FOLLOW-UP 1: Why does the DP only need a "falling" flag instead of the full history?
# =====================================================
# Answer: a valley is "down, then up". Once we have gone down, the only
# thing that matters is that we never go up again; which digits came
# before is irrelevant (the used-digit mask handles repeats).


# =====================================================
# FOLLOW-UP 3: What is the closed form without a DP?
# =====================================================


# =====================================================
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# =====================================================
# Answer: with_k_digits(k) above: choose(9, k) * 2^(k - 1).


# =====================================================
# FOLLOW-UP 2: How do you count the numbers in a range [A, B]?
# =====================================================


# =====================================================
# FOLLOW-UP 5: Count perfect numbers in a range [A, B]
# =====================================================
# ---- follow-up 2: function ----
# Answer: inclusion-exclusion: count(B) - count(A - 1). The DP depends
# on the upper bound only and is cheap, so call it twice.
```

```
# Complexity: Time the same as count_up_to, Space the same
def count_between(a, b):
    return count_up_to(b) - count_up_to(a - 1)


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert count_between(10, 99) == count_up_to(99) - count_up_to(9)
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72                       # every 2-digit number with different non-zero dig
its


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate."
Practice bitmask-over-digits + tight-flag state machines.
Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

**STUDY PATH: question 11 of 53 | topic 5 of 30: Recurrence on a position or a range (memo or table) | 3 of 3**

Template fit: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end) | asked 265 days ago | frequency Single report

HARD | IN APP LC 140 | ColorStack | technical interview | 11 of 53

# Word Break: All Sentences

ASKED 2026-01-28 | 265 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 207/288

## ABOUT THIS QUESTION

**Topic:** Recurrence on a position or a range (memo or table)

**Optimal answer from:** your app's interview approach (LeetCode #140)

**LeetCode match:** LC 140 Word Break II

Frequency: Single report | Last asked: 2026-01-28 | Roles: SWE Intern

## QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

**Function signature**

```
from typing import List
def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...
```

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

### EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]` |
| **Output** | `["cats and dog", "cat sand dog"]` |
| **Why** | `Two segmentations use only dictionary words.` |

| Example 2 | |
|---|---|
| **Input** | `s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]` |
| **Output** | `["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]` |
| **Why** | `All three segmentations consist entirely of dictionary words.` |

| Example 3 | |
|---|---|
| **Input** | `s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]` |
| **Output** | `[]` |
| **Why** | `No segmentation covers the whole string with dictionary words.` |

### CONSTRAINTS

- 1 <= len(s) <= 20
- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.

## QUESTION

- If no valid segmentation exists, return an empty list [] .

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute_word_break(s, word_dict):
    words, out = set(word_dict), []
    for mask in range(1_(len(s)_-_1)):
        pieces, start = [], 0
        for i in range(len(s)_-_1):
            if mask >> i & 1:
                pieces.append(s[start : i + 1])
                start = i + 1
        pieces.append(s[start:])
        if all(p in words for p in pieces):
            out.append(" ".join(pieces))
    return out
```

| TIME | SPACE |
|---|---|
| O(2^n * n) | O(n) |

### WHY IT IS SLOW

It tests all 2^(n-1) cuts although most start with a non-word.

## OPTIMAL SOLUTION · Memoized DFS on the start index

| TIME | SPACE |
|---|---|
| O(n^2 + output) | O(n + output) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Only count the segmentations.

A DP over suffixes: ways[i] is the sum of ways[j] over words s[i:j]. No strings are built, so it is O(n * L) with a set of words.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Word Break: All Sentences
#
# You are given a string s and a dictionary word_dict of
# distinct words. Return ALL possible ways to segment s
# into a sequence of space-separated words such that every
# word in the sequence appears in word_dict . A dictionary
# word may be reused any number of times. Each returned
# sentence is a single string of the chosen words joined
# by single spaces. The sentences may be returned in any
# order. Function signature: from typing import List def
# word_break_all(s: str, word_dict: List[str]) ->
# List[str]: ... Follow-up (as reported): Solve with
# backtracking: at each position try every dictionary word
# that is a prefix of the remaining string, then recurse.
# Add memoization keyed on the suffix (or start index) to
```

```
# reuse partial results and avoid exponential re-work.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the segmentations.
#
# INTERVIEW NOTES
# From a start index, try every dictionary word that is a
# prefix of the remaining string and solve the rest. The
# same suffix is reached by many routes, so cache the list
# of sentences per start index (memoization). Without the
# cache the work is exponential; with it each suffix is
# solved once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (memoised search on the start index): the sentences that can be
# made from the ending of s that starts at `start` do not depend on how
# we got there, so compute them once per start. Try every word that
# begins at `start` and put it in front of every sentence of the rest.
# Complexity: Time O(n^3 + size of the output) (n^2 substrings, each hashed in O(n)), Space O(size of the o
utput)
def word_break_all(s, word_dict):
```

```
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]                                 # nothing left: one empty sentence
        if start in memo:
            return memo[start]
        result = []
        for end in range(start + 1, len(s) + 1):
            word = s[start:end]
            if word in words:
                for rest in sentences(end):
                    if rest == "":
                        result.append(word)
                    else:
                        result.append(word + " " + rest)
        memo[start] = result
        return result

    return sentences(0)


# ---- main question: tests ----
def test_main_question():
    found = word_break_all("catsanddog", ["cat", "cats", "and", "sand", "dog"])
    assert sorted(found) == ["cat sand dog", "cats and dog"]
    assert word_break_all("catsandog", ["cats", "dog", "sand", "and", "cat"]) == []
    pineapple = word_break_all("pineapplepenapple", ["apple", "pen", "applepen", "pine", "pineapple"])
    assert len(pineapple) == 3


# ============================================================
```

```
# FOLLOW-UP 1: Only count the segmentations
# ========================================================
# ---- follow-up 1: function ----
# Answer: a DP over the endings: ways[i] is the sum of ways[j] over
# every word s[i:j]. No strings are built, so it is O(n * L) with a set of words.
# Complexity: Time O(n^3) in the worst case, Space O(n)
def count_sentences(s, word_dict):
    words = set(word_dict)
    ways = [0] * (len(s) + 1)
    ways[len(s)] = 1
    for i in range(len(s) - 1, -1, -1):
        for j in range(i + 1, len(s) + 1):
```

```
            if s[i:j] in words:
                ways[i] += ways[j]
    return ways[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_sentences("catsanddog", ["cat", "cats", "and", "sand", "dog"]) == 2
    assert count_sentences("catsandog", ["cats", "dog", "sand", "and", "cat"]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 6 OF 30 |

# Best-first search with a heap: Dijkstra and its minimax twin

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**4**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Best-first search with a heap: Dijkstra and its minimax twin | 4 questions | tap a title to jump to it**

**READ FIRST: the code template for these 4 questions (tap here) >**


| | | | |
|---|---|---|---|
| H | 12. Minimize maximum height along a grid path | PracHub | 2026-01-04 |
| H | 13. Find minimum threshold enabling grid path | PracHub | 2026-02-11 |
| H | 14. Minimize travel time with optional meeting point | PracHub | 2026-01-06 |
| H | 15. Compute minimax grid path and network delay | PracHub | 2026-01-22 |

CODE TEMPLATE | GROUP 6 OF 30 | 4 QUESTIONS, ONE PATTERN

# Best-first search with a heap: Dijkstra and its minimax twin

## The one idea these questions share

Keep a heap of (cost so far, node). Pop the cheapest, skip stale entries, and push any neighbour you improved. Dijkstra adds the edge weight to the cost; the minimax questions take the maximum of the cost and the edge instead. Everything else is the same loop.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def best_first(start, neighbors, combine, goal=None):
    best = {start: 0}
    heap = [(0, start)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:                      # a stale heap entry
            continue
        if u == goal:                           # CHANGE 1: stop at the goal
            return cost
        for v, w in neighbors(u):               # CHANGE 2: edges (or grid steps)
            nxt = combine(cost, w)              # CHANGE 3: cost + w, or max(cost, w)
            if nxt < best.get(v, float("inf")):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    return best
```

## 2. Helper functions that usually get written next to it

```
def grid_neighbors(grid):                   # (cell, cost of stepping onto it) for minimax grids
    rows, cols = len(grid), len(grid[0])
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc), grid[nr][nc]
    return step


def reverse_graph(adj):                     # for runs from the destination backwards
    rev = {u: [] for u in adj}
    for u, edges in adj.items():
        for v, w in edges:
            rev[v].append((u, w))
    return rev
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **12** | Minimize maximum height along a grid path | the same minimax loop on a height grid |
| **13** | Find minimum threshold enabling grid path | the same minimax loop: the smallest threshold that lets you walk from corner to corner |
| **14** | Minimize travel time with optional meeting point | three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd |

| # | Question | What you change in the template |
|---|---|---|
| **15** | Compute minimax grid path and network delay | combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 12 of 53 | topic 6 of 30: Best-first search with a heap: Dijkstra and its minimax twin | 1 of 4**

Template fit: the same minimax loop on a height grid | asked 289 days ago

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 12 of 53

# Minimize maximum height along a grid path

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 61/288

## ABOUT THIS QUESTION

**Topic:** Best-first search with a heap: Dijkstra and its minimax twin

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

**LeetCode match:** LC 778 Swim in Rising Water / LC 1631

Posted on PracHub: 2026-01-04 (updated 2026-06-26)

**Original:** https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

## QUESTION

You are given a grid of heights and walk from the top-left cell to the bottom-right cell, moving up, down, left or right. The cost of a path is the largest height of any cell on it. Return the smallest possible cost (and, with with_path=True, a path that achieves it).

### Function signature

def min_max_path(H: list, with_path=False)

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[[1, 9], [2, 3]]` |
| Output | `3` |
| Why | `Down then right passes cells 1, 2, 3: the largest is 3; going right first would have to cross the 9.` |

| Example 2 | |
|---|---|
| Input | `[[1, 2], [4, 3]], with_path = true` |
| Output | `(3, [(0, 0), (0, 1), (1, 1)])` |
| Why | `With with_path=True you also get the route.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

| TIME | SPACE |
|---|---|
| exponential | O(path) |

## BRUTE FORCE

**WHY IT IS SLOW**

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

## OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

| TIME | SPACE |
|---|---|
| **O(m * n log(m * n))** | **O(m * n)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

## FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

## FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize maximum height along a grid path
#
# You are given a grid of heights and walk from the top-
# left cell to the bottom-right cell, moving up, down,
# left or right. The cost of a path is the largest height
# of any cell on it. Return the smallest possible cost
# (and, with with_path=True, a path that achieves it).
#
# Signature:
#   def min_max_path(H: list, with_path=False)
#
# Examples:
# [[1, 9], [2, 3]]  ->  3
# [[1, 2], [4, 3]], with_path = true  ->  (3, [(0, 0), (0,
#     1), (1, 1)])
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
```

```
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def reachable(H, limit):
    # can we go top-left to bottom-right using only cells of height <= limit?
    rows, cols = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen = {(0, 0)}
    stack = [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (rows - 1, cols - 1):
            return True
        for nr, nc in neighbors(r, c, rows, cols):
            if (nr, nc) not in seen and H[nr][nc] <= limit:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return False


# ---- main question: main function ----
# Idea: Dijkstra where a path costs its highest cell. Heap of (highest
# cell so far, cell): take the smallest, extend to each neighbour with
# cost max(current, neighbour), keep only improvements.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(H, with_path=False):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    parent = {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
```

```
        if (r, c) == (rows - 1, cols - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
    return None


# the same answer by binary search over the sorted heights
# Complexity: Time O(m * n log(m * n)) (a search for each of about log(m * n) heights), Space O(m * n)
def min_max_path_bsearch(H):
    seen = set()
    for row in H:
        for h in row:
            seen.add(h)
    heights = sorted(seen)
    low, high = 0, len(heights) - 1
    while low < high:
        mid = (low + high) // 2
        if reachable(H, heights[mid]):
            high = mid                  # this height is enough: try lower
        else:
            low = mid + 1               # not enough: go higher
    return heights[low]


# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    highest = 0
```

```
    for r, c in path:
        if H[r][c] > highest:
            highest = H[r][c]
    assert highest == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6


# ======================================================
# FOLLOW-UP 1: The cost is the largest difference between adjacent cells on the path (LeetCode 1631)
# ======================================================
# ---- follow-up 1: function ----
# Answer: the same Dijkstra with max, but a step costs the absolute
# height DIFFERENCE instead of the cell value. (The binary-search
# version also works: allow an edge only if its difference is at most the limit.)
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_effort_path(H):
    rows, cols = len(H), len(H[0])
```

```
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            return cost
        for nr, nc in neighbors(r, c, rows, cols):
            step = H[nr][nc] - H[r][c]
            if step < 0:
                step = -step
            new_cost = cost
            if step > new_cost:
                new_cost = step
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, nr, nc))
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_effort_path([[1, 2, 2], [3, 8, 2], [5, 3, 5]]) == 2
```

```
# ============================================================
# FOLLOW-UP 2: Water rises over time (the cell is usable from time H[r][c])
# ============================================================
# Answer: same as the main question: you may stand on a cell once the
# time reaches its height, so the answer is min_max_path.


# ============================================================
# FOLLOW-UP 3: Return the path, not only the value
# ============================================================
# Answer: min_max_path(H, with_path=True) stores each cell's parent and
# walks back from the goal.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

### Test cases

| Input | Expected |
|---|---|
| ([[1, 2, 3], [4, 5, 6], [7, 8, 9]],) | 9 |

| Input | Expected |
| --- | --- |
| `([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)` | `8` |

**ALSO REPORTED · 1 more**

**Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low**

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.

Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

**STUDY PATH: question 13 of 53 | topic 6 of 30: Best-first search with a heap: Dijkstra and its minimax twin | 2 of 4**

Template fit: the same minimax loop: the smallest threshold that lets you walk from corner to corner | asked 251 days ago

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 13 of 53

# Find minimum threshold enabling grid path

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 60/288

## ABOUT THIS QUESTION

**Topic:** Best-first search with a heap: Dijkstra and its minimax twin

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

**LeetCode match:** LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-06-01)

**Original:** https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

## QUESTION

You are given a grid of heights. Find the smallest threshold T such that you can walk from the top-left cell to the bottom-right cell, moving up, down, left or right, using only cells of height at most T. Return T together with one path that achieves it.

### Function signature

def min_max_path(H: list) -> (int, list)

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `[[1, 3], [2, 2]]` |
| **Output** | `(2, [(0, 0), (1, 0), (1, 1)])` |
| **Why** | `Going down then right uses cells 1, 2, 2: the largest is 2, and no path stays below that.` |

**Example 2**

| | |
|---|---|
| **Input** | `[[7]]` |
| **Output** | `(7, [(0, 0)])` |
| **Why** | `A single cell: the threshold is its own height.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

| TIME | SPACE |
|---|---|
| exponential | O(path) |

## BRUTE FORCE

**WHY IT IS SLOW**

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

## OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

| TIME | SPACE |
|---|---|
| **O(m * n log(m * n))** | **O(m * n)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

## FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

## FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find minimum threshold enabling grid path
#
# You are given a grid of heights. Find the smallest
# threshold T such that you can walk from the top-left
# cell to the bottom-right cell, moving up, down, left or
# right, using only cells of height at most T. Return T
# together with one path that achieves it.
#
# Signature:
#   def min_max_path(H: list) -> (int, list)
#
# Examples:
# [[1, 3], [2, 2]]  ->  (2, [(0, 0), (1, 0), (1, 1)])
# [[7]]  ->  (7, [(0, 0)])
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
```

```
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def bottleneck_path(H, use_difference):
    # Dijkstra where a path costs the MAX of its steps (not their sum). A
    # step costs the height of the cell entered, or (use_difference) the
    # height difference between the two cells. Returns (cost, path).
    rows, cols = len(H), len(H[0])
    start_cost = 0
    if not use_difference:
        start_cost = H[0][0]
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue                                    # an outdated entry
        if (r, c) == (rows - 1, cols - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            if use_difference:
                step = H[nr][nc] - H[r][c]
                if step < 0:
                    step = -step
            else:
                step = H[nr][nc]
            new_cost = cost
            if step > new_cost:
                new_cost = step
            if new_cost < best.get((nr, nc), INF):
```

```
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
    return None, []


# ---- main question: main function ----
# Idea: Dijkstra with cost = the highest cell on the path. The first
# time a cell leaves the heap its cost is final. (Water rising over
# time is the same problem: a cell is usable once the water reaches
# its height.) Returns (threshold, path).
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_max_path(H):
    return bottleneck_path(H, False)


# ---- main question: tests ----
def test_main_question():
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    highest = 0
    for r, c in path:
        if H[r][c] > highest:
            highest = H[r][c]
    assert highest == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])


# ========================================================
# FOLLOW-UP 1: The cost is the largest difference between adjacent cells on the path (LeetCode 1631)
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same Dijkstra with max, but a step costs the absolute
# height DIFFERENCE instead of the cell value. (A binary search also
# works: allow an edge only if its difference is at most the limit.)
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def min_effort_path(H):
    return bottleneck_path(H, True)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    effort, path = min_effort_path([[1, 2, 2], [3, 8, 2], [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])


# ========================================================
# FOLLOW-UP 2: Water rises over time (the cell is usable from time H[r][c])
# ========================================================
# Answer: same as the main question: you may stand on a cell once the
# time reaches its height, so the answer is the highest cell on the
# best path, which is exactly min_max_path.


# ========================================================
# FOLLOW-UP 3: Return the path, not only the value
# ========================================================
# Answer: already returned: every improvement stores its parent cell,
# and the path is rebuilt by walking back from the goal.


# ========================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(grid: list[list[int]]) -> int:
```

### Sample call

```
solution([[0, 2], [1, 3]])
```

### Test cases

| Input | Expected |
|---|---|
| ([[0, 2], [1, 3]],) | 3 |
| ([[0, 100, 2], [1, 2, 2], [3, 4, 1]],) | 2 |

**STUDY PATH: question 14 of 53 | topic 6 of 30: Best-first search with a heap: Dijkstra and its minimax twin | 3 of 4**

Template fit: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd | asked 287 days ago

**HARD | PracHub | Technical Screen | 14 of 53**

# Minimize travel time with optional meeting point

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 66/288

## ABOUT THIS QUESTION

**Topic:** Best-first search with a heap: Dijkstra and its minimax twin

**Optimal answer from:** the interview site's own solution

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths

**LeetCode match:** LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-22)

**Original:** https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

## QUESTION

A weighted graph has n nodes and edges (u, v, w) (undirected unless directed=True). Two people start at nodes a and b and both must end at dest. They may meet at some node m and walk the rest of the way together, so the total time is max(time a to m, time b to m) + time m to dest. Return [the minimum total time, the meeting node] (or [-1, -1] when dest cannot be reached). Meeting at dest itself, or at a or b, is allowed.

### Function signature

def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4` |
| **Output** | `[7, 1]` |
| **Why** | `They meet at node 1: a needs 2, b needs 1, then 5 more to node 4: 2 + 5 = 7. Meeting at node 4 itself also takes 7, and the smaller node number is reported.` |

| Example 2 | |
|---|---|
| **Input** | `3, [(0, 1, 1)], 0, 1, 2` |
| **Output** | `[-1, -1]` |
| **Why** | `Node 2 is not connected to anything.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, A, B, dest):
    d = floyd_warshall(n, edges)  # all pairs
    best, best_m = INF, None
    for m in range(n):
        cost = max(d[A][m], d[B][m]) + d[m][dest]
        if cost < best:
            best, best_m = cost, m
    return best
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(V^3) | O(V^2) |

**WHY IT IS SLOW**

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

## OPTIMAL SOLUTION · Three Dijkstra runs (from A, from B, and into the destination)

| TIME | SPACE |
|---|---|
| O((V + E) log V) | O(V + E) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

## FOLLOW-UP 2 Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

## FOLLOW-UP 3 Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize travel time with optional
# meeting point
#
# A weighted graph has n nodes and edges (u, v, w)
# (undirected unless directed=True). Two people start at
# nodes a and b and both must end at dest. They may meet
# at some node m and walk the rest of the way together, so
# the total time is max(time a to m, time b to m) + time m
# to dest. Return [the minimum total time, the meeting
# node] (or [-1, -1] when dest cannot be reached). Meeting
# at dest itself, or at a or b, is allowed.
#
# Signature:
#   def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list
#
# Examples:
# 5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4  ->  [7,
#     1]
# 3, [(0, 1, 1)], 0, 1, 2  ->  [-1, -1]
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. More than two travellers.
# 2. Meeting is optional.
# 3. Edge weights change over time (traffic).
#
# INTERVIEW NOTES
# Break it into shortest paths from three places: from A,
# from B, and into the destination (a Dijkstra on the
# reversed graph if directed). For each node m the cost is
# max(distA[m], distB[m]) + dist(m to destination); take
# the minimum. Three single-source searches replace trying
# every meeting node with its own search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def dijkstra(graph, source):
    # shortest distance from source to every node (INF if unreachable)
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # the forward graph, and the reversed graph (used to get distances TO the destination)
    forward = []
    reverse = []
    for i in range(n):
        forward.append([])
        reverse.append([])
    for u, v, w in edges:
        forward[u].append((v, w))
        reverse[v].append((u, w))
        if not directed:
            forward[v].append((u, w))
            reverse[u].append((v, w))
    return forward, reverse


# ---- main question: main function ----
# Idea: both travellers start together, meet at some node m (the one
# who arrives first waits), then travel together to dest. Run Dijkstra
# from A, from B, and from dest on the reversed graph (distance from
```

```
# every node TO dest). The best meeting node minimises
# max(distA, distB) + distance to dest. Meeting AT dest is the same as
# "no meeting". Returns [total time, meeting node] or [-1, -1].
# Complexity: Time O((V + E) log V) (three Dijkstra runs), Space O(V + E)
def meeting_point(n, edges, a, b, dest, directed=False):
```

```
    forward, reverse = build(n, edges, directed)
    dist_a = dijkstra(forward, a)
    dist_b = dijkstra(forward, b)
    to_dest = dijkstra(reverse, dest)
    best_total = INF
    best_node = -1
    for m in range(n):
        wait_for_both = dist_a[m]
        if dist_b[m] > wait_for_both:
            wait_for_both = dist_b[m]
        total = wait_for_both + to_dest[m]
        if total < best_total:
            best_total = total
            best_node = m
    if best_total == INF:
        return [-1, -1]
    return [best_total, best_node]


# ---- main question: tests ----
STAR = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9), (1, 3, 9)]


def test_main_question():
    assert meeting_point(4, STAR, 0, 1, 3) == [6, 2]        # meet at 2: max(1, 2) + 4 = 6; alone costs 9
    assert meeting_point(3, [(0, 1, 1)], 0, 1, 2) == [-1, -1]   # nobody can reach the destination
    assert meeting_point(4, STAR, 0, 0, 3)[0] == 5          # same start: they walk together
    assert meeting_point(3, [(0, 1, 1), (1, 2, 1)], 0, 1, 2, True) == [2, 1]   # one-way roads


# ========================================================
# FOLLOW-UP 1: More than two travellers
# ========================================================
# ---- follow-up 1: function ----
# Answer: one Dijkstra per traveller and one from the destination; for
# each node the group waits for the last traveller, then shares the
# last leg. (If the cost is the total distance walked, add the
# distances instead of taking the maximum.)
# Complexity: Time O(k * (V + E) log V) for k travellers, Space O(k * V + E)
def meeting_point_many(n, edges, starts, dest, directed=False):
    forward, reverse = build(n, edges, directed)
    all_dists = []
    for s in starts:
        all_dists.append(dijkstra(forward, s))
    to_dest = dijkstra(reverse, dest)
```

```
    best_total = INF
    best_node = -1
    for m in range(n):
        wait_for_all = 0
        for dist in all_dists:
            if dist[m] > wait_for_all:
                wait_for_all = dist[m]
        if wait_for_all + to_dest[m] < best_total:
            best_total = wait_for_all + to_dest[m]
            best_node = m
    if best_total == INF:
        return [-1, -1]
    return [best_total, best_node]
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert meeting_point_many(4, STAR, [0, 1, 1], 3) == [6, 2]


# ========================================================
# FOLLOW-UP 2: Meeting is optional
# ========================================================
# Answer: already covered: meeting at the destination costs
# max(distA, distB) + 0, and each traveller going alone is just
# dist(a, dest); take the cheaper of "meet somewhere" and "go alone".


# ========================================================
# FOLLOW-UP 3: Edge weights change over time (traffic)
# ========================================================
# Answer: the edge cost becomes a function of the time you enter it.
# Dijkstra still works as long as leaving later never lets you arrive
# earlier (the FIFO property); otherwise search over (node, time).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(n: int, edges: list[list[int]], startA: int, startB: int, dest: int, directed: bool = False) -
> list[int]:
```

### Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)
```

### Test cases

| Input | Expected |
|---|---|
| (4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False) | [4, 1] |
| (4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True) | [5, 1] |

**STUDY PATH: question 15 of 53 | topic 6 of 30: Best-first search with a heap: Dijkstra and its minimax twin | 4 of 4**

Template fit: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell | asked 271 days ago

**HARD | PracHub | Technical Screen | 15 of 53**

# Compute minimax grid path and network delay

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 113/288

## ABOUT THIS QUESTION

**Topic:** Best-first search with a heap: Dijkstra and its minimax twin

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

**LeetCode match:** LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

**Original:** https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

## QUESTION

Two tasks. (1) minimax_path(H): given a grid of heights, find a path from the top-left to the bottom-right cell (moving up, down, left or right) whose largest height is as small as possible, and return that largest height. (2) network_delay(n, edges, source, node_delay=None): a signal starts at source and travels along directed weighted edges (u, v, w); return the time until every node has received it, or -1 if some node never does. node_delay[u], when given, is a pause at node u before it forwards the signal.

### Function signature

def minimax_path(H: list) -> int

def network_delay(n: int, edges: list, source: int, node_delay=None) -> int

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[[1, 5, 1], [2, 2, 5]]` |
| Output | `5` |
| Why | `Both ways around pass a cell of height 5, so the answer is 5.` |

| Example 2 | |
|---|---|
| Input | `4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0` |
| Output | `10` |
| Why | `The signal reaches node 1 at 2, node 2 at 5, node 3 at 10: everyone has it after 10.` |

| Example 3 | |
|---|---|
| Input | `3, [(0, 1, 1)], 0` |
| Output | `-1` |
| Why | `Node 2 has no incoming edge: it is never reached.` |

## QUESTION

| Example 4 | |
|---|---|
| Input | `4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0, node_delay = [1, 0, 0, 0]` |
| Output | `11` |
| Why | `With a pause of 1 at node 0, every arrival is 1 later: node 3 is reached at 11.` |

## BRUTE FORCE · Part 1: DFS over every simple path

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

| TIME | SPACE |
|---|---|
| exponential | O(path) |

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

## BRUTE FORCE · Part 2: Bellman-Ford from the source

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    worst = max(dist)
    return -1 if worst == INF else worst
```

| TIME | SPACE |
|---|---|
| O(V * E) | O(V) |

WHY IT IS SLOW

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

## OPTIMAL SOLUTION · Part 1: Dijkstra where the cost of a path is its maximum cell

| TIME | SPACE |
|---|---|
| O(m * n log(m * n)) | O(m * n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Part 2: Dijkstra from the source, answer is the largest distance

| TIME | SPACE |
|---|---|
| O((V + E) log V) | O(V + E) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

## FOLLOW-UP 2 Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. **Cost:** unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

## FOLLOW-UP 3 Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute minimax grid path and network
# delay
#
# Two tasks. (1) minimax_path(H): given a grid of heights,
# find a path from the top-left to the bottom-right cell
# (moving up, down, left or right) whose largest height is
# as small as possible, and return that largest height.
# (2) network_delay(n, edges, source, node_delay=None): a
# signal starts at source and travels along directed
# weighted edges (u, v, w); return the time until every
# node has received it, or -1 if some node never does.
# node_delay[u], when given, is a pause at node u before
# it forwards the signal.
#
# Signature:
#   def minimax_path(H: list) -> int
#   def network_delay(n: int, edges: list, source: int, node_delay=None) -> int
#
# Examples:
# [[1, 5, 1], [2, 2, 5]]  ->  5
# 4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0  ->  10
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 1 with a union-find (sort cells, add them one by
# one).
# 2. Part 2: weights are huge.
# 3. Part 2: the signal can be blocked at some nodes, or
# each node adds a processing delay.
#
# INTERVIEW NOTES
# Both are 'shortest path' with a different notion of
# cost. For the grid the cost of a path is its maximum,
```

```
# which still works in Dijkstra: the best way to reach a
# cell is the one with the smallest maximum, and extending
# a path can never lower it. For the network, Dijkstra
# from the source gives every arrival time, and the answer
# is the largest of them (-1 if some node is never
# reached).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# PART 1: walk from the top-left to the bottom-right so that the
# HIGHEST cell on the path is as low as possible.
# Idea: Dijkstra where the cost of a path is its maximum (not a sum).
# Heap of (highest cell so far, cell); pop the smallest.
# Complexity: Time O(m * n log(m * n)), Space O(m * n)
def minimax_path(H):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            return cost
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]                # the cost of a path is its maximum, never a sum
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, nr, nc))
    return -1
```

```
# PART 2: when does a signal sent from `source` reach every node?
# Plain Dijkstra; -1 if some node is never reached. `node_delay` is
# an optional processing delay paid when leaving each node (follow-up 3).
# Complexity: Time O((V + E) log V), Space O(V + E)
def network_delay(n, edges, source, node_delay=None):
```

```
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        pause = 0
        if node_delay:
            pause = node_delay[u]
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
    worst = 0
    for d in dist:
        if d > worst:
            worst = d
    if worst == INF:
        return -1
    return worst


# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
    assert network_delay(4, edges, 0) == -1              # node 3 cannot be reached


# ========================================================
```

```
# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add them one by one)
# ========================================================
# ---- follow-up 1: helper functions ----
class UnionFind:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x


# ---- follow-up 1: function ----
# Answer: add the cells from the lowest to the highest and join each
# new cell to its neighbours that are already added. The moment the
# start and the goal are in one group, that height is the answer:
# O(mn log mn) for the sort.
# Complexity: Time O(m * n log(m * n)) for the sort, Space O(m * n)
def minimax_path_union_find(H):
    rows, cols = len(H), len(H[0])
    cells = []
    for r in range(rows):
        for c in range(cols):
```

```
            cells.append((H[r][c], r, c))
    cells.sort()
    uf = UnionFind()
    for h, r, c in cells:
        uf.parent[(r, c)] = (r, c)
        for nr, nc in neighbors(r, c, rows, cols):
            if (nr, nc) in uf.parent:
                uf.parent[uf.find((r, c))] = uf.find((nr, nc))
        start, goal = (0, 0), (rows - 1, cols - 1)
        if start in uf.parent and goal in uf.parent and uf.find(start) == uf.find(goal):
            return h
    return -1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path_union_find(H) == minimax_path(H)
    assert minimax_path_union_find([[5]]) == 5
```

```
# ============================================================
# FOLLOW-UP 2: Part 2: weights are huge
# ============================================================
# Answer: nothing changes: Python integers do not overflow. In a language
# with fixed-size integers use 64 bits and a safe "infinity" value.


# ============================================================
# FOLLOW-UP 3: Part 2: the signal can be blocked at some nodes, or each node adds a processing delay
# ============================================================
# ---- follow-up 3: tests ----
# Answer: a processing delay is added when leaving a node (the
# `node_delay` parameter); a blocked node is simply never relaxed.
def test_follow_up_3():
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    assert network_delay(3, big, 0, [1, 1, 1]) == 2 * 10 ** 9 + 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

### Test cases

| Input | Expected |
| --- | --- |
| `[[1, 10, 6], [1, 3, 2], [7, 2, 2]]` | `3` |
| `[[5, 1, 7], [4, 8, 2], [3, 6, 0]]` | `6` |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 7 OF 30 |

# Backtracking: choose, recurse, undo

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Backtracking: choose, recurse, undo | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 16. Matrix Flower Placement with House Adjacency 1Point3Acres 2026-05-31

CODE TEMPLATE | GROUP 7 OF 30 | 1 QUESTIONS, ONE PATTERN

# Backtracking: choose, recurse, undo

## The one idea these questions share

Build the answer one choice at a time in a list. When the list is complete, record it. Otherwise loop over the allowed choices: append one, recurse, and then remove exactly what you added so the next choice starts from the same state. Pruning is just a check that skips a choice.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def permutations(nums):
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):                # CHANGE 1: when a path is complete
            out.append(cur[:])
            return
        for i, x in enumerate(nums):             # CHANGE 2: the choices at each step
            if not used[i]:                      # CHANGE 3: the rule that prunes a choice
                used[i] = True
                cur.append(x)
                go()
                cur.pop()                        # undo exactly what you did
                used[i] = False

    go()
    return out
```

## 2. Helper functions that usually get written next to it

```
def combinations(nums, k):                       # same shape, with a start index so order does not matter
    out, cur = [], []

    def go(start):
        if len(cur) == k:
            out.append(cur[:])
            return
        for i in range(start, len(nums)):
            cur.append(nums[i])
            go(i + 1)
            cur.pop()

    go(0)
    return out
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **16** | Matrix Flower Placement with House Adjacency | place row by row; reject a row early when the houses two rows above are still not next to a flower |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 16 of 53 | topic 7 of 30: Backtracking: choose, recurse, undo | 1 of 1**

Template fit: place row by row; reject a row early when the houses two rows above are still not next to a flower | asked 142 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 16 of 53

# Matrix Flower Placement with House Adjacency

ASKED 2026-05-31 | 142 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 250/288

## ABOUT THIS QUESTION

**Topic:** Backtracking: choose, recurse, undo

**Optimal answer from:** written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

## QUESTION

A grid has houses (H) and empty cells (0). Place flowers (F) on empty cells so that every row and every column holds at most one flower (exactly one of each when exactly_one is True) and every house has a flower directly beside it (up, down, left or right). Return one valid grid as a list of strings, or None if there is none.

**Function signature**

def place_flowers(grid: list, exactly_one=False) -> list | None

EXAMPLES

| Example 1 | |
|---|---|
| Input | `['H000', 'H000', '000H', '0000']` |
| Output | `['HF00', 'H0F0', 'F00H', '000F']` |
| Why | `Flowers at (0,1), (2,0) and (1,3): one per row and column, and each house touches a flower.` |

| Example 2 | |
|---|---|
| Input | `['H000', 'H000', '000H', '0000'], exactly_one = true` |
| Output | `['HF00', 'H0F0', 'F00H', '000F']` |
| Why | `Exactly one flower in every row and column: a fourth flower at (3,2) completes it.` |

| Example 3 | |
|---|---|
| Input | `['H', 'H']` |
| Output | `null` |
| Why | `The two houses can only be served by the same column, so no valid placement exists.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def brute(grid):
    empty = cells_with(grid, 0)
    count = 0
    for k in range(len(empty) + 1):
        for flowers in combinations(empty, k):
            if not one_per_row_and_col(flowers):
                continue
            if houses_covered(grid, flowers):
                count += 1
    return count
```

| TIME | SPACE |
|---|---|
| O(2^cells) | O(cells) |

**WHY IT IS SLOW**

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

## OPTIMAL SOLUTION · Row-by-row backtracking with used-column set and delayed house checks

| TIME | SPACE |
|---|---|
| exponential worst case, h... exponential worst case, heavily pruned | O(rows + cols) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

## FOLLOW-UP 2 How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. **Cost:** the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

## FOLLOW-UP 3 Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

## FOLLOW-UP 4 Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Matrix Flower Placement with House
# Adjacency
#
# A grid has houses (H) and empty cells (0). Place flowers
# (F) on empty cells so that every row and every column
# holds at most one flower (exactly one of each when
# exactly_one is True) and every house has a flower
# directly beside it (up, down, left or right). Return one
# valid grid as a list of strings, or None if there is
# none.
#
# Signature:
#   def place_flowers(grid: list, exactly_one=False) -> list | None
#
# Examples:
# ['H000', 'H000', '000H', '0000']  ->  ['HF00', 'H0F0',
#     'F00H', '000F']
# ['H000', 'H000', '000H', '0000'], exactly_one = true  ->
#     ['HF00', 'H0F0', 'F00H', '000F']
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes between 'at most one' and 'exactly one'
# flower per row and column?
# 2. How would you speed up the search?
# 3. Exactly one flower per row and column.
# 4. Count the solutions instead of listing them.
#
# INTERVIEW NOTES
# At most one flower per row and column is the N-Queens
# pattern: decide row by row which column (if any) gets
# the flower and remember the used columns. The extra rule
# about houses is local, so check a house as soon as the
# last row that can affect it is decided, and prune the
# branch if it fails.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# ---- main question: main function ----
# The grid has houses "H" and empty cells "0". Put at most one flower "F"
# in every row and every column, only on empty cells, so that every house
# has a flower right next to it (above, below, left or right).
# Idea: backtracking row by row. For each row either put a flower in a
# free column (remember the used columns) or leave the row empty; undo the
# choice afterwards. Houses are checked LATE: when we start row r, the
# rows before r are decided, so the houses in row r - 2 can no longer
# get a new neighbour and must already be satisfied. That prunes early.
```

```
# exactly_one=True: a flower in EVERY row and EVERY column (follow-up 3);
# find_all=True: return every solution (follow-up 4).
# Complexity: Time O(C^R) in the worst case (a flower in some free column or none, for every row), Space O(
R * C)
def place_flowers(grid, exactly_one=False, find_all=False):
    rows = len(grid)
    cols = len(grid[0])
    cells = []
    for row in grid:
        cells.append(list(row))
    houses = []
    for r in range(rows):
        for c in range(cols):
            if cells[r][c] == "H":
                houses.append((r, c))
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a = r + dr
            b = c + dc
            if 0 <= a < rows and 0 <= b < cols and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        for hr, hc in houses:
            if hr == r and not house_ok(hr, hc):
                return False
        return True

    def solve(r, used):
```

```
        if len(found) > 0 and not find_all:
            return                                  # one solution is enough
        if r >= 2 and not row_houses_ok(r - 2):
            return                                  # a house in row r - 2 can no longer be served
        if r == rows:
            full = len(used) == cols
            if row_houses_ok(rows - 1) and (full or not exactly_one):
                found.append(["".join(line) for line in cells])
            return
        for c in range(cols):                       # try a flower in every free column of this row
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                used.add(c)
                solve(r + 1, used)
                used.remove(c)                      # undo
                cells[r][c] = "0"
        if not exactly_one:
            solve(r + 1, used)                      # or leave the row empty

    solve(0, set())
    if find_all:
        return found
    if len(found) > 0:
        return found[0]
    return None


# ---- main question: tests ----
def test_main_question():
    grid = ["H00", "0H0", "00H"]
    solution = place_flowers(grid)
```

```
    assert solution is not None
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                has_flower = False
                for dr, dc in STEPS:
                    x = r + dr
                    y = c + dc
                    if 0 <= x < 3 and 0 <= y < 3 and solution[x][y] == "F":
                        has_flower = True
                assert has_flower                    # every house has a flower beside it
    for row in solution:
        assert row.count("F") <= 1                   # at most one flower per row
    for c in range(3):
        column = [row[c] for row in solution]
        assert column.count("F") <= 1                # and per column
    assert place_flowers(["H"]) is None              # a lone house in a 1 x 1 grid cannot be served


# =======================================================
# FOLLOW-UP 1: What changes between "at most one" and "exactly one" flower per row and column?
# =======================================================


# =======================================================
# FOLLOW-UP 3: Exactly one flower per row and column
# =======================================================
# Answer: with "exactly one" a row may not be left empty (skip the "leave the
# row empty" branch) and at the end every column must be used. That is
# the exactly_one flag. It also prunes much harder: only n! arrangements
# remain.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert place_flowers(["000", "000"], True) is None           # 2 rows, 3 columns: no flower in every c
olumn
    assert place_flowers(["00", "00"], True) is not None


# =======================================================
# FOLLOW-UP 2: How would you speed up the search?
# =======================================================
# Answer: use a bitmask for the used columns, stop as soon as a house can
# no longer be served (the delayed check above), try rows with houses
# first, and memoise on (row, used columns, the flowers of the last two
# rows) because nothing else matters for the houses still to be checked.


# =======================================================
# FOLLOW-UP 4: Count the solutions instead of listing them
# =======================================================
# ---- follow-up 4: function ----
# Answer: the same recursion, but add one to a counter at the end instead of
# copying the grid. With a bitmask of used columns and memoisation it
# becomes a DP. (Here the solutions are listed with find_all and counted.)
# Complexity: Time O(C^R), Space O(number of solutions * R * C)
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))


# ---- follow-up 4: tests ----
```

```
def test_follow_up_4():
    assert count_solutions(["00", "00"], True) == 2              # the two diagonals
    assert count_solutions(["00", "00"]) == 7                    # none, 4 singles, and the 2 diagonals
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 8 OF 30 |

# Union-find: merge the pairs, then ask who is together

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Union-find: merge the pairs, then ask who is together | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 17. Friend Graph with Timeline (Friend / Unfriend Events) **1Point3Acres 2026-09-29**

CODE TEMPLATE | GROUP 8 OF 30 | 1 QUESTIONS, ONE PATTERN

# Union-find: merge the pairs, then ask who is together

## The one idea these questions share

Keep a parent array. find climbs to the root (with path compression), union joins two roots. The only thing that changes between questions is where the pairs to merge come from and what you read at the end: same group, group size, number of groups, or the moment everything became one group.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.groups = n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[ra] = rb
        self.groups -= 1
        return True


# for a, b in pairs:        # CHANGE 1: where the pairs come from
#     dsu.union(a, b)
# CHANGE 2: read dsu.find(x) == dsu.find(y), dsu.groups, or the pair that made groups == 1
```

## 2. Helper functions that usually get written next to it

```
from collections import defaultdict


def cell_buckets(points, r):               # spatial hash: only nearby cells can hold neighbours
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // r, y // r)].append(i)
    return cells


def group_sizes(dsu, n):
    sizes = defaultdict(int)
    for x in range(n):
        sizes[dsu.find(x)] += 1
    return sizes
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **17** | Friend Graph with Timeline (Friend / Unfriend Events) | pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1 |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 17 of 53 | topic 8 of 30: Union-find: merge the pairs, then ask who is together | 1 of 1**

Template fit: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1 | asked 21 days ago | frequency Medium

**HARD | 1Point3Acres | onsite-coding, phone-screen | 17 of 53**

# Friend Graph with Timeline (Friend / Unfriend Events)

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** NO LEETCODE FREQUENCY PRIORITY 246/288

## ABOUT THIS QUESTION

**Topic:** Union-find: merge the pairs, then ask who is together

**Optimal answer from:** written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

## QUESTION

There are n people numbered 0 to n - 1. Each event is (time, a, b, kind). A 'friend' event makes a and b friends from that time on, and two people are connected when a chain of friends links them. Return the earliest time at which all n people are connected, or -1 if that never happens.

**Function signature**

def earliest_all_connected(n: int, events: list) -> int

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]` |
| **Output** | `5` |
| **Why** | `After time 1, {0, 1} are connected; at time 5 person 2 joins, so everyone is connected at 5.` |

| Example 2 | |
|---|---|
| **Input** | `4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]` |
| **Output** | `-1` |
| **Why** | `Person 3 is never befriended, so the group never becomes one.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

| TIME | SPACE |
|---|---|
| **O(T * (N + E))** | **O(N + E)** |
| for T distinct times | |

WHY IT IS SLOW

## BRUTE FORCE

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

## OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

| TIME | SPACE |
|---|---|
| O(E log E) | O(N) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

## FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

## FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

## FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Friend Graph with Timeline (Friend /
# Unfriend Events)
#
# There are n people numbered 0 to n - 1. Each event is
# (time, a, b, kind). A 'friend' event makes a and b
# friends from that time on, and two people are connected
# when a chain of friends links them. Return the earliest
# time at which all n people are connected, or -1 if that
# never happens.
#
# Signature:
#   def earliest_all_connected(n: int, events: list) -> int
#
# Examples:
# 3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]  ->  5
# 4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]  ->  -1
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. Why does unfriending break plain Union-Find?
# 2. What is the simple answer if the interviewer wants
# something quick?
# 3. Report how many friend groups exist after each event
# (friends only).
# 4. Queries ask 'were A and B connected at time t?' with
# no unfriending.
#
# INTERVIEW NOTES
# Everyone becomes connected at the first moment the graph
# has one component. If friendships only appear, sort by
# time and use union-find (components only merge).
# Unfriending breaks that: removing an edge can split a
# component and union-find cannot undo. With all events
# known up front, handle each edge's lifetime in a segment
# tree over time and use a rollback union-find.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def event_time(event):
    # events are (time, a, b, "friend" or "unfriend"); sort by the time
    return event[0]


class UnionFind:
    # union by size WITHOUT path compression, so every link keeps the time it was made
    # (follow-up 4 needs those times)
    def __init__(self, n):
        self.parent = list(range(n))      # everybody starts as their own leader
        self.size = [1] * n               # group sizes decide who hangs under whom
        self.link_time = [0] * n          # the time a person was linked under its parent
        self.groups = n                   # how many separate groups exist now

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, time):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                  # already together: nothing changes
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra               # ra is the bigger group
        self.parent[rb] = ra              # the smaller group hangs under the bigger one
        self.size[ra] += self.size[rb]
        self.link_time[rb] = time
        self.groups -= 1                  # two groups became one
        return True
```

```
# ---- main question: main function ----
# Idea: sort the events by time and union the two people of every
# friend event. The first time only one group is left is the answer;
# -1 if that never happens.
# Complexity: Time O(E log E + E log n) (the sort, then each union walks up a tree of height log n), Space
O(n)
def earliest_all_connected(n, events):
    if n <= 1:
        return 0
```

```
    uf = UnionFind(n)
    for time, a, b, kind in sorted(events, key=event_time):
        if kind == "friend":
            uf.union(a, b, time)
            if uf.groups == 1:
                return time
    return -1


# ---- main question: tests ----
def test_main_question():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]      # people A, B, C are 0, 1, 2
    assert earliest_all_connected(3, events) == 5
    assert earliest_all_connected(3, events[:1]) == -1       # a missing friendship: never connected
    assert earliest_all_connected(1, []) == 0


# =======================================================
# FOLLOW-UP 1: Why does unfriending break plain Union-Find?
# =======================================================
# Answer: a union merges two sets and forgets which edge caused it,
# so removing that edge cannot tell which members should separate.


# =======================================================
# FOLLOW-UP 2: What is the simple answer if the interviewer wants something quick?
# =======================================================
# ---- follow-up 2: function ----
# Answer: rebuild the graph from the events up to time t and run a BFS
# for each query: O(E + N) per query. State that cost first, then
# explain how the offline method avoids it.
# Complexity: Time O(E log E + n + E) per query, Space O(n + E)
def connected_with_unfriends(n, events, a, b, t):
    edges = set()
    for time, x, y, kind in sorted(events, key=event_time):
        if time > t:
            break
        if x < y:
            pair = (x, y)
        else:
            pair = (y, x)
        if kind == "friend":
            edges.add(pair)               # friend adds the edge
        else:
            edges.discard(pair)           # unfriend removes it
```

```
    adj = {}
    for x, y in edges:
        adj.setdefault(x, []).append(y)
        adj.setdefault(y, []).append(x)
    seen = {a}
    queue = deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
```

```
            return True
        for v in adj.get(u, []):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# ---- follow-up 2: tests ----
def test_follow_up_2():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend"), (7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, events, 0, 2, 6)
    assert not connected_with_unfriends(3, events, 0, 2, 8)      # the unfriend broke the link


# ======================================================
# FOLLOW-UP 3: Report how many friend groups exist after each event (friends only)
# ======================================================
# ---- follow-up 3: function ----
# Answer: union-find with a group counter: it goes down by one when a
# union joins two different groups. Record it after each event.
# Complexity: Time O(E log E + E log n), Space O(n)
def groups_after_each_event(n, events):
    uf = UnionFind(n)
    counts = []
    for time, a, b, kind in sorted(events, key=event_time):
        if kind == "friend":
            uf.union(a, b, time)
            counts.append(uf.groups)
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    assert groups_after_each_event(3, events) == [2, 1]        # groups: 3 -> 2 -> 1
```

```
# ======================================================
# FOLLOW-UP 4: Queries ask "were A and B connected at time t?" with no unfriending
# ======================================================
# ---- follow-up 4: function ----
# Answer: no path compression, and each link remembers its time. Two
# people were connected at time t when the NEWEST link on the path
# between them is not later than t.
# Complexity: Time O(log n) per query (the tree height with union by size), Space O(1)
def connected_at(uf, a, b, t):
    if a == b:
        return True
    newest_to = {a: 0}                    # climbing from a: ancestor -> newest link time so far
    newest = 0
    x = a
    while uf.parent[x] != x:
        if uf.link_time[x] > newest:
            newest = uf.link_time[x]
        x = uf.parent[x]
        newest_to[x] = newest
    newest = 0
    y = b
    while y not in newest_to:             # climb from b until we reach an ancestor of a
        if uf.parent[y] == y:
            return False                  # b's leader reached without meeting a
        if uf.link_time[y] > newest:
            newest = uf.link_time[y]
```

```
        y = uf.parent[y]
    if newest_to[y] > newest:
        newest = newest_to[y]
    return newest <= t


# ---- follow-up 4: tests ----
def test_follow_up_4():
    uf = UnionFind(3)
    for time, a, b in [(1, 0, 1), (5, 1, 2)]:
        uf.union(a, b, time)
    assert not connected_at(uf, 0, 2, 4)       # A and C are apart at time 4 ...
    assert connected_at(uf, 0, 2, 5)           # ... and together at time 5
    assert connected_at(uf, 1, 1, 0)


# ======================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Have weighted/path-compressed union-find ready and warm.
Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.
Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 9 OF 30 |

# Binary search on a yes/no condition

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**3**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Binary search on a yes/no condition | 3 questions | tap a title to jump to it**

**READ FIRST: the code template for these 3 questions (tap here) >**


H 18. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — **1Point3Acres 2026-10-09**

H 19. Find the median of two sorted arrays. — **Exponent no date**

H 20. Find longest duplicated substring — **PracHub 2025-12-08**

CODE TEMPLATE | GROUP 9 OF 30 | 3 QUESTIONS, ONE PATTERN

# Binary search on a yes/no condition

## The one idea these questions share

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_true(lo, hi, ok):                    # smallest x in [lo, hi) with ok(x); hi if none
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):                            # CHANGE: the yes/no test
            hi = mid
        else:
            lo = mid + 1
    return lo
```

## 2. Helper functions that usually get written next to it

```
def last_true(lo, hi, ok):                     # largest x in [lo, hi] with ok(x); lo - 1 if none
    return first_true(lo, hi + 1, lambda x: not ok(x)) - 1


def ceil_div(a, b):
    return -(-a // b)
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **18** | Median of Two Sorted Arrays (LC 4) with a Follow-up Variant | x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest |
| **19** | Find the median of two sorted arrays. | the same partition search on the shorter array |
| **20** | Find longest duplicated substring | x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 18 of 53 | topic 9 of 30: Binary search on a yes/no condition | 1 of 3**

Template fit: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest | asked 11 days ago | frequency Single report

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 18 of 53

# Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/288

## ABOUT THIS QUESTION

**Topic:** Binary search on a yes/no condition

**Optimal answer from:** your app's interview approach (LeetCode #4)

**LeetCode match:** LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

## QUESTION

Given two sorted arrays of numbers, return the median of all their elements together, without merging them into one array. The median is the middle element when the total count is odd, and the average of the two middle elements when it is even. Aim for O(log(min(m, n))) time.

### Function signature

def median_two(a: list, b: list) -> float

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[1, 3], [2]` |
| Output | `2` |
| Why | `Combined [1, 2, 3]: the middle element is 2.` |

| Example 2 | |
|---|---|
| Input | `[1, 2], [3, 4]` |
| Output | `2.5` |
| Why | `Combined [1, 2, 3, 4]: the two middle elements are 2 and 3, so the median is 2.5.` |

| Example 3 | |
|---|---|
| Input | `[], [7, 9]` |
| Output | `8` |
| Why | `One array may be empty: the median is just the other array's median.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O((m + n) log(m + n)) | O(m + n) |

**WHY IT IS SLOW**

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

## OPTIMAL SOLUTION · Binary search on the partition

| TIME | SPACE |
|---|---|
| O(log min(m, n)) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

## FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# Given two sorted arrays of numbers, return the median of
# all their elements together, without merging them into
# one array. The median is the middle element when the
# total count is odd, and the average of the two middle
# elements when it is even. Aim for O(log(min(m, n)))
# time.
#
# Signature:
#   def median_two(a: list, b: list) -> float
#
# Examples:
# [1, 3], [2]  ->  2
# [1, 2], [3, 4]  ->  2.5
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
```

```
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import bisect
```

```
import heapq
import random

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF


def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
    if a < b:
        return a
    return b


# ---- main question: main function ----
# Idea: make a the shorter array. Binary search i, the number of items of
# a in the left half, and take half - i items from b, so that the left
# half holds (m + n + 1) // 2 items. The cut is right when every left
# item is <= every right item; then the median is read off the four items
# next to the cuts.
```

```
# Complexity: Time O(log(min(m, n))), Space O(1)
def median_two(a, b):
    if len(a) > len(b):
        a, b = b, a
    m = len(a)
    n = len(b)
    if m + n == 0:
        raise ValueError("no elements")
    low = 0
    high = m
    half = (m + n + 1) // 2
    while low <= high:
        i = (low + high) // 2
        j = half - i
        a_left, a_right = left_border(a, i), right_border(a, i)
        b_left, b_right = left_border(b, j), right_border(b, j)
        if a_left <= b_right and b_left <= a_right:        # a valid cut
            if (m + n) % 2 == 1:
                return bigger(a_left, b_left)
            return (bigger(a_left, b_left) + smaller(a_right, b_right)) / 2
        if a_left > b_right:
            high = i - 1
        else:
            low = i + 1


# ---- main question: tests ----
def test_main_question():
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5


# ======================================================
# FOLLOW-UP 1: Now there are k sorted arrays instead of two. How do you find the median?
# ======================================================
# ---- follow-up 1: function ----
# Answer: binary search on the ANSWER VALUE. For a candidate x count the
# items that are <= x with one bisect per array (O(k log n)); the k-th
# smallest is the smallest x whose count reaches k.
# Complexity: Time O(k * log n * log(value range)), Space O(k)
def median_k_value(arrays):
    arrays = [a for a in arrays if a]
    total = 0
    for a in arrays:
        total += len(a)
    if total == 0:
        raise ValueError("no elements")
```

```
    def kth(rank):
        low = arrays[0][0]
        high = arrays[0][-1]
        for a in arrays:
            low = smaller(low, a[0])
            high = bigger(high, a[-1])
        while low < high:
            mid = (low + high) // 2
            count = 0
            for a in arrays:
                count += bisect.bisect_right(a, mid)
            if count >= rank:
                high = mid
            else:
```

```
                low = mid + 1
        return low

    if total % 2 == 1:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5


# ========================================================
# FOLLOW-UP 2: What is the heap version and when would you choose it?
# ========================================================
# ---- follow-up 2: function ----
# Answer: a min-heap holds the front item of every array. Pop the
# smallest again and again (like merging) until the middle one or two
# items have been seen. It is simple and fine when k is small or the
# arrays are streams you cannot index.
# Complexity: Time O(N log k) for N elements in k arrays (it stops at the middle), Space O(k)
def median_k_heap(arrays):
    heap = []
    total = 0
    for i in range(len(arrays)):
        total += len(arrays[i])
        if arrays[i]:
            heap.append((arrays[i][0], i, 0))
    if total == 0:
```

```
        raise ValueError("no elements")
    heapq.heapify(heap)
    low_rank = (total - 1) // 2
    high_rank = total // 2
    seen = 0
    middle = []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            heapq.heappush(heap, (arrays[i][j + 1], i, j + 1))
    if total % 2 == 1:
        return middle[0]
    return (middle[0] + middle[1]) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    random.seed(59)
    for round_number in range(300):                    # both versions must agree
        arrays = []
        for k in range(random.randint(1, 5)):
            numbers = []
            for i in range(random.randint(0, 6)):
                numbers.append(random.randint(-9, 9))
            numbers.sort()
            arrays.append(numbers)
        if sum([len(a) for a in arrays]) == 0:
            continue
```

```
        assert median_k_heap(arrays) == median_k_value(arrays)


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

**STUDY PATH: question 19 of 53 | topic 9 of 30: Binary search on a yes/no condition | 2 of 3**

Template fit: the same partition search on the shorter array | no date

**HARD | IN APP LC 4 | Exponent | 19 of 53**

**★ TOP 75: rank 6 in the Medium and Hard priority list**

# ★ Find the median of two sorted arrays.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 3/288

## ABOUT THIS QUESTION

**Topic:** Binary search on a yes/no condition

**Optimal answer from:** your app's interview approach (LeetCode #4)

**LeetCode match:** LC 4 Median of Two Sorted Arrays (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/4322/median-of-two-sorted-arrays

## QUESTION

Given two sorted arrays, return the median of all their elements together. The median is the middle value when the total count is odd and the average of the two middle values when it is even. The best solutions run in O(log(min(m, n))).

**Function signature**

def findMedianSortedArrays(nums1: list, nums2: list) -> float

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[1, 3], [2]` |
| Output | `2` |
| Why | `Combined: 1, 2, 3.` |

| Example 2 | |
|---|---|
| Input | `[1, 2], [3, 4]` |
| Output | `2.5` |
| Why | `Combined: 1, 2, 3, 4: the middle two are 2 and 3.` |

| Example 3 | |
|---|---|
| Input | `[], [5]` |
| Output | `5` |
| Why | `One empty array.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b):
    merged = sorted(a + b)
    n = len(merged)
    if n % 2:
        return merged[n // 2]
    mid = n // 2
    return (merged[mid - 1] + merged[mid]) / 2
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O((m + n) log(m + n)) | O(m + n) |

**WHY IT IS SLOW**

It sorts m + n numbers from scratch, O((m + n) log(m + n)), and stores them all, even though both inputs were already sorted. The partition binary search needs only O(log min(m, n)).

## OPTIMAL SOLUTION · Binary search on the partition

| TIME | SPACE |
|---|---|
| O(log(min(m, n))) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What if there are k sorted arrays instead of two?

Binary search on the answer value instead of on a position. For a candidate value v, count how many elements are <= v in each array with bisect; the median is the smallest v whose total count reaches the middle rank. That costs O(k log n log(range)) time and O(1) extra space and needs no merging.

## FOLLOW-UP 2 What if the arrays are too large for memory?

The two-array binary search only touches O(log n) positions, so with random access (an indexed file or a remote array) it reads almost nothing. If only sequential access is possible, do a streaming selection: a two-pointer merge that stops after (m + n) / 2 elements, using O(1) memory and O(m + n) time.

## FOLLOW-UP 3 Find the k-th smallest element, not just the median.

The same partition search: choose i elements from the shorter array and k - i from the other, and move i until the borders are in order. The median is just k = (m + n + 1) // 2.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the median of two sorted arrays.
#
# Given two sorted arrays, return the median of all their
# elements together. The median is the middle value when
# the total count is odd and the average of the two middle
# values when it is even. The best solutions run in
# O(log(min(m, n))).
#
# Signature:
#   def findMedianSortedArrays(nums1: list, nums2: list) -> float
#
# Examples:
# [1, 3], [2]  ->  2
# [1, 2], [3, 4]  ->  2.5
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if there are k sorted arrays instead of two?
# 2. What if the arrays are too large for memory?
```

```
# 3. Find the k-th smallest element, not just the median.
#
# INTERVIEW NOTES
# Because both arrays are sorted you never need to merge
# them. The median splits the combined data into a left
# half and a right half of equal size; you only have to
# decide how many elements the shorter array contributes
# to the left half. Whether a split is right depends on
# four border values, and moving the split one way or the
# other is a binary search.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_right

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
```

```
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF


def bigger(a, b):
    if a > b:
        return a
    return b


# Complexity: Time O(log(min(m, n))), Space O(1)
def kth_of_two(a, b, k):
    # the k-th smallest (k starts at 1) of two sorted arrays. Take i items
    # from a and k - i items from b as the "left part". The cut is right when
    # every left item is <= every right item, and only the four items next to
    # the two cuts have to be compared. Binary search on i: O(log n).
    # (This also answers follow-up 3: any k-th smallest, not only the median.)
    if len(a) > len(b):
        a, b = b, a
    low = k - len(b)
    if low < 0:
        low = 0
    high = k
    if len(a) < high:
        high = len(a)
    while low <= high:
        i = (low + high) // 2
```

```
        j = k - i
        if left_border(a, i) > right_border(b, j):
            high = i - 1                    # took too many from a
        elif left_border(b, j) > right_border(a, i):
            low = i + 1                     # took too few from a
        else:
            return bigger(left_border(a, i), left_border(b, j))
    raise ValueError("k is out of range")
```

```


# ---- main question: main function ----
# Idea: binary search how many items of the shorter array go into the
# left half. The median is the middle item (odd total) or the mean of
# the two middle items (even total).
# Complexity: Time O(log(min(m, n))), Space O(1)
def findMedianSortedArrays(nums1, nums2):
    total = len(nums1) + len(nums2)
    if total == 0:
        raise ValueError("no data")
    if total % 2 == 1:
        return kth_of_two(nums1, nums2, total // 2 + 1)
    low = kth_of_two(nums1, nums2, total // 2)
    high = kth_of_two(nums1, nums2, total // 2 + 1)
    return (low + high) / 2


# ---- main question: tests ----
def test_main_question():
    assert findMedianSortedArrays([1, 3], [2]) == 2
    assert findMedianSortedArrays([1, 2], [3, 4]) == 2.5
    assert findMedianSortedArrays([], [5]) == 5


# =====================================================
# FOLLOW-UP 1: What if there are k sorted arrays instead of two?
# =====================================================
# ---- follow-up 1: helper functions ----
def count_at_most(arrays, v):
    # how many items are <= v in all the arrays together
    total = 0
    for arr in arrays:
        total += bisect_right(arr, v)
    return total


# ---- follow-up 1: function ----
# Answer: binary search on the ANSWER VALUE instead of on a position. For
# a candidate value v, count with bisect how many items are <= v in each
# array; the k-th smallest is the smallest v with at least k items <= v.
# (Whole-number values.)
# Complexity: Time O(k * log n * log(value range)) for k arrays, Space O(k)
def kth_of_many(arrays, k):
    arrays = [arr for arr in arrays if arr]
```

```
    low = arrays[0][0]
    high = arrays[0][-1]
    for arr in arrays:
        if arr[0] < low:
            low = arr[0]
        if arr[-1] > high:
            high = arr[-1]
    while low < high:
        mid = (low + high) // 2
        if count_at_most(arrays, mid) >= k:
```

```
            high = mid
        else:
            low = mid + 1
    return low


# Complexity: Time O(k * log n * log(value range)), Space O(k)
def median_of_many(arrays):
    total = 0
    for arr in arrays:
        total += len(arr)
    if total % 2 == 1:
        return kth_of_many(arrays, total // 2 + 1)
    return (kth_of_many(arrays, total // 2) + kth_of_many(arrays, total // 2 + 1)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    arrays = [[1, 3], [2], [4, 5, 6]]
    assert median_of_many(arrays) == 3.5
    assert kth_of_many(arrays, 4) == 4


# ============================================================
# FOLLOW-UP 2: What if the arrays are too large for memory?
# ============================================================
# Answer: use the binary search on the VALUE above: it only needs "how
# many items are <= v", which can be computed with a binary search on
# each sorted file (random access) or one streaming pass per probe. The
# data never has to be loaded.


# ============================================================
# FOLLOW-UP 3: Find the k-th smallest element, not just the median
# ============================================================
```

```
# ---- follow-up 3: tests ----
# Answer: kth_of_two (above) is exactly that; the median is k = (m + n) // 2 + 1.
def test_follow_up_3():
    assert kth_of_two([1, 3, 5], [2, 4, 6], 4) == 4
    assert kth_of_two([1, 3, 5], [], 2) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**OFFICIAL PAGE TEXT**

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade now

**STUDY PATH: question 20 of 53 | topic 9 of 30: Binary search on a yes/no condition | 3 of 3**

Template fit: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok | asked 316 days ago

HARD | IN APP LC 1044 | PracHub | Technical Screen | 20 of 53

# Find longest duplicated substring

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 142/288

## ABOUT THIS QUESTION

**Topic:** Binary search on a yes/no condition

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #1044 Longest Duplicate Substring

**LeetCode match:** LC 1044 Longest Duplicate Substring

Posted on PracHub: 2025-12-08 (updated 2026-07-09)

**Original:** https://prachub.com/coding-questions/find-longest-duplicated-substring

## QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

### Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

### Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**
- No duplicated substring, so output should be **`""`** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `'banana'` |
| **Output** | `'ana'` |
| **Why** | `'ana' occurs twice (overlapping is allowed).` |

| Example 2 | |
|---|---|
| **Input** | `'abcd'` |
| **Output** | `''` |
| **Why** | `No substring repeats.` |

## QUESTION

| Example 3 | |
|---|---|
| Input | `'aaaaa'` |
| Output | `'aaaa'` |
| Why | `'aaaa' appears at positions 0 and 1.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for L in range(n - 1, 0, -1):
        seen = set()
        for i in range(n - L + 1):
            sub = s[i:i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n^2) |

### WHY IT IS SLOW

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

## OPTIMAL SOLUTION · Binary search on the length with a rolling hash (Rabin-Karp)

| TIME | SPACE |
|---|---|
| O(n log n) expected | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

## FOLLOW-UP 2 The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

## FOLLOW-UP 3 The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. **Cost:** O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find longest duplicated substring
#
# You are given a string s consisting of lowercase English
# letters. A **substring** of s is any contiguous sequence
# of characters, i.e., s[i..j) for 0 <= i < j <= len(s).
# Design an algorithm to find **any one** of the longest
# substrings that appears **at least twice** in s. The two
# occurrences may overlap. If no substring appears at
# least twice, return the empty string. ### Requirements -
# Input: a string s of length n (e.g., 1 <= n <= 2 × 10^5).
# - Output: a string representing one of the longest
# duplicated substrings. - Aim for an algorithm
# substantially better than the naive O(n^2) substring
# comparison approach. ### Examples - s = "banana" -
# Possible duplicated substrings: "a", "an", "ana" -
# Longest length is 3 ("ana"), so acceptable outputs
# include "ana". - s = "abcd" - No duplicated substring,
# so output should be "" (empty string).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Avoid hash collisions entirely.
# 2. The substring must appear at least k times, or must
# not overlap.
# 3. The string is huge and does not fit in memory.
#
# INTERVIEW NOTES
# If a substring of length L repeats, then a substring of
# every shorter length repeats too, so 'is there a
# duplicate of length L' is monotone and the answer length
# can be binary searched. Each check slides a window of
# length L over the string with a rolling hash and looks
# for a repeated hash, verifying the match to guard
# against collisions.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
MOD = (1 << 61) - 1          # a big prime: hash values stay small
BASE = 131


def find_length(s, length):
    # is there a substring of this length that appears twice? Return one, or None.
    # Rolling hash: the hash of every window is updated in O(1) when the
    # window slides by one character.
    codes = []
    for c in s:
        codes.append(ord(c))
    power = pow(BASE, length, MOD)                  # BASE ** length, used to remove the left character
    h = 0
```

```
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}                                  # hash -> start positions of windows with it
    for i in range(1, len(s) - length + 1):
        h = (h * BASE - codes[i - 1] * power + codes[i + length - 1]) % MOD
        if h in seen:
            for j in seen[h]:
                if s[j:j + length] == s[i:i + length]:    # equal hashes may be a collision: compare the te
xt
                    return s[i:i + length]
        else:
            seen[h] = []
        seen[h].append(i)
    return None


# ---- main question: main function ----
# Idea: if a substring of length L appears twice, so does every shorter
# one, so binary search the length. For each length find out whether a
# repeat exists with the rolling hash (Rabin-Karp) above.
# Complexity: Time O(n log n) expected (a rolling hash for each of log n lengths), Space O(n)
def longest_dup(s):
    low = 1
    high = len(s) - 1
    best = ""
    while low <= high:
        mid = (low + high) // 2
        found = find_length(s, mid)
        if found:
            best = found
            low = mid + 1                 # a repeat exists: try longer
        else:
            high = mid - 1
```

```
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_dup("banana") == "ana"
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4


# ======================================================
# FOLLOW-UP 1: Avoid hash collisions entirely
# ======================================================
# ---- follow-up 1: function ----
# Answer: use a suffix array with the LCP (longest common prefix) of
# neighbours: sort all suffixes; the longest duplicate is the largest
# common prefix between two neighbours in that order. Sorting the
# suffixes directly is O(n^2 log n); real suffix array algorithms are
# O(n log n) or O(n).
# Complexity: Time O(n^2 log n) (sorting the suffixes), Space O(n^2)
def longest_dup_suffix(s):
    suffixes = []
    for i in range(len(s)):
        suffixes.append(s[i:])
    suffixes.sort()
    best = ""
    for index in range(len(suffixes) - 1):
        a = suffixes[index]
        b = suffixes[index + 1]
        k = 0
        while k < len(a) and k < len(b) and a[k] == b[k]:
```

```
            k += 1
        if k > len(best):
            best = a[:k]
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        assert len(longest_dup(s)) == len(longest_dup_suffix(s))
```

```
# ======================================================
# FOLLOW-UP 2: The substring must appear at least k times, or must not overlap
# ======================================================
# ---- follow-up 2: function ----
# Answer: for "at least k times", count the windows with each text (need
# k of them). For "not overlapping", count only start positions that are
# at least `length` apart (greedy from the left). This simple version
# lists every window of each length instead of hashing.
# Complexity: Time O(n^3) in the worst case, Space O(n^2)
def longest_repeated(s, times=2, non_overlapping=False):
    best = ""
    n = len(s)
    for length in range(1, n):
        spots = {}                                  # window text -> start positions
        for i in range(n - length + 1):
            piece = s[i:i + length]
            if piece not in spots:
                spots[piece] = []
            spots[piece].append(i)
        found = None
        for text in spots:
            starts = spots[text]
            if non_overlapping:
                taken = 0
                last = -length
                for p in starts:
                    if p - last >= length:
                        taken += 1
                        last = p
            else:
                taken = len(starts)
            if taken >= times:
                found = text
                break
        if found is None:
            break                                   # no repeat of this length, so none of any longer len
gth
        best = found
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_repeated("banana", 3) == "a"             # three occurrences needed
    assert len(longest_repeated("aaaa", 2, True)) == 2      # "aa" twice without overlapping
    assert len(longest_repeated("aaaa", 2)) == 3            # overlapping copies are allowed
```

```
# ======================================================
# FOLLOW-UP 3: The string is huge and does not fit in memory
```

```
# ============================================================
# Answer: the binary search on the length still works; for each length
# stream the string once and keep only the hashes (a Bloom filter or a
# hash set on disk), then verify the candidate positions in a second pass.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(s) -> str:
```

### Sample call

```
solution('banana')
```

### Test cases

| Input | Expected |
|---|---|
| `('banana',)` | `'ana'` |
| `('abcd',)` | `''` |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 10 OF 30

# Sets and counters: remember the first collection, check the second against it

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Sets and counters: remember the first collection, check the second against it | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 21. Smallest Missing Positive Integer in Linear Time and Constant Extra Space **PracHub 2026-09-09**

CODE TEMPLATE | GROUP 10 OF 30 | 1 QUESTIONS, ONE PATTERN

# Sets and counters: remember the first collection, check the second against it

## The one idea these questions share

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def keep_not_in(first, second):
    seen = set(first)                          # CHANGE 1: a set, or a Counter if counts matter
    return [x for x in second if x not in seen]     # CHANGE 2: keep, count, or stop at the first hit
```

## 2. Helper functions that usually get written next to it

```
def common_counts(a, b):                   # how many of each value the two lists share
    ca, cb = Counter(a), Counter(b)
    return {v: min(ca[v], cb[v]) for v in ca if v in cb}


def first_repeat(items):
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **21** | Smallest Missing Positive Integer in Linear Time and Constant Extra Space | use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 21 of 53 | topic 10 of 30: Sets and counters: remember the first collection, check the second against it | 1 of 1**

Template fit: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer | asked 41 days ago

HARD | IN APP LC 41 | PracHub | Technical Screen | 21 of 53

# Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 48 | 30 DAYS 0 | PRIORITY 84/288

## ABOUT THIS QUESTION

**Topic:** Sets and counters: remember the first collection, check the second against it

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #41 First Missing Positive

**LeetCode match:** LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

**Original:** https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

## QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

### Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

### Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

### Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

### Examples

### Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

### Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

### Example 3

## QUESTION

- Input: **`nums = [1, 2, 2, 3]`**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

### WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

## OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

## FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

## FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
```

```
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# Find the smallest positive integer that is missing from the array.
```

```
# Idea (cyclic placement): the answer is between 1 and n + 1, so only
# values in 1..n matter. Put each value v at index v - 1 by swapping: for
# each position, while the value there is in 1..n and not yet at its home
# index, swap it home. Every swap puts one value in its final place, so there
# are at most n swaps in total: O(n) time, O(1) extra space. Then the first
# index that does not hold index + 1 gives the answer.
# Complexity: Time O(n) (every value moves at most once to its own place), Space O(1) extra (plus the copy)
def first_missing_positive(nums):
    a = list(nums)                                  # work on a copy: the input stays untouched
    n = len(a)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            home = a[i] - 1
            a[i], a[home] = a[home], a[i]
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# the easy version (say it first): O(n) extra memory
# Complexity: Time O(n), Space O(n)
```

```
def first_missing_set(nums):
    seen = set(nums)
    v = 1
    while v in seen:
        v += 1
    return v


# ---- main question: tests ----
CASES = [([3, 4, -1, 1], 2), ([1, 2, 0], 3), ([7, 8, 9, 11, 12], 1), ([], 1), ([1, 1, 1], 2), ([2, 1], 3)]


def test_main_question():
    for nums, expected in CASES:
        assert first_missing_positive(nums) == expected
        assert first_missing_set(nums) == expected
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]              # the input list is left untouched


# ========================================================
# FOLLOW-UP 1: Do it without swapping (sign marking)
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: replace the values outside 1..n by n + 1 (they cannot matter).
# Then for every value v in 1..n make the entry at index v - 1 NEGATIVE as a
# "seen" flag. The first entry that is still positive marks the missing
# number. Same O(n) time and O(1) extra space.
# Complexity: Time O(n), Space O(1) extra (plus the copy)
def first_missing_marking(nums):
    a = list(nums)
    n = len(a)
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for nums, expected in CASES:
        assert first_missing_marking(nums) == expected


# ========================================================
# FOLLOW-UP 2: The array must not be modified
# ========================================================
# Answer: then you cannot swap or mark in the array itself. Copy it (O(n)
# memory, as above), or use the set version, or binary-search the answer when
# you may count: for a guess g, count how many values are in 1..g.


# ========================================================
# FOLLOW-UP 3: Smallest missing non-negative integer, or the k-th missing positive
# ========================================================
# ---- follow-up 3: function ----
```

```
# Answer: for the smallest missing non-negative number start the search at 0
# instead of 1. For the k-th missing positive in a SORTED array: the number of
# missing values before index i is a[i] - (i + 1), which only grows, so binary
# search the first index where it reaches k.
```

```
# Complexity: Time O(log n), Space O(1)
def kth_missing_positive(sorted_nums, k):
    low = 0
    high = len(sorted_nums)
    while low < high:
        middle = (low + high) // 2
        if sorted_nums[middle] - (middle + 1) < k:
            low = middle + 1
        else:
            high = middle
    return low + k


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9      # the missing ones: 1, 5, 6, 8, 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

### Sample call

```
first_missing_positive([4, -2, 1, 2])
```

### Test cases

| Input | Expected |
|---|---|
| ([4, -2, 1, 2],) | 3 |
| ([5, 6, 100, -3],) | 1 |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 11 OF 30

# bisect on a sorted list: find a position, count a range, keep it sorted

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**bisect on a sorted list: find a position, count a range, keep it sorted | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 22. Find Maximum Envelope Nesting — **PracHub 2026-03-18**

H 23. Count of Smaller Numbers After Self — **HackTheRounds no date**

CODE TEMPLATE | GROUP 11 OF 30 | 2 QUESTIONS, ONE PATTERN

# bisect on a sorted list: find a position, count a range, keep it sorted

## The one idea these questions share

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from bisect import bisect_left, bisect_right, insort


def count_in_range(sorted_list, low, high):
    hi = bisect_right(sorted_list, high)       # CHANGE: which positions you read
    lo = bisect_left(sorted_list, low)
    return hi - lo


# insort(sorted_list, x)                    keeps the list sorted as items arrive
# sorted_list[i]                            rank queries: i-th smallest, sorted_list[-1 - k] = k-th largest
```

## 2. Helper functions that usually get written next to it

```
def lis_length(values):                        # the tails list for "longest increasing"
    tails = []
    for v in values:
        i = bisect_left(tails, v)
        if i == len(tails):
            tails.append(v)
        else:
            tails[i] = v
    return len(tails)


def nearest(sorted_list, x):
    i = bisect_left(sorted_list, x)
    near = [sorted_list[j] for j in (i - 1, i) if 0 <= j < len(sorted_list)]
    return min(near, key=lambda v: abs(v - x))
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **22** | Find Maximum Envelope Nesting | exactly the same sort and tails list |
| **23** | Count of Smaller Numbers After Self | go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version) |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 22 of 53 | topic 11 of 30: bisect on a sorted list: find a position, count a range, keep it sorted | 1 of 2**

Template fit: exactly the same sort and tails list | asked 216 days ago

HARD | IN APP LC 354 | PracHub | Technical Screen | 22 of 53

★ TOP 75: rank 14 in the Medium and Hard priority list

# ★ Find Maximum Envelope Nesting

POSTED 2026-03-18 | 216 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 33/288

## ABOUT THIS QUESTION

**Topic:** bisect on a sorted list: find a position, count a range, keep it sorted

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #354 Russian Doll Envelopes

**LeetCode match:** LC 354 Russian Doll Envelopes

Posted on PracHub: 2026-03-18 (updated 2026-05-06)

**Original:** https://prachub.com/coding-questions/find-maximum-envelope-nesting

## QUESTION

Envelopes are given as (width, height). One envelope fits inside another only when both its width and its height are strictly smaller. Return the largest number of envelopes that can be nested one inside the next (Russian dolls).

### Function signature

def max_envelopes(envelopes: list) -> int

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[(5, 4), (6, 4), (6, 7), (2, 3)]` |
| Output | `3` |
| Why | `(2, 3) fits in (5, 4), which fits in (6, 7): 3 nested.` |

| Example 2 | |
|---|---|
| Input | `[(1, 1), (1, 1), (1, 1)]` |
| Output | `1` |
| Why | `Equal envelopes cannot go inside each other.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**WHY IT IS SLOW**

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

## OPTIMAL SOLUTION · Sort by width ascending and height descending, then LIS by binary search

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

## FOLLOW-UP 2 Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

## FOLLOW-UP 3 Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Maximum Envelope Nesting
#
# Envelopes are given as (width, height). One envelope
# fits inside another only when both its width and its
# height are strictly smaller. Return the largest number
# of envelopes that can be nested one inside the next
# (Russian dolls).
#
# Signature:
#   def max_envelopes(envelopes: list) -> int
#
# Examples:
# [(5, 4), (6, 4), (6, 7), (2, 3)]  ->  3
# [(1, 1), (1, 1), (1, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual chain of envelopes.
# 2. Three dimensions (boxes).
# 3. Non-strict nesting (equal sizes allowed).
#
# INTERVIEW NOTES
```

```
# Two dimensions are hard, one is easy. Sort by width so
# that width order is settled, then the question becomes a
# longest strictly increasing subsequence of heights. The
# trap is envelopes with equal width: they cannot nest, so
# sort equal widths by height descending, which prevents
# two of them from forming an increasing pair. The LIS is
# then done with patience sorting (binary search over
# tails).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by width ascending and, for equal widths, height descending.
# Now a strictly increasing run of heights is a valid nesting chain: two
# envelopes of the same width are in decreasing height order, so they can
# never both be in an increasing run. Find the longest strictly increasing
# run of heights (LIS) with a binary search in O(n log n): tails[k] is the
# smallest possible last height of a run of length k + 1.
# Complexity: Time O(n log n), Space O(n)
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)          # longer than any run so far
        else:
            tails[i] = height             # a smaller ending for runs of length i + 1
    return len(tails)


# ---- main question: tests ----
def test_main_question():
    assert max_envelopes([[5, 4], [6, 4], [6, 7], [2, 3]]) == 3
    assert max_envelopes([]) == 0
    assert max_envelopes([[3, 3], [3, 3], [4, 4]]) == 2        # equal sizes do not nest (strict)


# ============================================================
# FOLLOW-UP 1: Return the actual chain of envelopes
# ============================================================
# ---- follow-up 1: function ----
# Answer: remember, for each element, which element it extended (a parent
# link), and rebuild the chain from the last tail of the longest run.
# The sorting and the binary search stay the same.
# Complexity: Time O(n log n), Space O(n)
def envelope_chain(envelopes):
```

```
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    tail_index = []                         # the index in `ordered` of each tail
    previous = [-1] * len(ordered)
    for i in range(len(ordered)):
```

```
        height = ordered[i][1]
        k = bisect_left(tails, height)
        if k == len(tails):
            tails.append(height)
            tail_index.append(i)
        else:
            tails[k] = height
            tail_index[k] = i
        if k > 0:
            previous[i] = tail_index[k - 1]        # it extends the run that ended one shorter
    chain = []
    if tail_index:
        i = tail_index[-1]
        while i != -1:
            chain.append(ordered[i])
            i = previous[i]
    chain.reverse()
    return chain


# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = envelope_chain([[5, 4], [6, 4], [6, 7], [2, 3]])
    assert len(chain) == 3
    for i in range(len(chain) - 1):                    # every envelope fits strictly inside the next
        assert chain[i][0] < chain[i + 1][0] and chain[i][1] < chain[i + 1][1]
    assert envelope_chain([]) == []


# ========================================================
# FOLLOW-UP 2: Three dimensions (boxes)
# ========================================================
# ---- follow-up 2: function ----
# Answer: sorting plus LIS no longer reduces the problem. Use an O(n^2)
# DP (best[i] = longest chain ending at box i), or CDQ divide and conquer
# with a Fenwick tree for O(n log^2 n).
# Complexity: Time O(n^2), Space O(n)
def max_boxes_3d(boxes):
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a = ordered[j]
            b = ordered[i]
            if a[0] < b[0] and a[1] < b[1] and a[2] < b[2]:
```

```
                if best[j] + 1 > best[i]:
                    best[i] = best[j] + 1
    answer = 0
    for length in best:
        if length > answer:
            answer = length
    return answer


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert max_boxes_3d([(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]) == 3
    assert max_boxes_3d([]) == 0
```

```
# ============================================================
# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed)
# ============================================================
# ---- follow-up 3: function ----
# Answer: sort by width AND height ascending, and use bisect_right (a
# longest NON-decreasing run) instead of bisect_left.
# Complexity: Time O(n log n), Space O(n)
def max_nested_non_strict(envelopes):
    tails = []
    for width, height in sorted(envelopes):
        i = bisect_right(tails, height)
        if i == len(tails):
            tails.append(height)
        else:
            tails[i] = height
    return len(tails)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert max_nested_non_strict([[3, 3], [3, 3], [4, 4]]) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

### Test cases

| Input | Expected |
|---|---|
| `[[5, 4], [6, 4], [6, 7], [2, 3]]` | `3` |
| `[]` | `0` |

**ALSO REPORTED · 1 more**

**Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report**

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

**ALSO REPORTED · 1 more**

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

**STUDY PATH: question 23 of 53 | topic 11 of 30: bisect on a sorted list: find a position, count a range, keep it sorted | 2 of 2**

Template fit: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version) | no date

HARD | IN APP LC 315 | HackTheRounds | Phone Screen | 23 of 53

# Count of Smaller Numbers After Self

STUDY: TALK ALL TIME 33.2 30 DAYS 0 PRIORITY 131/288

## ABOUT THIS QUESTION

**Topic:** bisect on a sorted list: find a position, count a range, keep it sorted

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #315 Count of Smaller Numbers After Self

**LeetCode match:** LC 315

## QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of elements to the right of nums[i] that are strictly smaller than nums[i].

**Function signature**

def count_smaller_after(nums: list) -> list

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[5, 2, 6, 1]` |
| Output | `[2, 1, 1, 0]` |
| Why | `5 has two smaller numbers to its right (2 and 1), 2 has one (1), 6 has one (1), 1 has none.` |

| Example 2 | |
|---|---|
| Input | `[-1]` |
| Output | `[0]` |
| Why | `A single number has nothing to its right.` |

| Example 3 | |
|---|---|
| Input | `[-1, -1]` |
| Output | `[0, 0]` |
| Why | `Equal numbers are not smaller.` |

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def brute(nums):
    counts = []
    for i in range(len(nums)):
        smaller = 0
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[i]:
                smaller += 1
        counts.append(smaller)
    return counts
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) extra |

**WHY IT IS SLOW**

Two nested loops, O(n^2): too slow at n = 10^5. A Fenwick tree over ranks scanned right to left (or merge sort) is O(n log n).

## OPTIMAL SOLUTION · Fenwick tree over compressed values, scanning right to left

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Do it with merge sort instead of a Fenwick tree.

Sort indices by value with merge sort; whenever you place an element from the left half, add the number of right-half elements already placed (they are smaller). Same O(n log n), and it avoids coordinate compression.

## FOLLOW-UP 2 Count larger numbers after self, or smaller numbers before self.

Larger after: query the suffix instead of the prefix of the Fenwick tree. Smaller before: scan left to right instead of right to left. **Cost:** still O(n log n) with the Fenwick tree.

## FOLLOW-UP 3 Count inversions in an array.

The sum of the answers above is the number of inversions; merge sort counts them directly in O(n log n).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count of Smaller Numbers After Self
#
# Given an integer array nums, return an array counts
# where counts[i] is the number of elements to the right
# of nums[i] that are strictly smaller than nums[i].
#
# Signature:
#   def count_smaller_after(nums: list) -> list
#
# Examples:
# [5, 2, 6, 1]  ->  [2, 1, 1, 0]
```

```
# [-1]  ->  [0]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with merge sort instead of a Fenwick tree.
# 2. Count larger numbers after self, or smaller numbers
# before self.
# 3. Count inversions in an array.
#
# INTERVIEW NOTES
# For each element you want the count of smaller elements
# to its right: an order-statistics query on the elements
# seen so far if you scan right to left. A Fenwick tree
# over compressed values gives rank counts in O(log n).
# The same count falls out of merge sort: when a left
# element is placed after right elements that are smaller,
# those right elements are exactly the smaller numbers
# after it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Fenwick:
    # a Fenwick tree (binary indexed tree): add(i) counts one more value with
```

```
    # rank i, prefix(i) tells how many counted values have a rank below i.
    # Both take O(log size).
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i                  # jump to the next block that covers i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i                  # jump to the previous block
        return total


def ranks(nums):
    # coordinate compression: give each distinct value a rank 0, 1, 2, ... by size
    rank = {}
    for i, v in enumerate(sorted(set(nums))):
        rank[v] = i
    return rank


# ---- main question: main function ----
# Idea: compress the values to ranks and scan from the RIGHT. The answer
```

```
# for x is how many already-seen values have a smaller rank (one prefix
# query on the Fenwick tree); then x itself is added to the tree.
# Complexity: Time O(n log n), Space O(n)
def count_smaller_after(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    for x in reversed(nums):
        answers.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    answers.reverse()
    return answers


# ---- main question: tests ----
```

```
def test_main_question():
    assert count_smaller_after([5, 2, 6, 1]) == [2, 1, 1, 0]
    assert count_smaller_after([-1]) == [0]
    assert count_smaller_after([-1, -1]) == [0, 0]
    assert count_smaller_after([]) == []


# ======================================================
# FOLLOW-UP 1: Do it with merge sort instead of a Fenwick tree
# ======================================================
# ---- follow-up 1: function ----
# Answer: merge-sort the INDEXES by value. While merging, whenever an item
# of the left half is placed, every item of the right half that was
# already placed is smaller than it and comes later in the array, so add
# that number to its answer. Same O(n log n), and no coordinate compression.
# Complexity: Time O(n log n), Space O(n)
def count_smaller_merge(nums):
    counts = [0] * len(nums)
    order = list(range(len(nums)))

    def sort(lo, hi):
        if hi - lo <= 1:
            return
        mid = (lo + hi) // 2
        sort(lo, mid)
        sort(mid, hi)
        merged = []
        i = lo
        j = mid
        while i < mid or j < hi:
            take_left = False
            if j >= hi:
                take_left = True
            elif i < mid and nums[order[i]] <= nums[order[j]]:
                take_left = True
            if take_left:
                counts[order[i]] += j - mid        # right items already placed are smaller
                merged.append(order[i])
                i += 1
            else:
                merged.append(order[j])
                j += 1
        order[lo:hi] = merged

    sort(0, len(nums))
```

```
    return counts
```

```python
# ---- follow-up 1: tests ----
def brute_smaller_after(nums):
    answers = []
    for i in range(len(nums)):
        count = 0
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[i]:
                count += 1
        answers.append(count)
    return answers


def test_follow_up_1():
    assert count_smaller_merge([5, 2, 6, 1]) == [2, 1, 1, 0]
    random.seed(31)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 12)):
            nums.append(random.randint(-5, 5))
        assert count_smaller_merge(nums) == brute_smaller_after(nums)
        assert count_smaller_after(nums) == brute_smaller_after(nums)


# ======================================================
# FOLLOW-UP 2: Count larger numbers after self, or smaller numbers before self
# ======================================================
# ---- follow-up 2: functions ----
# Answer: larger after: ask for the part of the tree ABOVE the rank instead
# of below it. Smaller before: scan from the left instead of the right.
# Both are still O(n log n).
# Complexity: Time O(n log n), Space O(n)
def count_larger_after(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    seen = 0
    for x in reversed(nums):
        answers.append(seen - tree.prefix(rank[x] + 1))    # seen values minus those <= x
        tree.add(rank[x])
        seen += 1
    answers.reverse()
    return answers
```

```python


# Complexity: Time O(n log n), Space O(n)
def count_smaller_before(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    for x in nums:
        answers.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return answers


# ---- follow-up 2: tests ----
def test_follow_up_2():
    sample = [5, 2, 6, 1]
    assert count_larger_after(sample) == [1, 1, 0, 0]
    assert count_smaller_before(sample) == [0, 0, 2, 0]
    random.seed(32)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 12)):
```

```
            nums.append(random.randint(-5, 5))
        larger = []
        smaller_before = []
        for i in range(len(nums)):
            bigger_after = 0
            for y in nums[i + 1:]:
                if y > nums[i]:
                    bigger_after += 1
            larger.append(bigger_after)
            before = 0
            for y in nums[:i]:
                if y < nums[i]:
                    before += 1
            smaller_before.append(before)
        assert count_larger_after(nums) == larger
        assert count_smaller_before(nums) == smaller_before


# ============================================================
# FOLLOW-UP 3: Count inversions in an array
# ============================================================
# ---- follow-up 3: function ----
# Answer: an inversion is a pair i < j with nums[i] > nums[j]. The
```

```
# answers of count_smaller_after add up to exactly that number.
# Complexity: Time O(n log n), Space O(n)
def inversions(nums):
    total = 0
    for count in count_smaller_merge(nums):
        total += count
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert inversions([2, 4, 1, 3, 5]) == 3
    assert inversions([3, 2, 1]) == 3
    assert inversions([]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 12 OF 30

# Sorted intervals: merge while they touch, then read the gaps or search

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Sorted intervals: merge while they touch, then read the gaps or search | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 24. Find safe travel intervals between planet influences — **PracHub 2025-11-22**

H 25. Implement a function that processes a stream of interval data. — **Exponent no date**

CODE TEMPLATE | GROUP 12 OF 30 | 2 QUESTIONS, ONE PATTERN

# Sorted intervals: merge while they touch, then read the gaps or search

## The one idea these questions share

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:            # CHANGE 1: overlap or touch? (< to keep touching ones apart)
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

## 2. Helper functions that usually get written next to it

```
from bisect import bisect_left, bisect_right


def gaps(merged, lo=float("-inf"), hi=float("inf")):
    edges = [lo] + [x for iv in merged for x in iv] + [hi]
    return [(edges[i], edges[i + 1]) for i in range(0, len(edges), 2)]


def containing(merged, x):
    starts = [s for s, _ in merged]
    i = bisect_right(starts, x) - 1
    return tuple(merged[i]) if i >= 0 and merged[i][1] >= x else None
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **24** | Find safe travel intervals between planet influences | turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends) |
| **25** | Implement a function that processes a stream of interval data. | the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 24 of 53 | topic 12 of 30: Sorted intervals: merge while they touch, then read the gaps or search | 1 of 2**

Template fit: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends) | asked 332 days ago

**HARD | PracHub | Technical Screen | 24 of 53**

**★ TOP 75: rank 10 in the Medium and Hard priority list**

# ★ Find safe travel intervals between planet influences

**POSTED 2025-11-22 | 332 DAYS AGO | OLDER**

**STUDY: TYPE** **ALL TIME 65.2** **30 DAYS 41.1** **PRIORITY 10/288**

## ABOUT THIS QUESTION

**Topic:** Sorted intervals: merge while they touch, then read the gaps or search

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #56 Merge Intervals

**LeetCode match:** LC 56 Merge Intervals (then take gaps)

Posted on PracHub: 2025-11-22 (updated 2026-05-23)

**Original:** https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

## QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **`c`** is the coordinate of the planet's center on the x-axis.
- **`r`** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [ |x - c_i| \le r_i \quad \text{for some planet } i ] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

## QUESTION

Constraints

0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

### Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(planets):
    pts = set()
    for c, r in planets:
        pts |= {c - r, c + r}
    bounds = [-INF] + sorted(pts) + [INF]
    safe = []
    for a, b in zip(bounds, bounds[1:]):
        mid = midpoint(a, b)
        if not any(abs(mid - c) < r
                   for c, r in planets):
            safe.append((a, b))
    return safe
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

## OPTIMAL SOLUTION · Merge the unsafe intervals, then read off the gaps

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

**FOLLOW-UP 2 Are the endpoints safe?**

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

**FOLLOW-UP 3 Planets arrive over time.**

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find safe travel intervals between planet
# influences
#
# You are planning a space route along a one-dimensional
# line (the x-axis). You are given a list of planets. Each
# planet is represented by an integer pair \((c, r)\): - c
# is the coordinate of the planet's center on the x-axis.
# - r is a non-negative integer radius of its
# gravitational influence. A point x on the x-axis is
# **unsafe** for a space traveler if it lies within the
# gravitational influence of at least one planet: \[ |x -
# c_i| \le r_i \quad \text{for some planet } i \]
# Equivalently, each planet makes the interval \([c_i -
# r_i,\; c_i + r_i]\) unsafe. Task: - Consider the entire
# real line as the possible travel path. - Compute all
# **maximal continuous intervals** on this line where the
# traveler is **safe**, i.e., points not covered by any
# planet's unsafe interval. - Return these safe intervals
# as a sorted list of disjoint intervals.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the part between two given positions matters (a
# route from A to B).
# 2. Are the endpoints safe?
# 3. Planets arrive over time.
#
# INTERVIEW NOTES
# The safe points are the complement of the union of the
# unsafe intervals. Merge the unsafe intervals first (sort
# by start, extend while they overlap or touch), and the
# safe intervals are exactly the gaps between consecutive
# merged intervals plus the two unbounded ends. Endpoints
# themselves are unsafe because the influence is closed
# (<=).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

INF = float("inf")
```

```
# ============================================================
# MAIN QUESTION
```

```
# ========================================================
# ---- main question: helper functions ----
def unsafe_blocks(planets):
    # planet (c, r) makes the stretch [c - r, c + r] unsafe; merge overlapping stretches
    spans = []
    for c, r in planets:
        spans.append((c - r, c + r))
    spans.sort()
    blocks = []
    for start, end in spans:
        if blocks and start <= blocks[-1][1]:            # sharing even one point joins two blocks
            if end > blocks[-1][1]:
                blocks[-1][1] = end
        else:
            blocks.append([start, end])
    return blocks


def gaps_between(blocks):
    # the open intervals between neighbouring blocks, and the two infinite ends
    if not blocks:
        return [(-INF, INF)]
    safe = [(-INF, blocks[0][0])]
    for i in range(len(blocks) - 1):
        safe.append((blocks[i][1], blocks[i + 1][0]))
    safe.append((blocks[len(blocks) - 1][1], INF))
    return safe


# ---- main question: main function ----
# Idea: sort the unsafe stretches by start and merge the ones that overlap or
# touch (they share a point, and that point is unsafe). The safe
# intervals are the gaps between the merged blocks, plus the two infinite ends.
# Complexity: Time O(n log n) for n planets, Space O(n)
def safe_intervals(planets):
    return gaps_between(unsafe_blocks(planets))


# ---- main question: tests ----
def test_main_question():
    planets = [(0, 2), (10, 1), (3, 1)]                  # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9), (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
```

```
    assert safe_intervals([(1, 1), (3, 1)]) == [(-INF, 0), (4, INF)]      # touching blocks [0, 2] and [2,
4] merge


# ========================================================
# FOLLOW-UP 1: Only the part between two given positions matters (a route from A to B)
# ========================================================
# ---- follow-up 1: function ----
# Answer: cut every safe interval to [A, B] and keep the non-empty pieces.
# Complexity: Time O(n log n), Space O(n)
def safe_between(planets, a, b):
    pieces = []
    for low, high in safe_intervals(planets):
        if a > low:
            low = a
        if b < high:
            high = b
        if low < high:
            pieces.append((low, high))
    return pieces
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    planets = [(0, 2), (10, 1), (3, 1)]
    assert safe_between(planets, 5, 20) == [(5, 9), (11, 20)]


# ========================================================
# FOLLOW-UP 2: Are the endpoints safe?
# ========================================================
# Answer: the unsafe stretches are CLOSED ([c - r, c + r]), so the endpoints
# of every gap are unsafe and the safe intervals are open (they leave out
# their ends). If the question says the stretches are open, the endpoints are
# safe and touching stretches no longer merge.


# ========================================================
# FOLLOW-UP 3: Planets arrive over time
# ========================================================
# ---- follow-up 3: function ----
# Answer: keep the merged unsafe blocks in a sorted list. A new planet
# absorbs every block it overlaps (like the stream-of-intervals question);
# the safe intervals are always the gaps between the blocks.
# Complexity: Time add is O(n log n) (it re-sorts), safe is O(n), Space O(n)
class LiveRoute:
    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        start = c - r
        end = c + r
        kept = []
        for block_start, block_end in self.blocks:
            if block_end < start or block_start > end:
                kept.append([block_start, block_end])      # does not touch the new block
            else:
                if block_start < start:
                    start = block_start                    # overlaps: absorb it
                if block_end > end:
                    end = block_end
        kept.append([start, end])
        kept.sort()
        self.blocks = kept

    def safe(self):
        return gaps_between(self.blocks)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    planets = [(0, 2), (10, 1), (3, 1)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    live.add(6, 3)                                         # a big planet at 6 swallows both blocks
    assert live.safe() == [(-INF, -2), (11, INF)]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

### Test cases

| Input | Expected |
|---|---|
| `[]` | `[(None, None)]` |
| `[(3, 1), (10, 2)]` | `[(None, 2), (4, 8), (12, None)]` |

**STUDY PATH: question 25 of 53 | topic 12 of 30: Sorted intervals: merge while they touch, then read the gaps or search | 2 of 2**

Template fit: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs | no date

**HARD | DESIGN-STYLE (skippable) | Exponent | 25 of 53**

# Implement a function that processes a stream of interval data.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 190/288

## ABOUT THIS QUESTION

**Topic:** Sorted intervals: merge while they touch, then read the gaps or search

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 352 Data Stream as Disjoint Intervals (likely) (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/6245/implement-function-process-interval-data

## QUESTION

Intervals arrive one at a time. add(start, end) receives a new interval, and intervals() returns the current list of disjoint intervals, sorted by start; intervals that overlap or touch are merged.

**Function signature**

class IntervalStream:

```
def add(self, start: int, end: int) -> None
def intervals(self) -> list
```

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `s = IntervalStream()`<br>`s.add(1, 3)`<br>`s.add(6, 9)`<br>`s.intervals()`<br>`s.add(2, 5)`<br>`s.intervals()`<br>`s.add(5, 6)`<br>`s.intervals()` |
| **Output** | `s.intervals() -> [(1, 3), (6, 9)]`<br>`s.intervals() -> [(1, 5), (6, 9)]`<br>`s.intervals() -> [(1, 9)]` |
| **Why** | `(2, 5) overlaps (1, 3) and extends it to (1, 5); (5, 6) touches (1, 5) at 5 and (6, 9) at 6, so everything merges into (1, 9).` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
intervals = []
def add(start, end):
    intervals.append((start, end))
def query():
    merged = []
    for s, e in sorted(intervals):
        if merged and s <= merged[-1][1]:
            last = merged[-1]
            last[1] = max(last[1], e)
        else:
            merged.append([s, e])
    return merged
```

| TIME | SPACE |
|---|---|
| **O(n log n)** per query | **O(n)** |

**WHY IT IS SLOW**

Every query re-sorts and re-merges the whole history, O(n log n) per query. Keeping the merged intervals sorted and only splicing the affected block makes an add cost O(log n) to find plus the shifted block.

## OPTIMAL SOLUTION · Sorted disjoint intervals with binary search

| TIME | SPACE |
|---|---|
| **O(log n)** to find the spot, O(n) worst-case list shift | **O(n)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How do you keep adds fast for a very large number of intervals?

Use an ordered map (TreeMap, or a balanced tree / skip list in Python via the sortedcontainers package) keyed by start so a merge removes the swallowed intervals in amortized O(log n) instead of shifting an array. Each interval is inserted once and removed at most once, so n adds cost O(n log n) overall.

## FOLLOW-UP 2 Should touching intervals such as [1, 3] and [4, 5] merge?

Agree on the semantics first. For integer points, [1, 3] and [4, 5] cover every point from 1 to 5, so they merge; use bisect_left(self.ends, start - 1). For real-valued closed intervals only overlap or a shared endpoint merges, which is what the code above does. For half-open intervals [a, b), a shared endpoint merges but nothing else does.

## FOLLOW-UP 3 Answer 'is this point covered?' and 'how much is covered in total?'

The sorted disjoint intervals already support both: a point is covered if the interval with the largest start at or before it ends at or after it (one bisect), and the total length is the sum of the interval lengths (or keep it updated on each add).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a function that processes a
# stream of interval data.
```

```
#
# Intervals arrive one at a time. add(start, end) receives
# a new interval, and intervals() returns the current list
# of disjoint intervals, sorted by start; intervals that
# overlap or touch are merged.
#
# Signature:
#   class IntervalStream:
#       def add(self, start: int, end: int) -> None
#       def intervals(self) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you keep adds fast for a very large number of
# intervals?
# 2. Should touching intervals such as [1, 3] and [4, 5]
# merge?
# 3. Answer 'is this point covered?' and 'how much is
# covered in total?'
#
# INTERVIEW NOTES
# Keep the data in its simplest canonical form: merged
# intervals sorted by start with gaps between them. A new
# interval can only touch a contiguous block of existing
# ones, found with two binary searches, and that block
# collapses into one interval. State what 'touching' means
# before you code (shared endpoint, or only real overlap).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Complexity: Time O(log n), Space O(1)
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


# Complexity: Time O(log n), Space O(1)
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
```

```
        else:
            high = middle
    return low


# ---- main question: main class ----
# Intervals arrive one at a time; after every add(start, end), intervals()
# must return the merged, sorted list.
# Idea: keep the merged intervals sorted by start (two lists: starts and
# ends). A new interval can only interact with the intervals found by two
# binary searches: the first interval that touches it and the first one
# completely to its right. Replace that slice by ONE merged interval.
# integer=True: [1, 3] and [4, 5] cover 1..5, so they merge (follow-up 2);
# integer=False: only overlapping intervals or a shared end merge.
# The sorted disjoint intervals also answer follow-up 3: is a point
# covered (contains) and how much is covered in total (total).
# Complexity: Time add is O(log n) to search plus O(n) to splice the lists, contains O(log n), total O(1),
Space O(n)
class IntervalStream:
    def __init__(self, integer=True):
        self.starts = []
        self.ends = []
        self.gap = 0                            # how far apart two intervals may be and still merge
        self.extra = 0                          # the length of [a, b] is b - a + extra
        if integer:
```

```
            self.gap = 1
            self.extra = 1
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        i = first_at_least(self.ends, start - self.gap)       # the first interval that touches the new one
        j = first_greater(self.starts, end + self.gap)        # the first one completely to its right
        for k in range(i, j):
            self.covered -= self.length(self.starts[k], self.ends[k])    # the swallowed intervals leave th
e total
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
                end = self.ends[j - 1]
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)               # and the merged one joins it

    def intervals(self):
        result = []
        for i in range(len(self.starts)):
            result.append((self.starts[i], self.ends[i]))
        return result

    def contains(self, point):
        # the interval with the largest start <= point must still reach the point
        i = first_greater(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    def total(self):
        return self.covered
```

```
# Complexity: Time O(S * n) for S steps (it copies the interval list after every step), Space O(S * n)
def process(stream, integer=True):
    # feed a stream of (start, end) pairs; give the merged list after each add
    cover = IntervalStream(integer)
    steps = []
    for start, end in stream:
        cover.add(start, end)
```

```
        steps.append(cover.intervals())
    return steps


# ---- main question: tests ----
def test_main_question():
    steps = process([(1, 3), (8, 10), (4, 5), (6, 7)])
    assert steps[0] == [(1, 3)]
    assert steps[1] == [(1, 3), (8, 10)]
    assert steps[2] == [(1, 5), (8, 10)]                    # (4, 5) touches (1, 3) for whole numbers
    assert steps[3] == [(1, 10)]                            # (6, 7) touches both sides
    real = process([(1, 3), (4, 5), (3, 4)], False)
    assert real[1] == [(1, 3), (4, 5)]
    assert real[2] == [(1, 5)]


# ========================================================
# FOLLOW-UP 1: How do you keep adds fast for a very large number of intervals?
# ========================================================
# Answer: the lists give O(log n) searches but O(n) inserts (the lists
# shift). For very many intervals use a balanced search tree or a skip list,
# which find and replace a slice in O(log n).


# ========================================================
# FOLLOW-UP 2: Should touching intervals such as [1, 3] and [4, 5] merge?
# ========================================================
# Answer: it depends on whether the values are whole numbers (then they
# cover 1..5 and should merge) or real numbers (then there is a gap between
# 3 and 4). The `integer` option chooses; ask which one is meant.


# ========================================================
# FOLLOW-UP 3: Answer "is this point covered?" and "how much is covered in total?"
# ========================================================
# ---- follow-up 3: tests ----
# Answer: the sorted disjoint intervals already support both: a point is
# covered if the interval with the largest start at or before it ends at or
# after it (one binary search); the total length is kept up to date on
# every add.
def test_follow_up_3():
    box = IntervalStream()
    for pair in [(5, 8), (1, 2), (3, 4), (20, 21)]:
        box.add(pair[0], pair[1])
    assert box.intervals() == [(1, 8), (20, 21)]
```

```
    assert box.total() == 10
    assert box.contains(7) and box.contains(20)
    assert not box.contains(9) and not box.contains(0)
    random.seed(21)
    for round_number in range(200):                 # random intervals: compare with a set of covered point
s
        box = IntervalStream()
        points = set()
        for i in range(random.randint(1, 8)):
            a = random.randint(0, 30)
            b = a + random.randint(0, 6)
```

```
            box.add(a, b)
            for p in range(a, b + 1):
                points.add(p)
            assert box.total() == len(points)
        for p in range(-1, 40):
            assert box.contains(p) == (p in points)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 13 OF 30

# Trie: nested dicts, one character per level

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Trie: nested dicts, one character per level | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 26. Implement Longest-Match Text Replacement PracHub 2026-01-20

CODE TEMPLATE | GROUP 13 OF 30 | 1 QUESTIONS, ONE PATTERN

# Trie: nested dicts, one character per level

## The one idea these questions share

Store words as nested dicts, one character per level, with an end marker that holds whatever you want to remember. Insert walks and creates; search walks and fails when a character is missing; prefix queries stop in the middle; longest-match keeps the last end marker you passed.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
END = "$"


def build_trie(words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node[END] = w                           # CHANGE 1: what you store at the end of a word
    return root


def walk(root, text):
    node = root
    for ch in text:
        node = node.get(ch)
        if node is None:
            return None
    return node
```

## 2. Helper functions that usually get written next to it

```
def words_under(node):
    out = []
    for key, child in node.items():
        if key == END:
            out.append(child)
        else:
            out.extend(words_under(child))
    return sorted(out)


def longest_match(root, text, start):
    node, best = root, None
    for j in range(start, len(text)):
        node = node.get(text[j])
        if node is None:
            break
        if END in node:
            best = node[END]
    return best
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **26** | Implement Longest-Match Text Replacement | at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents

Start the questions >

**STUDY PATH: question 26 of 53 | topic 13 of 30: Trie: nested dicts, one character per level | 1 of 1**

Template fit: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it | asked 273 days ago

HARD | IN APP LC 648 | PracHub | Technical Screen | 26 of 53

# Implement Longest-Match Text Replacement

POSTED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 31.9 | 30 DAYS 0 | PRIORITY 140/288

## ABOUT THIS QUESTION

**Topic:** Trie: nested dicts, one character per level

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #648 Replace Words

**LeetCode match:** LC 648 Replace Words / LC 833 Find And Replace in String

Posted on PracHub: 2026-01-20 (updated 2026-06-13)

**Original:** https://prachub.com/coding-questions/implement-longest-match-text-replacement

## QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

**Part 1:** Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

## QUESTION

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

**Hints**

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

**Part 2:** Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def replace(text, entries):
    out, i = [], 0
    while i < len(text):
        best = ""
        for token in entries:
            if text.startswith(token, i):
                if len(token) > len(best):
                    best = token
        if best:
            out.append(entries[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out
```

| TIME | SPACE |
|---|---|
| **O(len(text) * tokens * token length)** | **O(1)** extra |

WHY IT IS SLOW

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

## OPTIMAL SOLUTION · Trie walk that remembers the last token end

| TIME | SPACE |
|---|---|
| O(len(text) * longest token) | O(total token length) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

## FOLLOW-UP 2 The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

## FOLLOW-UP 3 Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. **Cost:** still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Longest-Match Text Replacement
#
# You are given a text string and a dictionary of
# replacement entries. Each dictionary entry has the form
# token:id, where token is a string pattern and id is the
# value that should be emitted when that token is matched.
# Implement a function that scans the text from left to
# right and produces an output sequence using greedy
# longest-prefix matching: 1. At the current text
# position, find all dictionary tokens that match the
# substring starting at that position. 2. If at least one
# token matches, choose the longest matching token, append
# its corresponding id to the output, and advance by the
# length of that token. 3. If no token matches, append the
# original character at the current position to the output
# and advance by one character. 4. Continue until the
# entire text has been processed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority is not longest match: insertion order, a
# score, or a custom comparator.
# 2. The text is huge or the dictionary changes often.
# 3. Overlapping matches or word boundaries.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary token
# starting there. Checking every token costs the size of
# the dictionary per character, so share prefixes in a
# trie: walk the text from the current position down the
# trie, remember the last node that ends a token, and stop
```

```
# when the trie has no further edge. The remembered node
# is the longest match.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: put the tokens in a trie (a tree with one letter per level); the
```

```
# last node of a token stores the token and its id. At a text position,
# walk down the trie along the text; every node on the way that stores
# an id is a token that matches here. Take the LONGEST one, output its id,
# and move on by its length. If nothing matches, output the character
# itself and move on by one.
# Complexity: Time build O(total token characters), replace O(n * longest token), Space O(total token chara
cters)
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id
        self.root = {}
        for token in mapping:
            node = self.root
            for ch in token:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (token, mapping[token])

    def candidates(self, text, i):
        # every token that matches at position i, shortest first (all the word
        # ends along the trie path, not only the last one)
        found = []
        node = self.root
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                found.append(node["$"])
        return found

    def replace(self, text, choose=None, word_boundary=False):
        # `choose(candidates)` picks the winner (follow-up 1); by default
        # the longest token. word_boundary=True accepts only whole-word
        # matches (follow-up 3).
        output = []
        i = 0
        while i < len(text):
            found = self.candidates(text, i)
            if word_boundary:
                whole = []
                for token, ident in found:
                    if is_whole_word(text, i, len(token)):
                        whole.append((token, ident))
                found = whole
```

```
            if found:
                if choose is None:
```

```
                    token, ident = found[-1]
                else:
                    token, ident = choose(found)
                output.append(ident)
                i += len(token)
            else:
                output.append(text[i])                # nothing matches here: keep the character
                i += 1
        return output


# Complexity: Time O(1), Space O(1)
def is_whole_word(text, start, size):
    # no letter or digit touches the match on either side
    before_ok = start == 0 or not text[start - 1].isalnum()
    end = start + size
    after_ok = end == len(text) or not text[end].isalnum()
    return before_ok and after_ok


# ---- main question: tests ----
TABLE = {"ab": 1, "abc": 2, "c": 3}


def test_main_question():
    r = Replacer(TABLE)
    assert r.replace("abcab") == [2, 1]               # "abc" beats "ab" at position 0
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    assert r.replace("cab") == [3, 1]


# ========================================================
# FOLLOW-UP 1: The priority is not longest match: insertion order, a score, or a custom comparator
# ========================================================
# ---- follow-up 1: tests ----
# Answer: collect EVERY token that matches at the position (the trie
# already gives all of them along the path) and choose with a separate
# policy function. The trie finds the candidates; the policy decides.
def first_inserted(candidates):
    order = list(TABLE)
    best = candidates[0]
    for candidate in candidates:
        if order.index(candidate[0]) < order.index(best[0]):
            best = candidate
    return best


def test_follow_up_1():
    r = Replacer(TABLE)
    assert r.replace("abc", first_inserted) == [1, 3]          # "ab" was inserted before "abc"


# ========================================================
# FOLLOW-UP 2: The text is huge or the dictionary changes often
# ========================================================
# Answer: the trie walk reads each text position once, so a huge text can
# be processed as a stream. For a changing dictionary, inserting or
# removing a token only changes one path of the trie.


# ========================================================
# FOLLOW-UP 3: Overlapping matches or word boundaries
# ========================================================
```

```
# ---- follow-up 3: tests ----
# Answer: word boundaries: accept a match only when no letter touches it
# on either side (the word_boundary flag). Overlapping matches: do not
# skip the matched text; output every candidate for each position.
def test_follow_up_3():
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

**Signature**

```
def solution(text: str, dictionary: dict[str, str]) -> list[str]:
```

**Sample call**

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

**Test cases**

| Input | Expected |
|---|---|
| `("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})` | `["2", "d", "3", "z"]` |
| `("aaaaa", {"a": "X", "aa": "Y"})` | `["Y", "Y", "X"]` |

**ALSO REPORTED · 1 more**

**Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low**

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.

Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.

Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).

If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

**ALSO REPORTED · 1 more**

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 14 OF 30

# A dict for lookup plus an ordered container for who is next (lazy deletion)

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 27. Implement LRU Cache. **Exponent no date**

CODE TEMPLATE | GROUP 14 OF 30 | 1 QUESTIONS, ONE PATTERN

# A dict for lookup plus an ordered container for who is next (lazy deletion)

## The one idea these questions share

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


class OrderedStore:
    def __init__(self):
        self.order = deque()                  # CHANGE 1: the container that says who is next
        self.live = {}                        # key -> ticket, for O(1) lookup and lazy deletion
        self.ticket = 0

    def add(self, key):
        self.ticket += 1
        self.live[key] = self.ticket
        self.order.append((self.ticket, key))

    def remove(self, key):
        self.live.pop(key, None)              # lazy: the entry stays in the deque

    def next(self):
        while self.order and self.live.get(self.order[0][1]) != self.order[0][0]:
            self.order.popleft()              # CHANGE 2: skip entries that were removed
        return self.order[0][1] if self.order else None
```

## 2. Helper functions that usually get written next to it

```
from collections import OrderedDict


class LRU:                                    # the short version of the same idea
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **27** | Implement LRU Cache. | dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end) |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 27 of 53 | topic 14 of 30: A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 of 1**

Template fit: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end) | no date

**HARD | IN APP LC 146 | DESIGN-STYLE (skippable) | Exponent | 27 of 53**

**★ TOP 75: rank 64 in the Medium and Hard priority list**

# ★ Implement LRU Cache.

**ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS**

**STUDY: TYPE** ALL TIME 57.3 30 DAYS 0 PRIORITY 55/288

## ABOUT THIS QUESTION

**Topic:** A dict for lookup plus an ordered container for who is next (lazy deletion)

**Optimal answer from:** your app's interview approach (LeetCode #146)

**LeetCode match:** LC 146 LRU Cache

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +2562 interviews

**Original:** https://www.aced.io/questions/3625/lru-cache

## QUESTION

Your program keeps re-reading the same data from disk, so you want an in-memory cache that holds only the n most recently used items (a least-recently-used cache). Write a class LRUCache(n) with get(key), which returns the stored value or -1, and put(key, value), which stores a value and, when the cache is full, first removes the item that was used least recently. Both get and put count as a use. Both must run in O(1).

**Function signature**

class LRUCache:
  def __init__(self, n: int)
  def get(self, key) -> int
  def put(self, key, value) -> None

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `c = LRUCache(2)`<br>`c.put(1, 1)`<br>`c.put(2, 2)`<br>`c.get(1)`<br>`c.put(3, 3)`<br>`c.get(2)`<br>`c.put(4, 4)`<br>`c.get(1)`<br>`c.get(3)`<br>`c.get(4)` |
| **Output** | `c.get(1) -> 1`<br>`c.get(2) -> -1`<br>`c.get(1) -> -1`<br>`c.get(3) -> 3`<br>`c.get(4) -> 4` |
| **Why** | `get(1) makes key 1 the most recent, so put(3, 3) evicts key 2; then put(4, 4) evicts key 1 (older than 3).` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
items = []  # (key, value), newest last
def get(k):
    for i, (key, value) in enumerate(items):
        if key == k:
            items.append(items.pop(i))  # newest
            return value
    return -1
def put(k, v):
    for i, (key, _) in enumerate(items):
        if key == k:
            items.pop(i)
            break
    else:
        if len(items) == capacity:
            items.pop(0)  # oldest
    items.append((k, v))
```

| TIME | SPACE |
|---|---|
| **O(n)** per operation | **O(capacity)** |

### WHY IT IS SLOW

Every operation scans the list, O(capacity). A hash map to nodes of a doubly linked list gives O(1) lookup and O(1) move-to-front and eviction.

## OPTIMAL SOLUTION · Hash map plus doubly linked list (built by hand)

| TIME | SPACE |
|---|---|
| **O(1)** per operation | **O(capacity)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

## FOLLOW-UP 2 Entries expire after a time to live (TTL).

Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

## FOLLOW-UP 3 Least frequently used (LFU) instead of least recently used.

Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement LRU Cache.
#
# Your program keeps re-reading the same data from disk,
# so you want an in-memory cache that holds only the n
# most recently used items (a least-recently-used cache).
# Write a class LRUCache(n) with get(key), which returns
# the stored value or -1, and put(key, value), which
# stores a value and, when the cache is full, first
# removes the item that was used least recently. Both get
# and put count as a use. Both must run in O(1).
#
# Signature:
#   class LRUCache:
#       def __init__(self, n: int)
#       def get(self, key) -> int
#       def put(self, key, value) -> None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many threads share the cache.
# 2. Entries expire after a time to live (TTL).
# 3. Least frequently used (LFU) instead of least recently
# used.
#
# INTERVIEW NOTES
# You need two things at once: find a key quickly (a hash
# map) and know which key is the oldest use (an ordering
# you can update in O(1)). A doubly linked list gives O(1)
# move-to-front and remove-from-back, and each hash map
# entry points at its list node. Python's OrderedDict is
# exactly that pair; in an interview be ready to build the
# linked list by hand.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import threading
import time
from collections import OrderedDict


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    # one entry of the doubly linked list
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


# ---- main question: main function ----
```

```python
# Idea: a dictionary maps each key to its node in a doubly linked list
# that is ordered by recency (most recently used at the FRONT). Two
# dummy nodes, `head` and `tail`, sit at the ends so no operation has
# to treat an empty list or the ends specially. get and put are O(1).
# Complexity: Time get and put are O(1), Space O(capacity)
class LRUCache:
    def __init__(self, n):
        self.capacity = n
        self.nodes = {}                             # key -> node
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.nodes:
            return -1
        node = self.nodes[key]
        self.unlink(node)                           # reading makes it the most recently used
        self.push_front(node)
        return node.value
```

```python
    def put(self, key, value):
        if key in self.nodes:
            node = self.nodes[key]
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.nodes) == self.capacity:
            oldest = self.tail.prev                 # full: evict the least recently used (at the back)
            self.unlink(oldest)
            del self.nodes[oldest.key]
        node = Node(key, value)
        self.nodes[key] = node
        self.push_front(node)


# ---- main question: tests ----
def test_main_question():
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    c.put(3, 3)                                     # 2 is now the least recently used, so 3 evicts it
    assert c.get(2) == -1 and c.get(3) == 3
    c.put(1, 10)
    assert c.get(1) == 10


# ========================================================
# FOLLOW-UP 1: Many threads share the cache
# ========================================================
# ---- follow-up 1: function ----
# Answer: put get and put behind ONE lock (even get is a write, because it
```

```
# moves the node). For more throughput shard the cache by the hash of the
# key, each shard with its own lock and its own capacity.
# Complexity: Time get and put are O(1) (plus waiting for the lock), Space O(capacity)
class SafeLRU:
    def __init__(self, n):
        self.cache = LRUCache(n)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return self.cache.get(key)
```

```
    def put(self, key, value):
        with self.lock:
            self.cache.put(key, value)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = SafeLRU(1)
    s.put("a", 1)
    s.put("b", 2)
    assert s.get("a") == -1 and s.get("b") == 2
    workers = []

    def hammer(start):
        for i in range(start, start + 300):
            s.put(i, i)
            s.get(i)

    for start in (0, 1000, 2000):
        worker = threading.Thread(target=hammer, args=(start,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(s.cache.nodes) == 1                  # the capacity held


# =====================================================
# FOLLOW-UP 2: Entries expire after a time to live (TTL)
# =====================================================
# ---- follow-up 2: function ----
# Answer: store an expiry time with each entry and check it in get: a
# stale entry counts as missing and is removed. (Optionally also sweep the
# old end of the list on put.) The recency order and the O(1) costs stay
# the same. `clock` is a function that returns the current time, so tests
# can use a fake one.
# Complexity: Time get is O(1), put is O(n) (it cleans the expiry table), Space O(capacity)
class TTLCache(LRUCache):
    def __init__(self, n, ttl, clock=time.monotonic):
        LRUCache.__init__(self, n)
        self.ttl = ttl
        self.clock = clock
        self.expiry = {}                            # key -> the time it expires

    def get(self, key):
```

```
        if key in self.nodes and self.expiry[key] <= self.clock():
            self.unlink(self.nodes[key])            # stale: remove it
            del self.nodes[key]
            del self.expiry[key]
        return LRUCache.get(self, key)

    def put(self, key, value):
```

```
        LRUCache.put(self, key, value)
        self.expiry[key] = self.clock() + self.ttl
        for k in list(self.expiry):                 # forget the expiry of keys that were evicted
            if k not in self.nodes:
                del self.expiry[k]


# ---- follow-up 2: tests ----
class FakeClock:
    def __init__(self):
        self.now = 0

    def __call__(self):
        return self.now


def test_follow_up_2():
    clock = FakeClock()
    t = TTLCache(2, 10, clock)
    t.put("x", 1)
    assert t.get("x") == 1
    clock.now = 11
    assert t.get("x") == -1                         # expired
    t.put("y", 2)
    assert t.get("y") == 2


# ============================================================
# FOLLOW-UP 3: Least frequently used (LFU) instead of least recently used
# ============================================================
# ---- follow-up 3: function ----
# Answer: keep a use count per key and "buckets": for each count, the keys
# with that count in order of use (an OrderedDict), plus a pointer to the
# lowest count in use. get and put move a key to the next bucket in O(1);
# eviction takes the OLDEST key of the lowest bucket (LeetCode 460).
# Complexity: Time get and put are O(1), Space O(capacity)
class LFUCache:
    def __init__(self, n):
```

```
        self.capacity = n
        self.value = {}                             # key -> value
        self.freq = {}                              # key -> how often it was used
        self.buckets = {}                           # count -> keys with that count, oldest first
        self.lowest = 0

    def bucket(self, count):
        if count not in self.buckets:
            self.buckets[count] = OrderedDict()
        return self.buckets[count]

    def touch(self, key):
        count = self.freq[key]
        del self.bucket(count)[key]
        if len(self.bucket(count)) == 0 and self.lowest == count:
            self.lowest += 1                        # the lowest bucket is empty now
        self.freq[key] = count + 1
        self.bucket(count + 1)[key] = None

    def get(self, key):
        if key not in self.value:
            return -1
        self.touch(key)
        return self.value[key]

    def put(self, key, value):
```

```
        if self.capacity == 0:
            return
        if key in self.value:
            self.value[key] = value
            self.touch(key)
            return
        if len(self.value) == self.capacity:
            oldest, nothing = self.bucket(self.lowest).popitem(last=False)
            del self.value[oldest]
            del self.freq[oldest]
        self.value[key] = value
        self.freq[key] = 1
        self.bucket(1)[key] = None
        self.lowest = 1


# ---- follow-up 3: tests ----
def test_follow_up_3():
    f = LFUCache(2)
```

```
    f.put(1, 1)
    f.put(2, 2)
    assert f.get(1) == 1
    f.put(3, 3)                                     # 2 was used least often, so it goes
    assert f.get(2) == -1 and f.get(3) == 3
    f.put(4, 4)                                     # 1 and 3 were used twice: the older one (1) goes
    assert f.get(1) == -1 and f.get(3) == 3 and f.get(4) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## OFFICIAL PAGE TEXT

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time?

```
Pythonclass LRUCache(n):
  set(key, value)
  get(key)
```

Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated.

## OFFICIAL PAGE TEXT

```
Pythoncache = LRUCache(2) # Limit of 2 items

cache.set('user1', 'Alex')
cache.set('user2', 'Brian')
cache.set('user3', 'Chris')
```

Here user1 is empty because it was the least recently used and thus removed to make room for user3:

```
Pythoncache.get('user1') # => None
cache.get('user2') # => 'Brian'
cache.get('user3') # => 'Chris'
```

Hint 1If you're having trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array won't work because moving items around takes O(n) time. Could you combine two different data structures to maintain this order and also get fast lookups?Hint 3You can use a doubly linked list to store the items in the correct order. When an item is used, just move its node to the front of the list. The least recently used node will always end up at the end of the list.Hint 4To look up items without traversing the entire list, you can also add a hash table that maps from the key to the item's node.Challenge 1Whats the time complexity of your solution?This problem is challenging because of the combination of constraints such as key-value lookup, size limits, and time complexity. Ultimately, there is no single data structure that fulfills all of these needs, so we have to get creative! Our solution is to implement the cache class with a linked list and a hash table. The linked list is doubly-linked, which allows us to insert and remove items in constant time without traversing the entire list. Every time an item is set, updated, or retrieved, we just move its Node to the front of the list. When the cache reaches its size limit, we can easily make room by removing the item at the end of the list. The linked list gives us O(1) insertion time, but by itself it has O(n) lookup time because we have to traverse the entire list to find a particular item. That's not ideal since we want to be able to look up an item quickly using its key. That's where the hash table comes in: we can use it add a mapping from an item's key to its Node. Together, these data structures give us a time complexity of O(1) for insertion, deletion, and retrieval in the worst case, and a space footprint of O(n). No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 15 OF 30

# Rolling state: the next state comes only from the previous one

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Rolling state: the next state comes only from the previous one | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 28. Constrained Dice Roll Sequences **ColorStack 2025-10-06**

CODE TEMPLATE | GROUP 15 OF 30 | 1 QUESTIONS, ONE PATTERN

# Rolling state: the next state comes only from the previous one

## The one idea these questions share

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def roll(items):
    take, skip = 0, 0                           # CHANGE 1: the state
    for x in items:
        take, skip = skip + x, max(take, skip)     # CHANGE 2: new state from the old state
    return max(take, skip)
```

## 2. Helper functions that usually get written next to it

```
def two_smallest(row):                    # the trick behind "best of the others"
    order = sorted(range(len(row)), key=lambda j: row[j])
    return order[0], order[1] if len(order) > 1 else order[0]


def fib_pair(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **28** | Constrained Dice Roll Sequences | state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit) |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 28 of 53 | topic 15 of 30: Rolling state: the next state comes only from the previous one | 1 of 1**

Template fit: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit) | asked 379 days ago | frequency Single report

HARD | ColorStack | technical interview | 28 of 53

# Constrained Dice Roll Sequences

ASKED 2025-10-06 | 379 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 205/288

## ABOUT THIS QUESTION

**Topic:** Rolling state: the next state comes only from the previous one

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 1223 Dice Roll Simulation

Frequency: Single report | Last asked: 2025-10-06 | Roles: SWE Intern

## QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

**Function signature**

def count_dice_sequences(n: int, roll_max: list[int]) -> int:

    ...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `n = 2, roll_max = [1, 1, 2, 2, 2, 3]` |
| **Output** | `34` |
| **Why** | `There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.` |

**Example 2**

| | |
|---|---|
| **Input** | `n = 2, roll_max = [1, 1, 1, 1, 1, 1]` |
| **Output** | `30` |
| **Why** | `No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.` |

**Example 3**

| | |
|---|---|
| **Input** | `n = 3, roll_max = [1, 1, 1, 2, 2, 3]` |
| **Output** | `181` |
| **Why** | `Hand-verified against the memoized DP over (rolls_used, last_value, run_length).` |

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .

## QUESTION

- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
from itertools import groupby, product
def brute_dice(n, roll_max):
    count = 0
    for seq in product(range(6),_repeatn):
        if all(
            len(list(g)) <= roll_max[v]
            for v, g in groupby(seq)
        ):
            count += 1
    return count
```

| TIME | SPACE |
|---|---|
| O(6^n * n) | O(n) |

### WHY IT IS SLOW

It generates all 6^n sequences and checks the runs of each.

## OPTIMAL SOLUTION · DP over the last value and its run length

| TIME | SPACE |
|---|---|
| O(n * 6 * R) | O(6 * R) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Each turn rolls a pair of dice together, and the limit applies to each die independently. How many joint sequences are there?

The two dice never influence each other, so the joint sequences are the pairs of one valid sequence per die: the answer is the single-die count squared (mod 10^9 + 7). Tracking both run lengths in one DP gives the same number with a much bigger state.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Constrained Dice Roll Sequences
#
# You roll a single 6-faced die exactly n times. You are
# given an array roll_max of length 6, where roll_max[i]
# is the maximum number of times the value i + 1 may
# appear consecutively. That is, you may never roll the
# same value more than roll_max[i] times in a row. Return
# the number of distinct sequences of n rolls that respect
# every consecutive-count limit. Because the answer can be
# very large, return it modulo 10**9 + 7 . Function
# signature: def count_dice_sequences(n: int, roll_max:
# list[int]) -> int: ... Follow-up (as reported): The
```

```
# interviewer's actual twist used two dice per turn (a
# pair rolled together, with the consecutive limit applied
# to each die independently). Extend the state to track
# the run lengths of both dice and count the joint
# sequences.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each turn rolls a pair of dice together, and the
# limit applies to each die independently. How many joint
# sequences are there?
#
# INTERVIEW NOTES
# Count sequences by what has to be remembered about the
# end of the prefix: the last value and how many times it
# was repeated in a row. dp[last][run] is the number of
# valid prefixes of the current length; rolling a
# different value resets the run to 1, rolling the same
# value extends it if the limit allows.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
MOD = 10 ** 9 + 7
```

```
def total_of(dp):
    # the sum of every number in the table
    total = 0
    for row in dp:
        for count in row:
            total += count
    return total


# ---- main question: main function ----
# n rolls of a six-sided die; the value v+1 may appear at most roll_max[v]
# times IN A ROW. Count the valid sequences (modulo 10^9 + 7).
# Idea (DP over the last value and how long it has been repeated):
# dp[v][r] = the number of valid sequences so far that end with value v
# repeated r times in a row. For the next roll:
#   - a DIFFERENT value than before starts a run of length 1:
#     (all sequences) - (those that already end in v);
#   - the SAME value extends its run by one (only while r < roll_max[v]).
# Complexity: Time O(n * S) where S is the sum of the roll limits (at most 90), Space O(S)
def count_dice_sequences(n, roll_max):
    dp = []
    for v in range(6):
        row = [0] * (roll_max[v] + 1)
        row[1] = 1                                          # one roll: every value once
        dp.append(row)
    for roll in range(n - 1):
        everything = total_of(dp) % MOD
        new = []
        for v in range(6):
            new.append([0] * (roll_max[v] + 1))
        for v in range(6):
            ending_in_v = 0
```

```
            for count in dp[v]:
                ending_in_v += count
            new[v][1] = (everything - ending_in_v) % MOD
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
        dp = new
    return total_of(dp) % MOD


# ---- main question: tests ----
def test_main_question():
    assert count_dice_sequences(2, [1, 1, 2, 2, 2, 3]) == 34
    assert count_dice_sequences(2, [1, 1, 1, 1, 1, 1]) == 30
```

```
    assert count_dice_sequences(3, [1, 1, 1, 2, 2, 3]) == 181
    assert count_dice_sequences(1, [1, 1, 1, 1, 1, 1]) == 6


# ========================================================
# FOLLOW-UP 1: Each turn rolls a pair of dice together, and the limit applies to each die independently. Ho
w many joint sequences are there?
# ========================================================
# ---- follow-up 1: function ----
# Answer: the two dice never influence each other, so a joint sequence is
# one valid sequence for each die: the answer is the single-die count
# SQUARED (modulo 10^9 + 7).
# Complexity: Time O(n * S), Space O(S)
def count_two_dice(n, roll_max):
    one = count_dice_sequences(n, roll_max)
    return one * one % MOD


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_two_dice(2, [1, 1, 2, 2, 2, 3]) == 34 * 34
    assert 0 <= count_dice_sequences(500, [15] * 6) < MOD


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 16 OF 30

# Walk two strings at once with two indices

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Walk two strings at once with two indices | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 29. Find Dictionary Matches for a Jammed String PracHub 2026-04-09

CODE TEMPLATE | GROUP 16 OF 30 | 1 QUESTIONS, ONE PATTERN

# Walk two strings at once with two indices

## The one idea these questions share

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_difference(a, b, skip="."):
    i = j = 0
    while i < len(a) and j < len(b):
        while i < len(a) and a[i] == skip:      # CHANGE 1: skip characters that do not matter
            i += 1
        while j < len(b) and b[j] == skip:
            j += 1
        if i == len(a) or j == len(b):
            break
        if a[i] != b[j]:                        # CHANGE 2: the rule when they differ
            return i, j
        i, j = i + 1, j + 1
    return None
```

## 2. Helper functions that usually get written next to it

```
def runs(s):                                    # run-length encoding: "aaab" -> [("a", 3), ("b", 1)]
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return [(c, n) for c, n in out]
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **29** | Find Dictionary Matches for a Jammed String | compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3 |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 29 of 53 | topic 16 of 30: Walk two strings at once with two indices | 1 of 1**

Template fit: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3 | asked 194 days ago

HARD | PracHub | Technical Screen | 29 of 53

# Find Dictionary Matches for a Jammed String

POSTED 2026-04-09 | 194 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 155/288

## ABOUT THIS QUESTION

**Topic:** Walk two strings at once with two indices

**Optimal answer from:** the interview site's own solution

**LeetCode match:** LC 809 Expressive Words (inferred from overview)

Posted on PracHub: 2026-04-09 (updated 2026-04-19)

**Original:** https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

## QUESTION

A word was typed on a keyboard where some keys got stuck, so a letter may be repeated. Given a dictionary of words, matches(query) returns the dictionary words the query could be a jammed version of: the runs of equal letters line up with the word's runs, and each run of the query is either exactly as long as in the word or longer and at least min_stretch (default 3) letters long.

**Function signature**

class JammedIndex:

def __init__(self, words: list)

def matches(self, query: str, min_stretch: int = 3) -> list

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `idx = JammedIndex(['hello', 'help', 'hel'])`<br>`idx.matches('heeello')`<br>`idx.matches('hello')`<br>`idx.matches('heello')`<br>`idx.matches('helllo')` |
| **Output** | `idx.matches('heeello') -> ['hello']`<br>`idx.matches('hello') -> ['hello']`<br>`idx.matches('heello') -> []`<br>`idx.matches('helllo') -> ['hello']` |
| **Why** | `'heeello' stretches the e to 3 letters, which is allowed. 'heello' only stretches it to 2 (shorter than min_stretch), and 'helllo' stretches the l run from 2 to 3 letters, which is allowed.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def matches(query, words):
    runs_q = runs(query)  # [(letter, count)]
    result = []
    for word in words:
        runs_w = runs(word)
        if compatible(runs_w, runs_q):
            result.append(word)
    return result
```

| TIME | SPACE |
|---|---|
| **O(W * L)** per query | **O(L)** |

**WHY IT IS SLOW**

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

## OPTIMAL SOLUTION · Index the dictionary by its run letters

| TIME | SPACE |
|---|---|
| **O(L)** per query plus the size of the matching bucket | **O(total dictionary length)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

## FOLLOW-UP 2 Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. **Cost:** O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

## FOLLOW-UP 3 The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Dictionary Matches for a Jammed
# String
#
# A word was typed on a keyboard where some keys got
# stuck, so a letter may be repeated. Given a dictionary
# of words, matches(query) returns the dictionary words
# the query could be a jammed version of: the runs of
```

```
# equal letters line up with the word's runs, and each run
# of the query is either exactly as long as in the word or
# longer and at least min_stretch (default 3) letters
# long.
#
# Signature:
#   class JammedIndex:
#       def __init__(self, words: list)
#       def matches(self, query: str, min_stretch: int = 3) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. A different stretch rule: any run may be longer, no
# minimum.
# 2. Millions of dictionary words and many queries.
# 3. The jammed string may also be missing letters.
#
# INTERVIEW NOTES
# 'Same letters, stretched' means compare run-length
# encodings, not characters. The letter order of the runs
# has to match exactly, so the sequence of distinct run
# letters is a hash key that cuts the dictionary down
# before any counts are compared.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]: each letter with how many times in a row
    result = []
    for ch in word:
```

```
        if result and result[-1][0] == ch:
            result[-1] = (ch, result[-1][1] + 1)
        else:
            result.append((ch, 1))
    return result


def run_letters(word):
    # "hello" -> "helo": the letters of the runs, repeats collapsed
    letters = ""
    for ch, count in runs(word):
        letters += ch
    return letters


def run_counts(word):
    counts = []
    for ch, count in runs(word):
        counts.append(count)
    return counts


def stretch_fits(query_count, word_count, min_stretch):
    # a run matches when it is the same length, or when the query stretched
    # it to at least min_stretch letters
    return query_count == word_count or (query_count > word_count and query_count >= min_stretch)
```

```
# ---- main question: main function ----
# Idea: group the dictionary words by their run letters ("hello" -> "helo").
# A query can only match words in its own group, so compare the run
# lengths only there. Build the index once and reuse it for many queries.
# min_stretch is the shortest stretch that is allowed (follow-up 1: use 1
# for "any run may be longer").
# Complexity: Time build O(total length of the words), matches O(words in the group * runs), Space O(total
length)
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for word in words:
            letters = run_letters(word)
            if letters not in self.groups:
                self.groups[letters] = []
            self.groups[letters].append((word, run_counts(word)))

    def matches(self, query, min_stretch=3):
```

```
        letters = run_letters(query)
        counts = run_counts(query)
        found = []
        for word, word_counts in self.groups.get(letters, []):
            all_fit = True
            for i in range(len(counts)):
                if not stretch_fits(counts[i], word_counts[i], min_stretch):
                    all_fit = False
            if all_fit:
                found.append(word)
        return sorted(found)


# ---- main question: tests ----
def test_main_question():
    index = JammedIndex(["hello", "help", "hi", "hip"])
    assert index.matches("heeeellooooo") == ["hello"]         # e stretched to 4 and o to 3
    assert index.matches("hello") == ["hello"]
    assert index.matches("heello") == []                      # a stretch of only 2 is not enough
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    assert runs("aabccc") == [("a", 2), ("b", 1), ("c", 3)]


# =========================================================
# FOLLOW-UP 1: A different stretch rule: any run may be longer, no minimum
# =========================================================
# ---- follow-up 1: tests ----
# Answer: only the comparison changes; the index is the same. That is why
# min_stretch is a parameter: min_stretch=1 means any longer run is fine.
def test_follow_up_1():
    index = JammedIndex(["hello", "help", "hi", "hip"])
    assert index.matches("heello", min_stretch=2) == ["hello"]
    assert index.matches("hiip", min_stretch=1) == ["hip"]


# =========================================================
# FOLLOW-UP 2: Millions of dictionary words and many queries
# =========================================================
# Answer: the index already does the heavy work once. For millions of
# words keep the index on disk or sharded by the run letters, and only
# load the one group a query needs.
```

```
# ========================================================
```

```
# FOLLOW-UP 3: The jammed string may also be missing letters
# ========================================================
# Answer: then run letters can differ, so the exact-group lookup is not
# enough. Use a trie over the words and walk it with the query, allowing
# a bounded number of skipped letters (edit distance style).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(dictionary_words: list[str], jammed_string: str) -> list[str]:
```

### Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

### Test cases

| Input | Expected |
|---|---|
| (['help', 'hello', 'shell', 'hellp'], 'hellp') | ['help', 'hellp'] |
| (['hello', 'halo', 'heeello'], 'hheeellllo') | ['hello', 'heeello'] |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 17 OF 30

# Simulate a machine: a class with state and one branch per kind of event

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Simulate a machine: a class with state and one branch per kind of event | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 30. Simulate In-Place Cellular State Updates **PracHub 2026-05-04**

CODE TEMPLATE | GROUP 17 OF 30 | 1 QUESTIONS, ONE PATTERN

# Simulate a machine: a class with state and one branch per kind of event

## The one idea these questions share

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class Machine:
    def __init__(self):
        self.score, self.lives = 0, 3             # CHANGE 1: the state
        self.over = False

    def process(self, event):
        if self.over:
            return
        if event == "coin":                       # CHANGE 2: one branch per kind of event
            self.score += 100
        elif event == "damage":
            self.lives -= 1
            self.over = self.lives == 0
```

## 2. Helper functions that usually get written next to it

```
def neighbors8(r, c):
    return [(r + dr, c + dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if dr or dc]


def run_through(board, r, c, dr, dc, player):    # length of the line of one player through a cell
    n, run = len(board), 1
    for sign in (1, -1):
        a, b = r + sign * dr, c + sign * dc
        while 0 <= a < n and 0 <= b < n and board[a][b] == player:
            run += 1
            a, b = a + sign * dr, b + sign * dc
    return run
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **30** | Simulate In-Place Cellular State Updates | encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 30 of 53 | topic 17 of 30: Simulate a machine: a class with state and one branch per kind of event | 1 of 1**

Template fit: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end | asked 169 days ago

HARD | IN APP LC 289 | PracHub | Technical Screen | 30 of 53

# Simulate In-Place Cellular State Updates

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 23.4 | 30 DAYS 0 | PRIORITY 164/288

## ABOUT THIS QUESTION

**Topic:** Simulate a machine: a class with state and one branch per kind of event

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #289 Game of Life

**LeetCode match:** LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-07)

**Original:** https://prachub.com/coding-questions/simulate-in-place-cellular-state-updates

## QUESTION

Conway's Game of Life on a grid of 0 (dead) and 1 (alive) cells. A live cell with 2 or 3 live neighbours (of its 8 neighbours) stays alive, a dead cell with exactly 3 live neighbours becomes alive, and every other cell is dead in the next generation. All cells change at the same time. Do it in place with O(1) extra space; with wrap=True the grid wraps around its edges.

### Function signature

def step_in_place(board: list, wrap=False) -> list

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[[0, 1, 0], [0, 1, 0], [0, 1, 0]]` |
| Output | `[[0, 0, 0], [1, 1, 1], [0, 0, 0]]` |
| Why | The vertical blinker turns into a horizontal one. |

| Example 2 | |
|---|---|
| Input | `[[1, 1], [1, 1]]` |
| Output | `[[1, 1], [1, 1]]` |
| Why | A 2x2 block is stable: every cell has 3 live neighbours. |

| Example 3 | |
|---|---|
| Input | `[[0, 0, 0], [0, 1, 0], [0, 0, 0]]` |
| Output | `[[0, 0, 0], [0, 0, 0], [0, 0, 0]]` |
| Why | A lonely cell dies of isolation. |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def step(board):
    old = [row[:] for row in board]
    for r, c in all_cells(old):
        live = live_neighbours(old, r, c)
        alive = old[r][c] == 1
        if alive and live in (2, 3):
            board[r][c] = 1
        elif not alive and live == 3:
            board[r][c] = 1
        else:
            board[r][c] = 0
```

| TIME | SPACE |
|---|---|
| O(m * n) | O(m * n) |

**WHY IT IS SLOW**

Time O(m * n) is already optimal; the cost is a second full board, O(m * n) extra space. Storing the next state in a spare bit of each cell removes the copy.

## OPTIMAL SOLUTION · Two-bit encoding in the same board

| TIME | SPACE |
|---|---|
| O(m * n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

## FOLLOW-UP 2 Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

## FOLLOW-UP 3 Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Simulate In-Place Cellular State Updates
#
# Conway's Game of Life on a grid of 0 (dead) and 1
# (alive) cells. A live cell with 2 or 3 live neighbours
# (of its 8 neighbours) stays alive, a dead cell with
# exactly 3 live neighbours becomes alive, and every other
```

```
# cell is dead in the next generation. All cells change at
# the same time. Do it in place with O(1) extra space;
# with wrap=True the grid wraps around its edges.
#
# Signature:
#   def step_in_place(board: list, wrap=False) -> list
#
# Examples:
# [[0, 1, 0], [0, 1, 0], [0, 1, 0]]  ->  [[0, 0, 0], [1,
#     1, 1], [0, 0, 0]]
# [[1, 1], [1, 1]]  ->  [[1, 1], [1, 1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The board is infinite, or huge and mostly empty.
# 2. Why is bit 0 used when counting neighbors?
# 3. Cells wrap around the edges (a torus).
#
# INTERVIEW NOTES
# 'Update every cell at once, in place' means a cell's new
# value must not disturb its neighbors' reads. The trick
# is to store both the old and the new state in one cell
# (two bits), then decode in a second pass. Whenever you
# need old and new values together with no extra grid,
# think 'pack two states into one number'.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
AROUND = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]     # the 8 neighbours
```

```
def live_neighbors(board, r, c, wrap=False):
    rows = len(board)
    cols = len(board[0])
    count = 0
    for dr, dc in AROUND:
        a = r + dr
        b = c + dc
        if wrap:
            a = a % rows                                # a torus: the edges are glued together
            b = b % cols
        elif not (0 <= a < rows and 0 <= b < cols):
            continue
        count += board[a][b] & 1                        # bit 0 is the OLD state, even for cells already upda
ted
    return count


# ---- main question: main function ----
# Idea (Game of Life without a second board): store TWO bits in every
# cell. Bit 0 is the current state, bit 1 will hold the NEXT state.
# Pass 1: count the live neighbours of each cell from bit 0 only
# (value & 1), apply the rules (3 neighbours -> alive; a live cell with 2
# -> stays alive) and write the result into bit 1. Pass 2: shift every cell
# right by one so the new state becomes bit 0.
# Complexity: Time O(m * n), Space O(1) extra (the second bit lives in the board)
def step_in_place(board, wrap=False):
```

```
    rows = len(board)
    cols = len(board[0])
    for r in range(rows):
        for c in range(cols):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            if live == 3 or (alive == 1 and live == 2):
                board[r][c] |= 2                      # bit 1 stores the NEW state
    for r in range(rows):
        for c in range(cols):
            board[r][c] >>= 1                         # the new state moves into bit 0
    return board


# ---- main question: tests ----
def test_main_question():
    assert step_in_place([[0, 1, 0], [0, 1, 0], [0, 1, 0]]) == [[0, 0, 0], [1, 1, 1], [0, 0, 0]]
    assert step_in_place([[1, 1], [1, 1]]) == [[1, 1], [1, 1]]
    assert step_in_place([[0, 0, 0], [0, 1, 0], [0, 0, 0]]) == [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
# ========================================================
# FOLLOW-UP 1: The board is infinite, or huge and mostly empty
# ========================================================
# ---- follow-up 1: function ----
# Answer: store only the LIVE cells in a set. Count, for every cell next
# to a live cell, how many live neighbours it has; the next live cells are
# those with 3, or with 2 when they are alive now.
# Complexity: Time O(L) for L live cells (8 neighbours each), Space O(L)
def step_sparse(live):
    counts = {}
    for r, c in live:
        for dr, dc in AROUND:
            cell = (r + dr, c + dc)
            counts[cell] = counts.get(cell, 0) + 1
    result = set()
    for cell in counts:
        if counts[cell] == 3 or (counts[cell] == 2 and cell in live):
            result.add(cell)
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert step_sparse({(1, 0), (1, 1), (1, 2)}) == {(0, 1), (1, 1), (2, 1)}       # a blinker flips
    assert step_sparse({(0, 0), (0, 1), (1, 0), (1, 1)}) == {(0, 0), (0, 1), (1, 0), (1, 1)}


# ========================================================
# FOLLOW-UP 2: Why is bit 0 used when counting neighbors?
# ========================================================
# Answer: while the board is being updated, some cells already carry
# their NEW state in bit 1. Counting with value & 1 reads only bit 0, the
# OLD state, so every cell is judged by the same generation. Without the
# mask a cell's new bit would be counted by its neighbours too.


# ========================================================
# FOLLOW-UP 3: Cells wrap around the edges (a torus)
# ========================================================
# ---- follow-up 3: tests ----
# Answer: take the neighbour's row and column modulo the board size (the
# `wrap` parameter above).
def test_follow_up_3():
    block = [[1, 1, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [1, 1, 0, 0]]
```

```
    copy_for_wrap = [row[:] for row in block]
    assert step_in_place(copy_for_wrap, wrap=True) == block        # rows 3 and 0 touch: one 2 x 2 block, a
 still life
    assert step_in_place(block, wrap=False) == [[0] * 4 for i in range(4)]     # without wrapping both pair
s die


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

### Sample call

```
solution([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]])
```

### Test cases

| Input | Expected |
|---|---|
| `([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]],)` | `[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]` |
| `([[1, 1], [1, 1]],)` | `[[1, 1], [1, 1]]` |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 18 OF 30

# Digits and characters: peel one at a time and carry

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Digits and characters: peel one at a time and carry | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 31. Check Digits and Combine Ranges — **PracHub 2026-03-29**

H 32. Permutation Sequence — **ColorStack 2025-10-17**

CODE TEMPLATE | GROUP 18 OF 30 | 2 QUESTIONS, ONE PATTERN

# Digits and characters: peel one at a time and carry

## The one idea these questions share

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def add_digits(a, b):                          # two digit lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10)                 # CHANGE 1: what you build per place
        carry = total // 10                    # CHANGE 2: what you carry
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

## 2. Helper functions that usually get written next to it

```
def digits_of(n):
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return out[::-1]


def shift_letter(ch, k):
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **31** | Check Digits and Combine Ranges | palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half |
| **32** | Permutation Sequence | factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 31 of 53 | topic 18 of 30: Digits and characters: peel one at a time and carry | 1 of 2**

Template fit: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half | asked 205 days ago

**HARD | PracHub | Technical Screen | 31 of 53**

**★ TOP 75: rank 4 in the Medium and Hard priority list**

# ★ Check Digits and Combine Ranges

POSTED 2026-03-29 | 205 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 73.6 | 30 DAYS 63.4 | PRIORITY 2/288

## ABOUT THIS QUESTION

**Topic:** Digits and characters: peel one at a time and carry

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

**LeetCode match:** LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Posted on PracHub: 2026-03-29 (updated 2026-05-19)

**Original:** https://prachub.com/coding-questions/check-digits-and-combine-ranges

## QUESTION

You are asked to solve two independent coding tasks.

**Task 1:**

Check Whether an Integer Reads the Same Backward Given an integer **`x`**, return **`true`** if its decimal representation reads the same from left to right and from right to left; otherwise return **`false`**. Requirements and edge cases:

- Negative integers should return **`false`** because of the leading minus sign.
- You may assume **`x`** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

**Task 2:**

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

## QUESTION

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

**Part 1:** Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

**Hints**
A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
**Part 2:** Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
**Input:** [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
**Input:** [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints
Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

## BRUTE FORCE · Task 1: convert to a string and compare with its reverse

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(x):
    s = str(x)
    return x >= 0 and s == s[::-1]
```

| TIME | SPACE |
|---|---|
| O(d) | O(d) |

WHY IT IS SLOW

## BRUTE FORCE · Task 1: convert to a string and compare with its reverse

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

## BRUTE FORCE · Task 2: merge any two overlapping ranges until none overlap

### BRUTE FORCE · PSEUDO-CODE

```
def combine(ranges):
    ranges = list(ranges)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(ranges, 2):
            if overlaps(a, b):
                ranges.remove(a)
                ranges.remove(b)
                ranges.append(union(a, b))
                merged = True
                break
    return sorted(ranges)
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

### WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

## OPTIMAL SOLUTION · Task 1: reverse only the second half arithmetically

| TIME | SPACE |
|---|---|
| O(d) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Task 2: sort by start, then one sweep

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

## FOLLOW-UP 2 Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

## FOLLOW-UP 3 Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check Digits and Combine Ranges
#
# You are asked to solve two independent coding tasks. ###
# Task 1: Check Whether an Integer Reads the Same Backward
# Given an integer x, return true if its decimal
# representation reads the same from left to right and
# from right to left; otherwise return false. Requirements
# and edge cases: - Negative integers should return false
# because of the leading minus sign. - You may assume x
# fits in a signed 32-bit integer. - Try to solve it
# without converting the integer to a string. Example:
# text Input: x = 121 Output: true text Input: x = -121
# Output: false text Input: x = 10 Output: false ### Task
# 2: Combine Overlapping Ranges Given an array of
# intervals intervals, where each interval is represented
# as [start, end], combine all overlapping intervals and
# return an array of non-overlapping intervals that covers
# exactly the same ranges.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Adjacent integer ranges ([1, 2] and [3, 4]) should
# merge too.
# 2. Summarise a sorted list of integers as ranges
# (LeetCode 228).
# 3. Palindrome in another base, or as a linked list.
#
# INTERVIEW NOTES
# Task 1: a number is a palindrome if its second half
# reversed equals its first half, so reverse only half the
# digits (stop when the reversed part catches up) and
# avoid overflow. Negatives and numbers ending in 0 are
# never palindromes. Task 2: after sorting by start, an
# overlap can only be with the last merged range, so one
# pass with a 'current' range is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main functions ----
# PART 1: is the number a palindrome, without turning it into text?
# Idea: negative numbers and positive numbers ending in 0 can never be
# palindromes. Build the REVERSED SECOND HALF of the number digit by digit
# and stop when it catches up with the first half. An odd number of digits
# has a middle digit, which is dropped from the reversed half.
# (base is the number base; follow-up 3 asks for another one.)
```

```python
# Complexity: Time O(d) for d digits (half of them are processed), Space O(1)
def is_palindrome(x, base=10):
    if x < 0 or (x % base == 0 and x != 0):
        return False
    reversed_half = 0
    while x > reversed_half:
        reversed_half = reversed_half * base + x % base
        x //= base
    return x == reversed_half or x == reversed_half // base


# PART 2: combine overlapping ranges.
# Idea: sort by start so overlapping ranges are neighbours. A range that
# starts before (or exactly where) the last merged one ends is absorbed.
# With adjacent=True, ranges that merely touch ([1, 2] and [3, 4])
# are merged too (follow-up 1).
# Complexity: Time O(n log n), Space O(n)
def merge_ranges(intervals, adjacent=False):
    gap = 0
    if adjacent:
        gap = 1
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ---- main question: tests ----
def test_main_question():
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)       # binary 101 and 110
```

```python
    assert merge_ranges([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]            # sharing one point counts
    assert merge_ranges([[8, 10], [1, 3]]) == [[1, 3], [8, 10]]  # any input order
    assert merge_ranges([]) == []


# ========================================================
# FOLLOW-UP 1: Adjacent integer ranges ([1, 2] and [3, 4]) should merge too
# ========================================================
# ---- follow-up 1: tests ----
# Answer: allow a gap of 1 when comparing (the `adjacent` parameter above).
def test_follow_up_1():
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]


# ========================================================
# FOLLOW-UP 2: Summarise a sorted list of integers as ranges (LeetCode 228)
# ========================================================
# ---- follow-up 2: function ----
# Answer: one pass: extend the current range while the next number is
# exactly one more, otherwise start a new range. The same "extend or
# start new" shape as the merge.
# Complexity: Time O(n), Space O(n)
def summarize_ranges(nums):
```

```
    ranges = []
    for x in nums:
        if ranges and x == ranges[-1][1] + 1:
            ranges[-1][1] = x
        else:
            ranges.append([x, x])
    return ranges


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [[0, 2], [4, 5], [7, 7]]
    assert summarize_ranges([]) == []


# =======================================================
# FOLLOW-UP 3: Palindrome in another base, or as a linked list
# =======================================================
# Answer: another base: pass `base` (already supported). Linked list:
```

```
# find the middle with a slow and a fast pointer, reverse the second
# half, and compare the two halves node by node.


# =======================================================
# RUN EVERYTHING
# =======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

### Test cases

| Input | Expected |
|---|---|
| 121 | True |
| -121 | False |

**STUDY PATH: question 32 of 53 | topic 18 of 30: Digits and characters: peel one at a time and carry | 2 of 2**

Template fit: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits | asked 368 days ago | frequency Single report

**HARD | ColorStack | technical interview | 32 of 53**

# Permutation Sequence

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 206/288

## ABOUT THIS QUESTION

**Topic:** Digits and characters: peel one at a time and carry

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 60 Permutation Sequence

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

## QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

**Function signature**

```
def get_permutation(n: int, k: int) -> str:
    ...
```

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

### EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `n = 3, k = 3` |
| **Output** | `"213"` |
| **Why** | `Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".` |

| Example 2 | |
|---|---|
| **Input** | `n = 4, k = 9` |
| **Output** | `"2314"` |
| **Why** | `The 9th permutation of 1234 in lexicographic order is "2314".` |

| Example 3 | |
|---|---|
| **Input** | `n = 3, k = 1` |
| **Output** | `"123"` |
| **Why** | `The first permutation is the sorted sequence.` |

### CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import permutations
def brute_permutation(n, k):
    for i, p in enumerate(
        permutations(range(1, n + 1)), 1
    ):
        if i == k:
            return "".join(map(str, p))
```

| TIME | SPACE |
|---|---|
| O(n! * n) | O(n) |

WHY IT IS SLOW

It builds permutations one by one until it reaches the k-th.

## OPTIMAL SOLUTION · Factorial number system

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Given a permutation, what is its position k in the list?

Run the idea backwards: for every position count how many unused digits are smaller than the current one and multiply by the factorial of the digits that remain; add one at the end.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Permutation Sequence
#
# The set [1, 2, ..., n] has n! distinct permutations.
# Listing them in strictly increasing (lexicographic)
# order and numbering them from 1 , return the k -th
# permutation as a string. Aim for an O(n) -ish
# construction using the factorial number system rather
# than generating all permutations. Function signature:
# def get_permutation(n: int, k: int) -> str: ... Follow-
# up (as reported): A classmate asked whether a brute-
# force "generate all and index" is acceptable; discuss
# why the factorial-base method avoids O(n!) .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Given a permutation, what is its position k in the
# list?
#
# INTERVIEW NOTES
# The permutations in order are grouped: the first (n -
# 1)! start with the smallest digit, the next (n - 1)!
# with the second smallest, and so on. So the index k - 1
# divided by (n - 1)! picks the first digit from the
# sorted remaining digits, the remainder picks the rest:
```

```
# the factorial number system. No permutation needs to be
# generated.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def factorial(n):
    # 1 * 2 * ... * n  (the number of orderings of n different things)
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

```
# ---- main question: main function ----
# Idea (the factorial number system): the permutations of 1..n are in
# blocks. The first digit decides a block of (n - 1)! permutations, so
# k // (n - 1)! says which unused digit comes first; then k % (n - 1)!
# is the position inside that block, and the same step repeats on the
# remaining digits. k starts at 1, so subtract 1 first.
# Complexity: Time O(n^2) (removing from the digit list), Space O(n)
def get_permutation(n, k):
    digits = []
    for d in range(1, n + 1):
        digits.append(d)
    k -= 1
    answer = ""
    for left in range(n, 0, -1):
        block = factorial(left - 1)
        chosen = digits.pop(k // block)
        answer += str(chosen)
        k %= block
    return answer


# ---- main question: tests ----
def test_main_question():
    assert get_permutation(3, 3) == "213"
    assert get_permutation(4, 9) == "2314"
    assert get_permutation(3, 1) == "123"
    assert get_permutation(3, 6) == "321"


# ============================================================
# FOLLOW-UP 1: Given a permutation, what is its position k in the list?
# ============================================================
# ---- follow-up 1: function ----
# Answer: run the idea backwards. For every position, count how many
# UNUSED digits are smaller than the current one and multiply by the
# factorial of the digits that remain after it; add everything up and add
# one at the end.
# Complexity: Time O(n^2), Space O(n)
def permutation_rank(perm):
    unused = []
    for ch in perm:
        unused.append(int(ch))
    unused.sort()
```

```
    rank = 0
    for i in range(len(perm)):
```

```
        smaller = unused.index(int(perm[i]))          # unused digits smaller than this one
        rank += smaller * factorial(len(perm) - i - 1)
        unused.pop(smaller)
    return rank + 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for n in range(1, 7):
        step = factorial(n) // 40
        if step < 1:
            step = 1
        for k in range(1, factorial(n) + 1, step):
            assert permutation_rank(get_permutation(n, k)) == k


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 19 OF 30

# Look at the input once and keep only a small summary (and count your calls)

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Look at the input once and keep only a small summary (and count your calls) | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 33. Find All Bad Pairs Using runTest Oracle — 1Point3Acres 2025-11-27

H 34. Find maximum value in huge text file — PracHub 2025-12-08

CODE TEMPLATE | GROUP 19 OF 30 | 2 QUESTIONS, ONE PATTERN

# Look at the input once and keep only a small summary (and count your calls)

## The one idea these questions share

When the input is too big to hold, read it one item at a time and keep a tiny summary: a maximum, a set from the smaller file, a sorted window. When asking a question is expensive (an oracle), use binary search on how many items you include so the number of calls stays small.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def stream_max(lines):
    best = None                                 # CHANGE 1: the small summary you keep
    for line in lines:                          # a file object gives one line at a time
        line = line.strip()
        if not line:
            continue
        value = int(line)                       # CHANGE 2: parse what matters from the line
        if best is None or value > best:
            best = value                        # CHANGE 3: update the summary
    return best
```

## 2. Helper functions that usually get written next to it

```
def first_failing_prefix(items, passes):        # binary search with an oracle: fewest calls
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if passes(items[:mid + 1]):
            lo = mid + 1
        else:
            hi = mid
    return lo if lo < len(items) else -1
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **33** | Find All Bad Pairs Using runTest Oracle | for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left |
| **34** | Find maximum value in huge text file | the template as written; the file is never loaded as a whole |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 33 of 53 | topic 19 of 30: Look at the input once and keep only a small summary (and count your calls) | 1 of 2**

Template fit: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left | asked 327 days ago | frequency Single report

**HARD | 1Point3Acres | onsite-coding | 33 of 53**

# Find All Bad Pairs Using runTest Oracle

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 252/288

## ABOUT THIS QUESTION

**Topic:** Look at the input once and keep only a small summary (and count your calls)

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, RS

**Original:** https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

## QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
**Output:** every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair containing items from this set.
... binary search to isolate.
```

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for i in range(n):
    for j in range(i + 1, n):
        if not runTest({i, j}):
            bad.append((i, j))
```

| TIME | SPACE |
|---|---|
| O(N^2) calls | O(1) |

WHY IT IS SLOW

Always N(N - 1) / 2 calls. Testing whole groups and binary searching a failing group finds each bad pair in about log N calls.

## OPTIMAL SOLUTION · Group the tests and binary search for partners

| TIME | SPACE |
|---|---|
| **O(K log N + N * G)** | **O(N)** |
| about calls (K bad pairs, G groups) | |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How many calls are needed in the best case and the worst case?

With no bad pairs it takes N - 1 calls, one per test after the first. With K bad pairs each one costs about log N calls to locate, so roughly K log N plus the cost of building clean groups. The brute force is always N(N-1)/2, which is far larger as soon as bad pairs are rare.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find All Bad Pairs Using runTest Oracle
#
# PhD intern coding round 2: given a black-box oracle
# runTest(S) that returns true iff a subset S contains no
# bad pair, identify all bad pairs across N test items
# with as few oracle calls as possible. Items t_1, ...,
# t_N are given. Oracle: runTest(S: Set[Item]) -> bool
# returns true iff S contains zero bad pairs; otherwise
# false. Output: every bad pair (t_i, t_j). Minimize total
# oracle calls. Examples: N=4, bad pairs = {(1,2)}.
# runTest({1,2,3,4}) -> false -> at least one bad pair
# somewhere. runTest({1,3,4}) -> true -> 2 is in every bad
# pair containing items from this set. ... binary search
# to isolate.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls are needed in the best case and the
# worst case?
#
# INTERVIEW NOTES
# Testing every pair costs N(N-1)/2 calls. A single call
# on a whole set answers 'is there any bad pair inside',
# so you can discard good sets fast and binary-search for
# the culprit inside a bad one. Add the tests one at a
# time to groups known to be clean: if a group plus the
# new test fails, binary search the group for a partner;
# otherwise the new test joins it.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
# run_test(set_of_items) is the expensive oracle: it returns True when the
# items in the set pass TOGETHER (no bad pair inside it).
```

```
def first_partner(group, t, run_test):
    # binary search the shortest prefix of the group that fails together
    # with t; its LAST member is a partner of t
    low = 1
    high = len(group)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(group[:mid]) | {t}):
            low = mid + 1
        else:
            high = mid
    return group[low - 1]


def partners(group, t, run_test):
    # all members of the group that are bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        rest.remove(x)                              # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep groups of items that pass together (no bad pair inside a
# group). A new item t is checked against every group: one call says
# "no partner here", otherwise a binary search finds each partner. t joins
# the first group in which it has no partner, or starts a new group.
# Returns the set of bad pairs (smaller item, bigger item).
# Complexity: About n + p log n runTest calls for p bad pairs (best case about n, worst case about n^2 / 2)
, Space O(n)
def find_bad_pairs(items, run_test):
    groups = []
    pairs = set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
                    pairs.add((x, t))
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


# ---- main question: tests ----
class Oracle:
    # a pretend run_test that knows the bad pairs and counts its calls
    def __init__(self, bad):
        self.bad = bad
        self.calls = 0

    def __call__(self, subset):
        self.calls += 1
        ordered = sorted(subset)
```

```
        for i in range(len(ordered)):
            for j in range(i + 1, len(ordered)):
                if (ordered[i], ordered[j]) in self.bad:
                    return False
        return True


def test_main_question():
    bad = {(3, 17), (5, 29)}
    oracle = Oracle(bad)
    assert find_bad_pairs(list(range(30)), oracle) == bad
    assert oracle.calls < 435 // 2                      # far fewer than the 435 possible pairs
    oracle = Oracle(set())
    assert find_bad_pairs(list(range(30)), oracle) == set()
    assert oracle.calls <= 30                           # no bad pairs: about one call per item
    assert find_bad_pairs([1, 2], Oracle({(1, 2)})) == {(1, 2)}


# =============================================================
# FOLLOW-UP 1: How many calls are needed in the best case and the worst case?
# =============================================================
# Answer: best case, no bad pair: about one call per item (n calls), because
# each new item passes with the one group. Worst case, many bad pairs: each
# bad pair costs about log2(n) calls to find, plus the calls that prove a
# group has no more partners; with p bad pairs about p * log2(n) + n calls,
# and in the extreme (every pair bad) about n^2 / 2 calls.


# =============================================================
# RUN EVERYTHING
# =============================================================
```

```
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## PREPARATION

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

**STUDY PATH: question 34 of 53 | topic 19 of 30: Look at the input once and keep only a small summary (and count your calls) | 2 of 2**

Template fit: the template as written; the file is never loaded as a whole | asked 316 days ago

HARD | PracHub | Technical Screen | 34 of 53

# Find maximum value in huge text file

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 245/288

## ABOUT THIS QUESTION

**Topic:** Look at the input once and keep only a small summary (and count your calls)

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**LeetCode match:** No LC match (systems / scalability question; the Hard tag looks inflated)

Posted on PracHub: 2025-12-08 (updated 2026-06-27)

**Original:** https://prachub.com/interview-questions/find-maximum-value-in-huge-text-file

## QUESTION

A text file is too large to load into memory (tens or hundreds of gigabytes). Each line holds one signed 64-bit integer. Return the largest value by reading the file sequentially, holding only a small amount in memory.

### Function signature

def max_in_file(path: str) -> int

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `nums.txt contains the lines:`<br>`5`<br>`-3`<br>`12`<br>`7` |
| **Output** | `max_in_file(path) -> 12` |
| **Why** | `Only the current line and the best value so far are in memory, so the file size does not matter.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(path):
    numbers = []
    for line in open(path):
        numbers.append(int(line))  # in memory
    return max(numbers)
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

WHY IT IS SLOW

Needs memory for the whole file, which is exactly what is not available at tens of gigabytes. Streaming with one running maximum uses O(1) memory.

**OPTIMAL SOLUTION · Single machine: stream the file, keep one running maximum**

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(1)** plus the read buffer |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**OPTIMAL SOLUTION · Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce)**

| TIME | SPACE |
| --- | --- |
| **O(n / workers)** plus I/O limits | **O(workers)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

**FOLLOW-UP 1 Edge cases to mention.**

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

**FOLLOW-UP 2 How do you scale to many machines and handle failures?**

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. **Cost:** O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

**FOLLOW-UP 3 Is it I/O-bound or CPU-bound, and why does that matter?**

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. **Cost:** reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

**FOLLOW-UP 4 The file has billions of lines and one machine is too slow.**

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. **Cost:** O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

**FOLLOW-UP 5 Now return the K largest values instead of only the maximum.**

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find maximum value in huge text file
#
# A text file is too large to load into memory (tens or
# hundreds of gigabytes). Each line holds one signed
# 64-bit integer. Return the largest value by reading the
# file sequentially, holding only a small amount in
# memory.
#
# Signature:
#   def max_in_file(path: str) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. Edge cases to mention.
# 2. How do you scale to many machines and handle
# failures?
# 3. Is it I/O-bound or CPU-bound, and why does that
# matter?
# 4. The file has billions of lines and one machine is too
# slow.
# 5. Now return the K largest values instead of only the
# maximum.
#
# INTERVIEW NOTES
# The answer is a running maximum, which needs one
# variable, so memory should not depend on the file size.
# Read the file as a stream. To go faster across cores,
# give each worker a byte range that starts and ends on
# line boundaries; the global maximum is the maximum of
# the partial maxima.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import os
import tempfile


# ========================================================
# MAIN QUESTION
# ========================================================
```

```
# ---- main question: main function ----
# Idea: never load the file. Read it line by line (the file object reads
# in buffered chunks) and keep ONE variable: the maximum so far. It
# starts as None, not 0, so that a file of only negative numbers, or of one
# line, gives the right answer. Blank lines are ignored (state this
# contract). An empty file has no maximum: None.
# Complexity: Time O(n) for n lines (one pass), Space O(1)
def max_in_file(path):
    best = None
    with open(path) as f:
        for line in f:
            text = line.strip()
            if text == "":
```

```
                continue
            value = int(text)
            if best is None or value > best:
                best = value
    return best


# ---- main question: tests ----
def write_file(lines):
    # a small test file; returns its path
    f = tempfile.NamedTemporaryFile("w", delete=False)
    f.write("".join(lines))
    f.close()
    return f.name


def test_main_question():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
    assert max_in_file(p) == 99
    q = write_file(["-7\n", "-2\n", "-9\n"])
    assert max_in_file(q) == -2                          # all negative numbers
    big = write_file([str(2 ** 63 - 1) + "\n", str(-2 ** 63) + "\n"])
    assert max_in_file(big) == 2 ** 63 - 1               # extreme 64-bit values
    e = write_file([])
    assert max_in_file(e) is None                        # an empty file has no maximum
    for name in (p, q, big, e):
        os.remove(name)


# =====================================================
# FOLLOW-UP 1: Edge cases to mention
# =====================================================
```

```
# Answer: an empty file, blank or malformed lines (skip or fail? say which),
# one line only, ALL numbers negative (do not start the maximum at 0),
# numbers bigger than 64 bits, a last line without a newline, and a file
# that is being written while you read it.


# =====================================================
# FOLLOW-UP 2: How do you scale to many machines and handle failures?
# =====================================================


# =====================================================
# FOLLOW-UP 3: Is it I/O-bound or CPU-bound, and why does that matter?
# =====================================================
# Answer: reading a file and doing one comparison per line is I/O-BOUND:
# the disk or network is the limit, not the processor. So more threads on
# one machine do not help; more machines (each reading its own part of
# the data) do. Tasks are idempotent, so a failed worker is simply retried.


# =====================================================
# FOLLOW-UP 4: The file has billions of lines and one machine is too slow
# =====================================================
# ---- follow-up 4: helper functions ----
def max_in_range(path, start, end):
    # one worker: the maximum of the lines that START inside [start, end)
    best = None
    with open(path, "rb") as f:
        if start > 0:
            f.seek(start - 1)
            f.readline()                 # we may be in the middle of a line: the worker before owns it
        while f.tell() < end:
```

```
            line = f.readline()
            if not line:
                break
            if line.strip() != b"":
                value = int(line)
                if best is None or value > best:
                    best = value
    return best


# ---- follow-up 4: function ----
# Answer: split the file by BYTE offsets across the machines (each one
# skips to the next newline), compute a local maximum on each, and take
```

```
# the maximum of those few numbers. Here the "machines" are just
# loops over the byte ranges.
# Complexity: Time O(n / workers) per worker plus O(workers) to combine, Space O(1) per worker
def parallel_max(path, workers=4):
    size = os.path.getsize(path)
    best = None
    for i in range(workers):
        start = size * i // workers
        end = size * (i + 1) // workers
        local = max_in_range(path, start, end)
        if local is not None and (best is None or local > best):
            best = local
    return best


# ---- follow-up 4: tests ----
def test_follow_up_4():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
    assert parallel_max(p, 3) == 99
    e = write_file([])
    assert parallel_max(e) is None
    numbers = []
    lines = []
    for i in range(2000):
        number = (i * 7919) % 10007 - 5000
        numbers.append(number)
        lines.append(str(number) + "\n")
    r = write_file(lines)
    for workers in (1, 2, 5, 8):                          # every worker count agrees
        assert parallel_max(r, workers) == max(numbers)
    for name in (p, e, r):
        os.remove(name)


# ========================================================
# FOLLOW-UP 5: Now return the K largest values instead of only the maximum
# ========================================================
# ---- follow-up 5: function ----
# Answer: keep a MIN-heap of size K while streaming: push a value while
# the heap is not full, otherwise replace the root (the smallest of the K
# best) when the new value is larger. Memory O(K), time O(n log K).
# For a parallel version, merge the per-worker heaps.
# Complexity: Time O(n log k), Space O(k)
def top_k_in_file(path, k):
    heap = []
```

```
    with open(path) as f:
        for line in f:
            text = line.strip()
            if text == "":
                continue
```

```
            value = int(text)
            if len(heap) < k:
                heapq.heappush(heap, value)
            elif value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)


# ---- follow-up 5: tests ----
def test_follow_up_5():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
    assert top_k_in_file(p, 2) == [99, 12]
    assert top_k_in_file(p, 10) == [99, 12, 5, -3]
    os.remove(p)


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FULL QUESTION TEXT FROM THE SITE

You are given the path to a very large text file on disk. The file has the following properties:

Each line contains exactly one signed 64-bit integer (in decimal).
The file size can be tens or hundreds of gigabytes, so it cannot be fully loaded into memory.
You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).
Tasks
Single-machine solution
Design an algorithm to compute the maximum integer value in this file.
Assume you have limited RAM (much smaller than the file size).
Discuss how you would read the file and track the result.
Give high-level pseudocode.
Analyze time and space complexity.
Performance improvement / parallelization
Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.
How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?
Explain how you would combine partial results to get the global maximum.
Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file).

Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 20 OF 30

# Grid DP: each cell from its neighbours above and to the left

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Grid DP: each cell from its neighbours above and to the left | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 35. Find the Longest Straight Run of One Color — **PracHub 2026-08-23**

H 36. Count same-color squares in a character grid — **PracHub 2025-11-22**

CODE TEMPLATE | GROUP 20 OF 30 | 2 QUESTIONS, ONE PATTERN

# Grid DP: each cell from its neighbours above and to the left

## The one idea these questions share

Walk the grid row by row and keep only the previous row. A cell's value depends on the cell above, the cell to the left, and the diagonal. Change the condition for a cell to count and the formula that combines the three neighbours.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def grid_dp(g):
    m, n = len(g), len(g[0])
    prev, total = [0] * n, 0
    for i in range(m):
        cur = [1] * n
        for j in range(n):
            if i and j and g[i][j] == g[i - 1][j] == g[i][j - 1] == g[i - 1][j - 1]:   # CHANGE 1
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])                      # CHANGE 2
        total += sum(cur)                                                               # CHANGE 3
        prev = cur
    return total
```

## 2. Helper functions that usually get written next to it

```
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))      # the four line directions on a grid


def inside(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **35** | Find the Longest Straight Run of One Color | the same four run lengths; extend a run when the neighbour in that direction has the same colour |
| **36** | Count same-color squares in a character grid | exactly the same recurrence on characters |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 35 of 53 | topic 20 of 30: Grid DP: each cell from its neighbours above and to the left | 1 of 2**

Template fit: the same four run lengths; extend a run when the neighbour in that direction has the same colour | asked 58 days ago

HARD | PracHub | Technical Screen | 35 of 53

# Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 166/288

## ABOUT THIS QUESTION

**Topic:** Grid DP: each cell from its neighbours above and to the left

**Optimal answer from:** the interview site's own solution

**LeetCode match:** LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

**Original:** https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

## QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

### Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

### Output

Return the maximum length as an integer.

### Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

### Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

### Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

EXAMPLES

## QUESTION

| Example 1 | |
|---|---|
| Input | ['RRRG', 'RRBB', 'GRRR', 'BGBR'] |
| Output | 4 |
| Why | Four R cells lie on the main diagonal. |

| Example 2 | |
|---|---|
| Input | ['RGB', 'BGR', 'RGB'] |
| Output | 3 |
| Why | Three G cells in the middle column. |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

| TIME | SPACE |
|---|---|
| O(m * n * min(m, n)) | O(1) |

### WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

## OPTIMAL SOLUTION · DP per direction with a rolling row

| TIME | SPACE |
|---|---|
| O(m * n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

## FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

## FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, down-right, down-left


def all_runs(grid):
    # every maximal straight run of one colour as (length, start cell, direction).
    # A run is walked only from its FIRST cell (the cell before it in that
    # direction has another colour or does not exist), so the total work is
```

```
    # linear in the number of cells.
    rows = len(grid)
    cols = len(grid[0])
    found = []
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                before_r = r - dr
                before_c = c - dc
                if 0 <= before_r < rows and 0 <= before_c < cols:
                    if grid[before_r][before_c] == grid[r][c]:
                        continue                        # not the first cell of its run
                length = 0
                a = r
                b = c
                while 0 <= a < rows and 0 <= b < cols and grid[a][b] == grid[r][c]:
                    length += 1
                    a += dr
                    b += dc
                found.append((length, (r, c), (dr, dc)))
    return found


# ---- main question: main function ----
# Idea: look for the longest straight line (across, down, or along
# either diagonal) made of one colour. Walk each run once from its first
# cell and keep the longest.
# Complexity: Time O(m * n), Space O(m * n)
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    best = 0
    for length, start, direction in all_runs(grid):
        if length > best:
            best = length
    return best
```

```
# ---- main question: tests ----
GRID = ["RRRG", "RRBB", "GRRR", "BGBR"]


def test_main_question():
    assert longest_run(GRID) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0


# ======================================================
# FOLLOW-UP 1: Return where the run is, not only its length
# ======================================================
# ---- follow-up 1: function ----
# Answer: all_runs already starts a walk only at a cell that begins a run,
# which keeps the work linear; just keep the start cell and direction of
# the best run.
# Complexity: Time O(m * n), Space O(m * n)
def longest_run_where(grid):
    if not grid or not grid[0]:
        return 0, None, None
    best = None
    for run in all_runs(grid):
        if best is None or run[0] > best[0]:
            best = run
    return best
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    length, start, direction = longest_run_where(GRID)
    assert length == 4
    r, c = start
    for i in range(length):                          # walk the run and check the colour
        assert GRID[r + i * direction[0]][c + i * direction[1]] == GRID[r][c]


# ============================================================
# FOLLOW-UP 2: Require the run to be at least K and count how many such runs exist
# ============================================================
# ---- follow-up 2: function ----
# Answer: count a run ONCE, at its first cell, when its length is at
# least K (all_runs already lists each run once). Counting at every cell
```

```
# would count a long run many times.
# Complexity: Time O(m * n), Space O(m * n)
def count_runs_at_least(grid, k):
    if not grid or not grid[0]:
        return 0
    count = 0
    for length, start, direction in all_runs(grid):
        if length >= k:
            count += 1
    return count


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_runs_at_least(GRID, 3) == 4         # runs of length 3 or more are counted once each


# ============================================================
# FOLLOW-UP 3: The grid is huge and arrives row by row
# ============================================================
# ---- follow-up 3: function ----
# Answer: the run length ending at a cell depends only on the PREVIOUS
# row, so keep just the run lengths of the last row (across, down,
# down-right, down-left): O(width) memory for any number of rows.
# Complexity: Time O(m * n), Space O(n) (only the previous row is kept)
def longest_run_rows(row_stream):
    best = 0
    previous = None
    down = []
    diagonal_right = []
    diagonal_left = []
    for row in row_stream:
        n = len(row)
        across_now = [1] * n
        down_now = [1] * n
        right_now = [1] * n
        left_now = [1] * n
        for c in range(n):
            if c > 0 and row[c] == row[c - 1]:
                across_now[c] = across_now[c - 1] + 1
            if previous is not None:
                if row[c] == previous[c]:
                    down_now[c] = down[c] + 1
                if c > 0 and row[c] == previous[c - 1]:
                    right_now[c] = diagonal_right[c - 1] + 1
```

```
                if c + 1 < n and row[c] == previous[c + 1]:
                    left_now[c] = diagonal_left[c + 1] + 1
            for length in (across_now[c], down_now[c], right_now[c], left_now[c]):
                if length > best:
                    best = length
        previous = row
        down = down_now
        diagonal_right = right_now
        diagonal_left = left_now
    return best


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_run_rows(iter(GRID)) == 4
    assert longest_run_rows(["RBG", "BGR", "GRB"]) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def longestColorRun(grid: list[str]) -> int:
```

### Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

### Test cases

| Input | Expected |
|---|---|
| `(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)` | 4 |
| `(['RBG', 'BGR', 'GRB'],)` | 3 |

**ALSO REPORTED · 1 more**

**Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report**

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

**ALSO REPORTED · 1 more**

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.

Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

**STUDY PATH: question 36 of 53 | topic 20 of 30: Grid DP: each cell from its neighbours above and to the left | 2 of 2**

Template fit: exactly the same recurrence on characters | asked 332 days ago

**HARD | PracHub | Technical Screen | 36 of 53**

# Count same-color squares in a character grid

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 76/288

## ABOUT THIS QUESTION

**Topic:** Grid DP: each cell from its neighbours above and to the left

**Optimal answer from:** the interview site's own solution

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #1277 Count Square Submatrices with All Ones, LeetCode #221 Maximal Square

**LeetCode match:** LC 1277 Count Square Submatrices with All Ones / LC 221 Maximal Square

Posted on PracHub: 2025-11-22 (updated 2026-06-13)

**Original:** https://prachub.com/coding-questions/count-same-color-squares-in-a-character-grid

## QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

### EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

## QUESTION

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

**Hints**

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

| TIME | SPACE |
|---|---|
| O(m * n * min(m, n)^3) | O(1) |

### WHY IT IS SLOW

The same cells are rechecked for every size: O(m * n * min(m, n)^3). A table of the largest same-colour square ending at each cell, summed over all cells, is O(m * n).

## OPTIMAL SOLUTION · DP: side of the largest square ending at each cell, summed

| TIME | SPACE |
|---|---|
| O(m * n) | O(n)<br>with a rolling row |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

## FOLLOW-UP 2 Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

## FOLLOW-UP 3 Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count same-color squares in a character
# grid
#
```

```
# You are given a 2D grid (matrix) of characters. Each
# character represents a color: cells with the same
# character are considered the same color. Formally: - The
# grid has m rows and n columns. - grid[i][j] is an
# alphabetic character (e.g., 'a'–'z' or 'A'–'Z'). A
# **square** in this grid is defined as a contiguous k × k
# submatrix aligned with the grid axes, for some integer k
# >= 1. A square is **monochromatic** if **all** of its
# cells contain the **same** character (i.e., the same
# color). Task: - Count and return the total number of
# monochromatic squares in the grid. - Count all sizes of
# squares (1×1, 2×2, ..., up to the largest possible that
# fits in the grid). Example (just for clarity, not
# necessarily exhaustive): If the grid is: a a b a a b b b
# b Then some monochromatic squares include: - All 1×1
# cells (each single cell is a monochromatic square).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the largest square (LeetCode 221).
# 2. Allow at most one different cell inside a square.
# 3. Count only squares of colour c, or only side >= 2.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That is also
# the number of squares ending at that corner (one per
# size from 1 up to it). A square of side k at (i, j)
# needs squares of side k - 1 at the three cells above,
# left and above-left, all of the same colour, hence the
# min of those three plus one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def square_table(grid):
    # side[i][j] = the side of the biggest same-colour square whose
    # BOTTOM-RIGHT corner is (i, j)
    sides = []
    for i in range(len(grid)):
        row_sides = [1] * len(grid[i])
        for j in range(len(grid[i])):
            ch = grid[i][j]
            if i > 0 and j > 0 and grid[i - 1][j] == ch and grid[i][j - 1] == ch and grid[i - 1][j - 1] ==
ch:
                # the cells above, to the left and diagonally must match too
                smallest = sides[i - 1][j]
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
```

```
# Idea: if a cell matches its top, left and top-left neighbours its square
# side is 1 + the smallest of their sides; otherwise it is 1. A cell with
# side s is the corner of s squares (sizes 1..s), so adding up all the
# sides counts every square exactly once, at its bottom-right corner.
# Complexity: Time O(m * n), Space O(m * n)
def count_same_color_squares(grid):
    total = 0
    for row_sides in square_table(grid):
        for side in row_sides:
            total += side
    return total


# ---- main question: tests ----
def test_main_question():
    assert count_same_color_squares(["aab", "aab", "bbb"]) == 10      # 9 single cells plus the 2 x 2 block
 of a's
    assert count_same_color_squares(["zzz", "zzz", "zzz"]) == 14      # 9 + 4 + 1
    assert count_same_color_squares(["a"]) == 1


# =========================================================
# FOLLOW-UP 1: Only the largest square (LeetCode 221)
# =========================================================
# ---- follow-up 1: function ----
```

```
# Answer: keep the biggest number of the same table instead of the sum.
# Complexity: Time O(m * n), Space O(m * n)
def largest_square(grid):
    best = 0
    for row_sides in square_table(grid):
        for side in row_sides:
            if side > best:
                best = side
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_square(["aab", "aab", "bbb"]) == 2


# =========================================================
# FOLLOW-UP 2: Allow at most one different cell inside a square
# =========================================================
# Answer: keep two tables per cell: the biggest square with no mistake and
# the biggest with one mistake (a different cell inside); the second one
# can also grow from a neighbour's "no mistake" square when this cell is
# the one that differs.


# =========================================================
# FOLLOW-UP 3: Count only squares of colour c, or only side >= 2
# =========================================================
# ---- follow-up 3: function ----
# Answer: filter while adding up: count only cells of colour c, and for
# squares of side at least m add side - m + 1 per cell (the sizes m..side).
# Complexity: Time O(m * n), Space O(m * n)
def count_of_color(grid, color, min_side=1):
    total = 0
    sides = square_table(grid)
    for i in range(len(grid)):
        for j in range(len(grid[i])):
            if grid[i][j] == color and sides[i][j] >= min_side:
                total += sides[i][j] - min_side + 1
```

```
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = ["aab", "aab", "bbb"]
```

```
    assert count_of_color(g, "a") == 5
    assert count_of_color(g, "b") == 5
    full = ["zzz", "zzz", "zzz"]
    assert count_of_color(full, "z", 2) == 5                # 4 squares of side 2 and 1 of side 3
    assert count_of_color(full, "q") == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(grid: list[str]) -> int:
SAMPLE INPUT
([])
```

### Test cases

| Input | Expected |
|---|---|
| `([])` | `0` |
| `(["aab", "aab", "bbb"])` | `10` |

**ALSO REPORTED · 1 more**

**Count Same-Color Squares in Matrix | 1Point3Acres | onsite-coding | Low**

https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

Variant reported in both intern VO and an additional NG round.
Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

Preparation: Write the all-1-squares DP from memory in under 8 min.
Generalize to multi-color in <2 min — only the equality check changes.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 21 OF 30

# Counting pairs by looking up each partner in a Counter or a bucket

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Counting pairs by looking up each partner in a Counter or a bucket | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 37. Maximum-Area Rectangle from Planar Points 1Point3Acres 2026-09-29

CODE TEMPLATE | GROUP 21 OF 30 | 1 QUESTIONS, ONE PATTERN

# Counting pairs by looking up each partner in a Counter or a bucket

## The one idea these questions share

For each value, compute the partner it would need (v + k, 2v, the other corner of a square) and look it up in a Counter or a set. When the partner depends on two things, enumerate the pairs once and bucket them by a shared key.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def count_pairs(nums, k):
    counts = Counter(nums)                        # CHANGE 1: what you index
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)     # CHANGE 2: the partner you look up
```

## 2. Helper functions that usually get written next to it

```
from collections import defaultdict


def bucket_pairs(points):                    # group point pairs by a shared key (here: the midpoint and length)
    buckets = defaultdict(list)
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            p, q = points[i], points[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            buckets[key].append((p, q))
    return buckets
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **37** | Maximum-Area Rectangle from Planar Points | bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 37 of 53 | topic 21 of 30: Counting pairs by looking up each partner in a Counter or a bucket | 1 of 1**

Template fit: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle | asked 21 days ago | frequency Low

**HARD | 1Point3Acres | onsite-coding | 37 of 53**

# Maximum-Area Rectangle from Planar Points

**ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 201/288**

## ABOUT THIS QUESTION

**Topic:** Counting pairs by looking up each partner in a Counter or a bucket

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #963 Minimum Area Rectangle II

**LeetCode match:** LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

## QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

**Input:** a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

### EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `[(1, 1), (1, 3), (3, 1), (3, 3)]` |
| **Output** | `4` |
| **Why** | `The four points form a 2 x 2 axis-aligned square.` |

| Example 2 | |
|---|---|
| **Input** | `[(0, 1), (1, 0), (2, 1), (1, 2)]` |
| **Output** | `2` |
| **Why** | `A tilted rectangle: a square with diagonal 2 has area 2.` |

| Example 3 | |
|---|---|
| **Input** | `[(0, 0), (1, 0), (0, 1)]` |
| **Output** | `0` |
| **Why** | `Three points never make a rectangle.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

### WHY IT IS SLOW

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

## OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

| TIME | SPACE |
|---|---|
| O(n^2) pairs (worst case O(n^3) if every pair shares a bucket) | O(n^2) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
```

```
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def cross(o, p, q):
    # twice the signed area of the triangle o, p, q
    ax = p[0] - o[0]
    ay = p[1] - o[1]
    bx = q[0] - o[0]
    by = q[1] - o[1]
    return ax * by - ay * bx
```

```
def all_rectangle_areas(points):
    # the area of every rectangle that the points form (the sides need not be
    # parallel to the axes). Idea: the two diagonals of a rectangle have the
    # SAME middle and the SAME length, so put every pair of points in a bucket
    # keyed by (sum of x, sum of y, squared length); any two pairs in one bucket
    # are the diagonals of a rectangle.
    unique = []
    seen = set()
    for x, y in points:
        if (x, y) not in seen:                          # duplicates cannot help
            seen.add((x, y))
            unique.append((x, y))
    buckets = {}
    for i in range(len(unique)):
        for j in range(i + 1, len(unique)):
            p = unique[i]
            q = unique[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            if key not in buckets:
                buckets[key] = []
            buckets[key].append((p, q))
    areas = []
    for pairs in buckets.values():
        for a in range(len(pairs)):
            for b in range(a + 1, len(pairs)):
                p, q = pairs[a]
                r, s = pairs[b]
                area = cross(p, r, s)                   # the two sides leaving p are p-r and p-s: their cro
ss product is the rectangle's area
                if area < 0:
                    area = -area
                areas.append(area)
    return areas


# ---- main question: main function ----
# The biggest rectangle (any angle) with corners among the points, or 0.
```

```python
# Complexity: Time O(n^2), Space O(n)
def max_rect_area(points):
    best = 0
    for area in all_rectangle_areas(points):
        if area > best:
            best = area
    return best
```

```python
# the related variant: the SMALLEST area (LeetCode 963)
# Complexity: Time O(n^2), Space O(n)
def min_rect_area(points):
    smallest = 0
    for area in all_rectangle_areas(points):
        if smallest == 0 or area < smallest:
            smallest = area
    return smallest


# ---- main question: tests ----
def test_main_question():
    square = [(0, 0), (1, 1), (0, 2), (-1, 1)]                # a tilted square of area 2
    assert max_rect_area(square) == 2
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]           # a 2 x 1 rectangle plus a stray point
    assert max_rect_area(rect) == 2
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0       # collinear points make no rectangle
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2                            # the minimum version picks the small one
    assert max_rect_area(two) == 6


# =========================================================
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# =========================================================
# Answer: not in general. The answer can depend on every pair of points, and
# even deciding whether ANY rectangle exists is believed to need about n^2
# work, so the bucket method is the practical optimum.


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 22 OF 30

# Tree DP: build a parent order once, then add up from the leaves

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Tree DP: build a parent order once, then add up from the leaves | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 38. Compute distance-sum from every tree node — PracHub 2026-01-22

H 39. Tree Distance Sum (Re-root DP) — 1Point3Acres 2026-01-19

CODE TEMPLATE | GROUP 22 OF 30 | 2 QUESTIONS, ONE PATTERN

# Tree DP: build a parent order once, then add up from the leaves

## The one idea these questions share

Get an order where every parent comes before its children (a BFS order from the root), then sweep it backwards so every child is finished before its parent. Subtree sizes and sums fall out of that one backwards sweep; the re-rooting questions add a second forward sweep.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def subtree_sums(n, adj, root=0):
    parent, order = tree_order(n, adj, root)
    size, down = [1] * n, [0] * n
    for u in reversed(order):             # children before parents
        p = parent[u]
        if p != -1:
            size[p] += size[u]
            down[p] += down[u] + size[u]  # CHANGE: what you add up
    return parent, order, size, down
```

## 2. Helper functions that usually get written next to it

```
def tree_order(n, adj, root=0):
    parent, order = {root: -1}, [root]
    for u in order:                       # BFS order: parents before children
        for v in adj[u]:
            if v not in parent:
                parent[v] = u
                order.append(v)
    return parent, order


def adjacency_from_edges(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **38** | Compute distance-sum from every tree node | down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child]) |
| **39** | Tree Distance Sum (Re-root DP) | exactly the same two sweeps (this is the re-rooting version of the question) |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 38 of 53 | topic 22 of 30: Tree DP: build a parent order once, then add up from the leaves | 1 of 2**

Template fit: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child]) | asked 271 days ago

HARD | IN APP LC 834 | PracHub | Technical Screen | 38 of 53

# Compute distance-sum from every tree node

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 143/288

## ABOUT THIS QUESTION

**Topic:** Tree DP: build a parent order once, then add up from the leaves

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #834 Sum of Distances in Tree

**LeetCode match:** LC 834 Sum of Distances in Tree

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

**Original:** https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

## QUESTION

A tree has n nodes numbered 0 to n - 1 and n - 1 edges (u, v). For every node, return the sum of the distances from that node to all the other nodes.

**Function signature**

def distance_sums(n: int, edges: list) -> list

EXAMPLES

| Example 1 | |
|---|---|
| Input | `4, [(0, 1), (0, 2), (2, 3)]` |
| Output | `[4, 6, 4, 6]` |
| Why | `Node 0: 1 + 1 + 2 = 4. Node 1 is a leaf: 1 + 2 + 3 = 6. Node 2: 1 + 2 + 1 = 4. Node 3: 1 + 2 + 3 = 6.` |

| Example 2 | |
|---|---|
| Input | `3, [(0, 1), (1, 2)]` |
| Output | `[3, 2, 3]` |
| Why | `A path of three nodes: the middle one is closest to the others.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = []
    for node in range(n):
        dist = bfs(adj, node)
        ans.append(sum(dist))
    return ans
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

## BRUTE FORCE

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

## OPTIMAL SOLUTION · Re-rooting with subtree sizes

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

## FOLLOW-UP 2 The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

## FOLLOW-UP 3 Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute distance-sum from every tree node
#
# A tree has n nodes numbered 0 to n - 1 and n - 1 edges
# (u, v). For every node, return the sum of the distances
# from that node to all the other nodes.
#
# Signature:
#   def distance_sums(n: int, edges: list) -> list
#
# Examples:
# 4, [(0, 1), (0, 2), (2, 3)]  ->  [4, 6, 4, 6]
# 3, [(0, 1), (1, 2)]  ->  [3, 2, 3]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Find the node with the smallest distance sum (the
# tree's centroid).
# 2. The tree has weighted edges.
# 3. Maximum distance (eccentricity) from each node
# instead of the sum.
#
# INTERVIEW NOTES
# Computing the sum of distances from each node separately
# costs O(n) per node. Instead compute it once for the
# root, then derive the neighbour's answer in O(1): moving
# the root from a parent to its child makes every node in
# the child's subtree one step closer and every other node
# one step farther. That needs only the subtree sizes.
```

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    # an edge is [u, v] or [u, v, weight]; neighbours are (node, weight)
    adj = []
    for i in range(n):
        adj.append([])
    for e in edges:
        u, v = e[0], e[1]
        w = 1
        if len(e) > 2:
            w = e[2]
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (in a tree one walk is enough)
    dist = {s: 0}
    stack = [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# ---- main question: main function ----
# Idea (re-rooting): root the tree at 0 and compute size[v] (the number
# of nodes below v) and the answer for the root: the sum of its
# distances. Moving the "root" from a parent p across an edge of weight w
# to its child u makes the size[u] nodes below u closer by w and the
# other n - size[u] nodes farther by w:
#   answer[u] = answer[p] + w * (n - 2 * size[u]).
# Edges may carry a weight (follow-up 2); an edge without one costs 1.
# Complexity: Time O(n), Space O(n)
def distance_sums(n, edges):
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    up = [0] * n                                    # weight of the edge to the parent
    order = [0]                                     # parents before children
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v] = u
                up[v] = w
                order.append(v)
    size = [1] * n
    for i in range(n - 1, -1, -1):                  # children before parents
        u = order[i]
```

```
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    answer = [0] * n
    for d in distances_from(adj, 0).values():
        answer[0] += d                              # the root's answer is a plain sum of distances
    for u in order[1:]:
        answer[u] = answer[parent[u]] + up[u] * (n - 2 * size[u])
    return answer


# ---- main question: tests ----
EDGES = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]


def test_main_question():
    assert distance_sums(6, EDGES) == [8, 12, 6, 10, 10, 10]
    assert distance_sums(1, []) == [0]


# ========================================================
# FOLLOW-UP 1: Find the node with the smallest distance sum (the tree's centroid)
# ========================================================
# ---- follow-up 1: function ----
# Answer: it is the position of the smallest number in the answer list
# (the centroid, which you can also find by walking toward the heavier
# subtree).
# Complexity: Time O(n), Space O(n)
def best_center(n, edges):
    sums = distance_sums(n, edges)
    best = 0
    for v in range(n):
        if sums[v] < sums[best]:
            best = v
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_center(6, EDGES) == 2


# ========================================================
# FOLLOW-UP 2: The tree has weighted edges
# ========================================================
# Answer: distance_sums already multiplies the shift by the weight of
```

```
# the edge it crosses.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    weighted = [[0, 1, 2], [1, 2, 3]]               # the path 0 -(2)- 1 -(3)- 2
    assert distance_sums(3, weighted) == [7, 5, 8]


# ========================================================
# FOLLOW-UP 3: Maximum distance (eccentricity) from each node instead of the sum
# ========================================================
# ---- follow-up 3: helper functions ----
def farthest_node(dist):
    # the node with the largest distance in a {node: distance} table
    far = None
    for node in dist:
        if far is None or dist[node] > dist[far]:
            far = node
    return far
```

```
# ---- follow-up 3: function ----
# Answer: the farthest node from ANY node is one end of the tree's
# diameter. Find both ends with two walks, measure from each end, and the
# answer for a node is the larger of its two distances. O(n).
# Complexity: Time O(n) (three searches), Space O(n)
def eccentricities(n, edges):
    adj = build_adjacency(n, edges)
    from_start = distances_from(adj, 0)
    end_a = farthest_node(from_start)
    from_a = distances_from(adj, end_a)
    end_b = farthest_node(from_a)
    from_b = distances_from(adj, end_b)
    result = []
    for v in range(n):
        if from_a[v] > from_b[v]:
            result.append(from_a[v])
        else:
            result.append(from_b[v])
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert eccentricities(6, EDGES) == [2, 3, 2, 3, 3, 3]
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

### Sample call

```
solution(1, [])
```

### Test cases

| Input | Expected |
|---|---|
| (1, []) | [0] |
| (4, [[0, 1], [1, 2], [1, 3]]) | [5, 3, 5, 5] |

**ALSO REPORTED · 1 more**

**Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report**

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the $O(n^2)$ and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree).

Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

**STUDY PATH: question 39 of 53 | topic 22 of 30: Tree DP: build a parent order once, then add up from the leaves | 2 of 2**

Template fit: exactly the same two sweeps (this is the re-rooting version of the question) | asked 274 days ago | frequency Single report

**HARD** | **IN APP** **LC 834** | **1Point3Acres** | **onsite-coding** | **39 of 53**

# Tree Distance Sum (Re-root DP)

ASKED 2026-01-19 | 274 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 144/288

## ABOUT THIS QUESTION

**Topic:** Tree DP: build a parent order once, then add up from the leaves

**Optimal answer from:** your app's interview approach (LeetCode #834)

**LeetCode match:** LC 834 Sum of Distances in Tree

Frequency: Single report | Last asked: 2026-01-19 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

## QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

**Input:** undirected tree with N nodes and N-1 edges.
**Output:** array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = [0] * n
    for s in range(n):
        dist = bfs(adj, s)
        ans[s] = sum(dist)
    return ans
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

n BFS runs of O(n) each, O(n^2). Moving the root across one edge changes every distance by exactly 1, so two passes over the tree give all answers in O(n).

## OPTIMAL SOLUTION · Two passes: subtree sizes, then re-root

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, v) + 1. If w is outside it, dist(w, v) = dist(w, p) + 1. There are size[v] nodes of the first kind and n - size[v] of the second, so ans[v] = ans[p] - size[v] + (n - size[v]).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tree Distance Sum (Re-root DP)
#
# Onsite coding round 1: given an undirected tree on N
# nodes, return for every node the sum of distances to all
# other nodes. Equivalent to LC 834. Interviewer expects
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the re-root formula correct?
#
# INTERVIEW NOTES
# Running a BFS from every node costs O(n^2). Notice that
# moving the root from a parent p to its child v changes
# every distance by exactly one: the nodes inside v's
# subtree get closer by 1, all the others get farther by
# 1. So compute the answer for one root, then derive each
# child's answer from its parent's in O(1) using the
# subtree sizes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    adj = []
    for i in range(n):
        adj.append([])
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# ---- main question: main function ----
# Idea: two passes. Root the tree at 0 and list the nodes parents-first
# (a loop, no recursion). Going through that list backwards compute
# size[u] (nodes in u's subtree) and inside[u] (the sum of distances
```

```
# from u to the nodes of its own subtree). For the root that is already
# the answer. Going forwards, moving the root from p to its child u makes
```

```
# the size[u] nodes below 1 closer and the other n - size[u] nodes 1
# farther: answer[u] = answer[p] + n - 2 * size[u].
# Complexity: Time O(n), Space O(n)
def tree_distance_sums(n, edges):
    if n == 0:
        return []
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    order = [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    size = [1] * n
    inside = [0] * n
    for i in range(n - 1, -1, -1):                # children before parents
        u = order[i]
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]      # every node below u is one step farther from p
    answer = [0] * n
    answer[0] = inside[0]
    for u in order[1:]:
        answer[u] = answer[parent[u]] + n - 2 * size[u]
    return answer


# ---- main question: tests ----
def test_main_question():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    assert tree_distance_sums(6, edges) == [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    n = 100000                                     # a chain of 100000 nodes: no recursion trouble
    chain = []
    for i in range(1, n):
        chain.append((i, i - 1))
    assert tree_distance_sums(n, chain)[0] == n * (n - 1) // 2


# ============================================================
# FOLLOW-UP 1: Why is the re-root formula correct?
# ============================================================
```

```
# Answer: when the root moves across the edge p - u, every node in u's
# subtree (size[u] nodes) gets one step closer and every other node
# (n - size[u] nodes) gets one step farther, so the sum changes by
# (n - size[u]) - size[u] = n - 2 * size[u].


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## PREPARATION

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.
Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 23 OF 30

# Prefix sums, Tier 2: running total plus a dict of earlier totals

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Prefix sums, Tier 2: running total plus a dict of earlier totals | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 40. Given an integer array nums and an integer k, return true if nums has a subarray of at least two el... **Exponent no date**

CODE TEMPLATE | GROUP 23 OF 30 | 1 QUESTIONS, ONE PATTERN

# Prefix sums, Tier 2: running total plus a dict of earlier totals

## The one idea these questions share

Keep a running total. At each step ask the dict about an earlier total: total - k (subarray sums to k), total % k (sum divisible by k), or the smallest earlier total (best subarray ending here). Store the first index, a count, or the minimum.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def count_sum_k(nums, k):
    seen, total, count = {0: 1}, 0, 0       # CHANGE 1: what seen stores (count, first index, minimum)
    for x in nums:
        total += x
        count += seen.get(total - k, 0)      # CHANGE 2: the earlier total you look for
        seen[total] = seen.get(total, 0) + 1 # CHANGE 3: what you remember about this total
    return count
```

## 2. Helper functions that usually get written next to it

```
def max_subarray(nums):
    best, smallest, total = float("-inf"), 0, 0
    for x in nums:
        total += x
        best = max(best, total - smallest)   # best subarray that ends here
        smallest = min(smallest, total)      # update after, so the subarray is never empty
    return best
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **40** | Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. | key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 40 of 53 | topic 23 of 30: Prefix sums, Tier 2: running total plus a dict of earlier totals | 1 of 1**

Template fit: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k | no date

HARD | IN APP LC 523 | Exponent | 40 of 53

# Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 156/288

## ABOUT THIS QUESTION

**Topic:** Prefix sums, Tier 2: running total plus a dict of earlier totals

**Optimal answer from:** your app's interview approach (LeetCode #523)

**LeetCode match:** LC 523 Continuous Subarray Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +610 interviews

**Original:** https://www.aced.io/questions/2765/subarray-sum-multiple-of-k

## QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `[23, 2, 4, 6, 7], 6` |
| **Output** | `true` |
| **Why** | `[2, 4] adds up to 6, a multiple of 6.` |

| Example 2 | |
|---|---|
| **Input** | `[23, 2, 6, 4, 7], 13` |
| **Output** | `false` |
| **Why** | `No subarray of at least two elements adds up to a multiple of 13.` |

| Example 3 | |
|---|---|
| **Input** | `[23, 2, 6, 4, 7], 6` |
| **Output** | `true` |
| **Why** | `[23, 2, 6, 4, 7] adds up to 42, a multiple of 6.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    for i in range(len(nums)):
        total = nums[i]
        for j in range(i + 1, len(nums)):
            total += nums[j]
            if total % k == 0:
                return True
    return False
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

### WHY IT IS SLOW

O(n^2) pairs. Two prefix sums with the same remainder mod k bound a subarray whose sum is a multiple of k, so a dictionary of first-seen remainders is O(n).

## OPTIMAL SOLUTION · Prefix sums mod k with the first index of each remainder

| TIME | SPACE |
|---|---|
| O(n) | O(min(n, k)) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Count the subarrays whose sum is a multiple of k (any length, LeetCode 974).

Count how many times each remainder has occurred and add that count each time the remainder repeats (Python's % is already non-negative for negative sums). O(n).

## FOLLOW-UP 2 Why store only the first index of each remainder?

A later occurrence of the same remainder would make the subarray shorter, never longer, so the first index is the best partner for the length-at-least-two condition.

## FOLLOW-UP 3 The values can be negative or k huge.

Python's modulo gives a non-negative remainder; in languages where % can be negative normalise with ((s % k) + k) % k. A huge k just means the dictionary holds up to n entries. **Cost:** O(n) time with up to min(n, k) dictionary entries.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an integer array nums and an
# integer k, return true if nums has a subarray of at
# least two elements whose sum is a multiple of k.
#
# Determine if a given array contains a subarray of at
# least two elements whose sum is a multiple of a
# specified number k. An array is considered to have a
# "good subarray" if there exists at least one subarray
```

```
# (consisting of two or more elements) such that the sum
# of the elements in this subarray is a multiple of k. For
# example, the array [23, 2, 4, 7] with k = 6 has a "good
# subarray" ([2, 4]), as the sum 6 is a multiple of k = 6.
# The array [5, 0, 0, 0] with k = 3 also has a "good
# subarray" ([0, 0]), since the sum is 0, and 0 is a
# multiple of 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays whose sum is a multiple of k (any
# length, LeetCode 974).
# 2. Why store only the first index of each remainder?
# 3. The values can be negative or k huge.
#
# INTERVIEW NOTES
# A subarray sum is a difference of two prefix sums, and a
# multiple of k means the two prefix sums have the same
# remainder mod k. So remember the first index at which
# each remainder appeared; seeing it again at least two
# positions later gives a subarray of length at least two.
# Seed the table with remainder 0 at index -1 for
# subarrays that start at the beginning.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def find_good_subarray(nums, k):
    # the (start, end) indexes of a subarray of at least 2 elements whose
    # sum is a multiple of k, or None.
```

```
    # Idea: prefix sums MODULO k. Two equal remainders at indexes at least 2
    # apart mean the elements between them add up to a multiple of k. Keep
    # only the FIRST index of each remainder: a later one only gives a
    # shorter block. (k == 0 means "equal to zero": compare the sums themselves.)
    first = {0: -1}                         # remainder -> the first index where it occurred
    total = 0
    for i in range(len(nums)):
        total += nums[i]
        if k != 0:
            key = total % k
        else:
            key = total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return first[key] + 1, i
    return None


# ---- main question: main function ----
# Complexity: Time O(n), Space O(min(n, k))
def has_good_subarray(nums, k):
    return find_good_subarray(nums, k) is not None


# ---- main question: tests ----
def test_main_question():
```

```
    assert has_good_subarray([23, 2, 4, 7], 6)
    assert has_good_subarray([23, 2, 6, 4, 7], 6)
    assert not has_good_subarray([23, 2, 6, 4, 7], 13)
    assert find_good_subarray([23, 2, 4, 7], 6) == (1, 2)
    assert not has_good_subarray([6], 6)                # a single item is not enough
    assert has_good_subarray([0, 0], 5)
    assert not has_good_subarray([0, 1, 0], 0)
    assert has_good_subarray([0, 0], 0)
    assert has_good_subarray([5, -5], 7)
    assert has_good_subarray([1, 2], 10 ** 9) is False


# ============================================================
# FOLLOW-UP 1: Count the subarrays whose sum is a multiple of k (any length, LeetCode 974)
# ============================================================
# ---- follow-up 1: function ----
# Answer: count how many times each remainder has occurred, and add that
# count every time a remainder repeats (each earlier equal remainder is one
```

```
# more subarray). In Python % is never negative; in other languages use
# ((s % k) + k) % k. O(n).
# Complexity: Time O(n), Space O(min(n, k))
def count_multiples(nums, k):
    seen = {0: 1}
    total = 0
    answer = 0
    for x in nums:
        total += x
        key = total % k
        answer += seen.get(key, 0)
        seen[key] = seen.get(key, 0) + 1
    return answer


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_multiples([4, 5, 0, -2, -3, 1], 5) == 7


# ============================================================
# FOLLOW-UP 2: Why store only the first index of each remainder?
# ============================================================
# Answer: the later we meet the same remainder again, the shorter the
# block between them. The FIRST index gives the longest block, and the
# only thing we need is "at least 2 apart".


# ============================================================
# FOLLOW-UP 3: The values can be negative or k huge
# ============================================================
# Answer: Python's % gives a non-negative remainder even for negative sums;
# in other languages normalise it. A huge k costs nothing: only the
# remainders that occur are stored in the dictionary.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

## OFFICIAL PAGE TEXT

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples

```
nums = [23, 2, 4, 7], k = 6
output: true

nums = [5, 0, 0, 0], k = 3
output: true

nums = null, k = 1
output: false
```

In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?Our solution processes the input array by iterating through it while maintaining a hash map (remainder_index_map) of remainders and their corresponding indices. For each element in the array, we accumulate the sum (sum_so_far) and compute the remainder when divided by k. We check if this remainder has been encountered before in the hash map. If the remainder is not in the hash map, we store the current index associated with this remainder. If the remainder has been seen before, we calculate the distance between the current index and the index of the previously seen remainder. If this distance is greater than or equal to the minimum subarray length (MIN_GOOD_SUBARRAY_LIMIT), we return True, indicating that a subarray with the required properties exists. If the loop completes without finding such a subarray, we return False. The use of a hash map ensures that each element is processed efficiently, as it allows constant-time lookups and insertions. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (hash map lookups and insertions) for each element. Space Complexity: The solution uses O(n) space for the hash map that stores the remainders and their indices. In the worst case, all remainders are unique, resulting in the hash map containing n entries.Let’s say you are given an integer array nums and an integer k. Return true if nums has a subarray where its length is at least two, and the sum of the elements in the subarray is a multiple of k. Watch Neeraj, a Senior Engineering Manager, give his answer to this software engineering interview question.No answers contributed by the community yet.Related questionsGiven an array of n integers and a number k, find the maximum number of elements whose sum is less than k, and return their sum.Given an array of integers and an integer k, find the number of continuous subarrays whose sum equals k.Given an array of integers and a number N, find the length of the longest contiguous subarray such that the difference between any two elements in the subarray is less than N.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta,

Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 24 OF 30

# Group by a canonical key

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
HARD

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Group by a canonical key | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 41. Given two strings and two allowed operations, swapping positions of letters and converting all occ... **Exponent no date**

CODE TEMPLATE | GROUP 24 OF 30 | 1 QUESTIONS, ONE PATTERN

# Group by a canonical key

## The one idea these questions share

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import defaultdict


def group_by_key(words, key):
    groups = defaultdict(list)
    for w in words:
        groups[key(w)].append(w)              # CHANGE: the key function decides everything
    return list(groups.values())
```

## 2. Helper functions that usually get written next to it

```
from collections import Counter


def letter_counts(word):                    # a hashable signature that ignores order
    return tuple(sorted(Counter(word).items()))


def shift_signature(word):
    return tuple((ord(word[i]) - ord(word[i - 1])) % 26 for i in range(1, len(word)))
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **41** | Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? | same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 41 of 53 | topic 24 of 30: Group by a canonical key | 1 of 1**

Template fit: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values | no date

HARD | Exponent | 41 of 53

# Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 183/288

## ABOUT THIS QUESTION

**Topic:** Group by a canonical key

**Optimal answer from:** written and tested here for this exact question

This question has several parts, so each part has its own answer below.

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #1153 String Transforms Into Another String

**LeetCode match:** No direct match; related LC 1153 String Transforms Into Another String (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/6330/determine-string-transformability-allowed-operations

## QUESTION

Two strings are 'close' if you can turn one into the other with these operations, used any number of times: (1) swap the positions of any two letters; (2) take two letters that occur in the string and swap all of their occurrences (every a becomes b and every b becomes a). Return whether the first string can be transformed into the second.

**Function signature**

def close_strings(s: str, t: str) -> bool

EXAMPLES

| Example 1 | |
|---|---|
| Input | `'abc', 'bca'` |
| Output | `true` |
| Why | `Swapping positions turns 'abc' into 'bca'.` |

| Example 2 | |
|---|---|
| Input | `'a', 'aa'` |
| Output | `false` |
| Why | `The lengths differ.` |

| Example 3 | |
|---|---|
| Input | `'cabbba', 'abbccc'` |
| Output | `true` |
| Why | `Use operation 1 to reorder, then operation 2 to swap the letters a and b, and c and ... until the counts match.` |

## QUESTION

| Example 4 | |
|---|---|
| Input | `'uau', 'ssx'` |
| Output | `false` |
| Why | `'u' appears in the first string but not the second, so they cannot be close.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(s, t):
    seen, queue = {s}, deque([s])
    while queue:
        cur = queue.popleft()
        if cur == t:
            return True
        for nxt in swap_positions(cur):
            add_if_new(nxt, seen, queue)
        for nxt in convert_letters(cur):
            add_if_new(nxt, seen, queue)
    return False
```

| TIME | SPACE |
|---|---|
| exponential | exponential |

WHY IT IS SLOW

The number of reachable strings grows exponentially with the length. Because swaps make order irrelevant, comparing the sets of letters and their sorted counts decides it in O(n).

## OPTIMAL SOLUTION · Reading 1 (LeetCode 1657): same letter set and same multiset of frequencies

| TIME | SPACE |
|---|---|
| O(n) | O(1)<br>for a fixed alphabet |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## OPTIMAL SOLUTION · Reading 2 (LeetCode 1153): one-directional conversion, no swaps

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What if the two operations are combined (one-directional conversion plus swaps)?

Swaps make positions irrelevant, so only letter counts matter. Conversions can only merge letter classes, never split one. So t is reachable only if the counts of t can be formed by grouping the counts of s into the t letters (each s letter's count goes wholly to one t letter) and, unless the relabeling is an identity or chain, there is a spare letter. With at most 26 classes the grouping is a small bin-packing search.

**FOLLOW-UP 2 Why does a full 26-letter target make the one-directional version impossible?**

Every conversion rewrites a letter into another letter, and if a mapping contains a cycle (a to b and b to a) you need a temporary letter not currently in use to break it. If t uses all 26 letters then s must use all 26 as well (the mapping is a function onto 26 distinct letters), so the mapping is a permutation and any non-identity permutation has a cycle. The only possible case is therefore s equal to t.

**FOLLOW-UP 3 Only position swaps are allowed.**

Then s can become t exactly when they are anagrams: compare the letter counts. O(n).

**FOLLOW-UP 4 Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 1153).**

Each source letter must map to one target letter, and if the strings differ and the target already uses all 26 letters there is no spare letter to break mapping cycles, so it is impossible. Verified against brute force on a small alphabet.

**FOLLOW-UP 5 Return the sequence of operations, or the minimum number.**

Minimum operations for the swap-only case is n minus the number of cycles in the permutation that sorts s into t; for letter-identity swaps it is the number of cycles in the letter mapping minus one per cycle of length greater than one.

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given two strings and two allowed
# operations, swapping positions of letters and converting
# all occurrences of one character to another, can the
# first string be transformed into the second?
#
# Two strings are 'close' if you can turn one into the
# other with these operations, used any number of times:
# (1) swap the positions of any two letters; (2) take two
# letters that occur in the string and swap all of their
# occurrences (every a becomes b and every b becomes a).
# Return whether the first string can be transformed into
# the second.
#
# Signature:
#   def close_strings(s: str, t: str) -> bool
#
# Examples:
# 'abc', 'bca'  ->  true
# 'a', 'aa'  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the two operations are combined (one-
# directional conversion plus swaps)?
# 2. Why does a full 26-letter target make the one-
# directional version impossible?
# 3. Only position swaps are allowed.
# 4. Conversion is one-directional (every a becomes b) and
# there are no position swaps (LeetCode 1153).
# 5. Return the sequence of operations, or the minimum
# number.
#
# INTERVIEW NOTES
# Decide which operations you really have. If you can swap
```

```
# any two letters (positions) and swap the identities of
# two letters, the answer depends only on which letters
# appear and how often (the multiset of counts). If
# conversion is one-way (every a becomes b), letters can
# merge but never split, and cycles in the mapping need a
# spare letter. Name the reading before you code.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ============================================================

ALPHABET = 26


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def count_letters(text):
    counts = {}
    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def can_group(have, want):
    # can the counts in `have` be grouped (each used once) so that the group
    # sums are exactly the counts in `want`? Letters can merge, never split.
    # Backtracking: put each count (biggest first) into a bin of `want`.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    room = list(want)                                   # how much each bin can still take

    def place(i):
        if i == len(have):
            for r in room:
                if r != 0:
                    return False
            return True
        tried = set()
        for j in range(len(room)):
            if room[j] >= have[i] and room[j] not in tried:    # equal bins behave the same: try each size
once
                tried.add(room[j])
                room[j] -= have[i]
                if place(i + 1):
                    return True
                room[j] += have[i]
        return False

    return place(0)


# ---- main question: main function ----
```

```
# Two operations: swap the positions of two letters, and convert ALL
# occurrences of one letter into another letter. Can s become t?
# Idea: swaps make the positions irrelevant, so only the letter COUNTS
# matter. Converting a letter can only rename it or merge it into another one. So
# if the counts already match, yes; otherwise the counts of s must be
```

```
# groupable into the counts of t (can_group). If both strings already use all
# 26 letters there is no spare letter left to rename safely.
# Complexity: Time O(n + A log A) for strings of length n and alphabet size A, Space O(A)
def close_strings(s, t, alphabet=ALPHABET):
    if len(s) != len(t):
        return False
    count_s = count_letters(s)
    count_t = count_letters(t)
    if count_s == count_t:
        return True
    if not can_group(list(count_s.values()), list(count_t.values())):
        return False
    if len(count_s) == len(count_t) == alphabet:
        return False
    return True


# ---- main question: tests ----
def test_main_question():
    assert close_strings("abc", "bca")
    assert not close_strings("a", "aa")
    assert close_strings("cabbba", "abbccc")


# =======================================================
# FOLLOW-UP 1: What if the two operations are combined (one-directional conversion plus swaps)?
# =======================================================
# Answer: that is close_strings above: swaps make the positions irrelevant,
# and the conversion works on the letter counts.


# =======================================================
# FOLLOW-UP 2: Why does a full 26-letter target make the one-directional version impossible?
# =======================================================
# Answer: converting a letter to another merges two letters into one, and
# to break a cycle of conversions (a -> b, b -> a) you need a SPARE letter as a
# temporary home. If the target uses all 26 letters there is none.


# =======================================================
```

```
# FOLLOW-UP 3: Only position swaps are allowed
# =======================================================
# ---- follow-up 3: function ----
# Answer: then s can become t exactly when they are anagrams: compare the
# letter counts. O(n).
# Complexity: Time O(n), Space O(A)
def swaps_only(s, t):
    return count_letters(s) == count_letters(t)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert swaps_only("listen", "silent")
    assert not swaps_only("ab", "aa")


# =======================================================
# FOLLOW-UP 4: Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode
1153)
# =======================================================
# ---- follow-up 4: function ----
# Answer: each source letter must map to ONE target letter; and if the
# strings differ and the target already uses all 26 letters there is no spare
# letter to break mapping cycles, so it is impossible.
```

```python
# Complexity: Time O(n), Space O(A)
def convert_only(s, t, alphabet=ALPHABET):
    if len(s) != len(t):
        return False
    if s == t:
        return True
    mapping = {}
    for i in range(len(s)):
        if s[i] in mapping and mapping[s[i]] != t[i]:
            return False                                # one letter would need two targets
        mapping[s[i]] = t[i]
    return len(set(t)) < alphabet


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert convert_only("aabcc", "ccdee")
    assert not convert_only("leetcode", "codeleet")


# ========================================================
# FOLLOW-UP 5: Return the sequence of operations, or the minimum number
```

```python
# ========================================================
# ---- follow-up 5: function ----
# Answer: for swaps only, with all letters different, the fewest swaps is
# n minus the number of CYCLES of the permutation that puts s into the order
# of t. swap_sequence writes the swaps down: for each position put the right
# letter there, one swap per misplaced letter.
# Complexity: Time O(n), Space O(n)
def min_swaps(s, t):
    where = {}
    for i in range(len(t)):
        where[t[i]] = i
    seen = set()
    cycles = 0
    for start in range(len(s)):
        if start not in seen:
            cycles += 1
            i = start
            while i not in seen:
                seen.add(i)
                i = where[s[i]]
    return len(s) - cycles


# Complexity: Time O(n), Space O(n)
def swap_sequence(s, t):
    current = list(s)
    where = {}
    for i in range(len(current)):
        where[current[i]] = i
    steps = []
    for i in range(len(t)):
        if current[i] != t[i]:
            j = where[t[i]]
            where[current[i]] = j
            where[t[i]] = i
            current[i], current[j] = current[j], current[i]
            steps.append((i, j))
    return steps


# ---- follow-up 5: tests ----
def test_follow_up_5():
```

```
    assert min_swaps("abc", "bca") == 2
    assert len(swap_sequence("abc", "bca")) == 2
    assert swap_sequence("abc", "abc") == []
```

```


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 25 OF 30

# K-way merge: a heap of the next item from each source

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**K-way merge: a heap of the next item from each source | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**

H 42. Merge k sorted linked lists. **Exponent no date**

H 43. Return top-k subset sums in descending order **PracHub 2026-01-07**

CODE TEMPLATE | GROUP 25 OF 30 | 2 QUESTIONS, ONE PATTERN

# K-way merge: a heap of the next item from each source

## The one idea these questions share

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def k_way(lists, k):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]   # CHANGE 1: what you seed with
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):                                     # CHANGE 2: the successor
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

## 2. Helper functions that usually get written next to it

```
def merge_two(a, b):                      # the simple two-list case of the same idea
    i = j = 0
    out = []
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **42** | Merge k sorted linked lists. | seed with each list head (value, list number, node); the successor of a node is node.next |
| **43** | Return top-k subset sums in descending order | heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 42 of 53 | topic 25 of 30: K-way merge: a heap of the next item from each source | 1 of 2**

Template fit: seed with each list head (value, list number, node); the successor of a node is node.next | no date

HARD | IN APP LC 23 | Exponent | 42 of 53

# Merge k sorted linked lists.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 49.4 | 30 DAYS 0 | PRIORITY 81/288

## ABOUT THIS QUESTION

**Topic:** K-way merge: a heap of the next item from each source

**Optimal answer from:** your app's interview approach (LeetCode #23)

**LeetCode match:** LC 23 Merge k Sorted Lists

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +816 interviews

**Original:** https://www.aced.io/questions/1167/merge-k-sorted-linked-lists

## QUESTION

You are given k linked lists, each sorted in ascending order. Merge them into one sorted linked list and return its head. (In the examples the linked lists are written as arrays.)

**Function signature**

def merge_k_heap(lists: list) -> ListNode

EXAMPLES

| Example 1 | |
|---|---|
| Input | `lists = [[1, 4, 5], [1, 3, 4], [2, 6]]` |
| Output | `_to_list(merge_k_heap([mk([1, 4, 5]), mk([1, 3, 4]), mk([2, 6])])) -> [1, 1, 2, 3, 4, 4, 5, 6]` |
| Why | `The three sorted lists merge into one sorted list.` |

| Example 2 | |
|---|---|
| Input | `lists = []` |
| Output | `_to_list(merge_k_heap([])) -> []` |
| Why | `No lists: an empty result.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(lists):
    values = []
    for head in lists:
        while head:
            values.append(head.val)
            head = head.next
    values.sort()
    dummy = tail = ListNode(0)
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
```

## BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(N log N) | O(N) |

**WHY IT IS SLOW**

O(N log N) time and O(N) extra memory, ignoring that each list is already sorted. A min-heap of the k current heads is O(N log k) with O(k) memory.

## OPTIMAL SOLUTION · Min-heap of the current heads

| TIME | SPACE |
|---|---|
| O(N log k) | O(k) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

## FOLLOW-UP 2 The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). **Cost:** O(N log k) time and O(k) memory; one sequential read per list.

## FOLLOW-UP 3 Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import heapq

END = object()                                  # marks "this stream has no more items"


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

```
def make(values):
    # a linked list from a Python list
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


def merge_two(a, b):
    # merge two sorted linked lists into one
    dummy = ListNode()
    tail = dummy
    while a and b:
```

```
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    if a:
        tail.next = a
    else:
        tail.next = b
    return dummy.next


# ---- main question: main function ----
# Idea: a min-heap holds the CURRENT head of every list as (value, list
# number, node). Take the smallest out, add it to the result and put
# its successor in the heap. The list number breaks ties so two nodes are
# never compared. O(N log k) for N nodes in k lists.
# Complexity: Time O(N log k) for N nodes in total, Space O(k)
def merge_k_heap(lists):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            heap.append((lists[i].val, i, lists[i]))
    heapq.heapify(heap)
    dummy = ListNode()
    tail = dummy
    while heap:
        value, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next


# ---- main question: tests ----
DATA = [[1, 4, 5], [1, 3, 4], [2, 6]]
WANTED = [1, 1, 2, 3, 4, 4, 5, 6]
```

```
def build_all():
    lists = []
    for values in DATA:
        lists.append(make(values))
    return lists


def test_main_question():
    assert to_list(merge_k_heap(build_all())) == WANTED
    assert merge_k_heap([]) is None
    assert to_list(merge_k_heap([None, make([1])])) == [1]
    assert to_list(merge_two(make([1, 3]), make([2]))) == [1, 2, 3]


# =====================================================
# FOLLOW-UP 1: Merge the lists in pairs instead of using a heap
# =====================================================
# ---- follow-up 1: function ----
# Answer: repeatedly merge neighbouring pairs of lists with the two-list
# merge. There are log k rounds and each touches all N nodes: O(N log k)
# time and no heap.
# Complexity: Time O(N log k), Space O(1) extra (the lists are relinked)
def merge_k_pairs(lists):
    lists = [node for node in lists if node]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                merged.append(merge_two(lists[i], lists[i + 1]))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_list(merge_k_pairs(build_all())) == WANTED
    assert merge_k_pairs([None]) is None


# =====================================================
# FOLLOW-UP 2: The lists are too large for memory (files)
# =====================================================
# ---- follow-up 2: function ----
# Answer: the heap approach already works on streams: it holds just one
# item per list. Read each input through an iterator and write the output
# as you go. This is the merge step of an external sort.
# Complexity: Time O(N log k), Space O(k)
def merge_streams(iterators):
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    while heap:
        value, i = heapq.heappop(heap)
        yield value
```

```
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    iterators = []
    for values in DATA:
        iterators.append(iter(values))
    assert list(merge_streams(iterators)) == WANTED


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## OFFICIAL PAGE TEXT

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.

**STUDY PATH: question 43 of 53 | topic 25 of 30: K-way merge: a heap of the next item from each source | 2 of 2**

Template fit: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next | asked 286 days ago

HARD | PracHub | Technical Screen | 43 of 53

# Return top-k subset sums in descending order

POSTED 2026-01-07 | 286 DAYS AGO | OLDER

STUDY: TALK ALL TIME 5 30 DAYS 0 PRIORITY 194/288

## ABOUT THIS QUESTION

**Topic:** K-way merge: a heap of the next item from each source

**Optimal answer from:** the interview site's own solution

**LeetCode match:** LC 2386 Find the K-Sum of an Array

Posted on PracHub: 2026-01-07 (updated 2026-05-21)

**Original:** https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

## QUESTION

Given a list of integers (possibly negative), return the k largest subset sums in descending order. The empty subset counts and has sum 0; different subsets with the same sum are listed separately.

**Function signature**

def top_k_sums(nums: list, k: int) -> list

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[1, 2, 3], 4` |
| Output | `[6, 5, 4, 3]` |
| Why | `The eight subset sums are 6, 5, 4, 3, 3, 2, 1, 0; the top four are 6, 5, 4, 3.` |

| Example 2 | |
|---|---|
| Input | `[3, -2, 1], 3` |
| Output | `[4, 3, 2]` |
| Why | `With a negative number: 3 + 1 = 4, then 3, then 3 + (-2) + 1 = 2.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    sums = []
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            sums.append(sum(subset))
    sums.sort(reverse=True)
    return sums[:k]
```

| TIME | SPACE |
|---|---|
| O(2^n * n) | O(2^n) |

WHY IT IS SLOW

## BRUTE FORCE

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

## OPTIMAL SOLUTION · Heap of the k smallest sums of absolute values

| TIME | SPACE |
|---|---|
| **O(n log n + k log k)** | **O(n + k)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

## FOLLOW-UP 2 k can be as large as 2^n.

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

## FOLLOW-UP 3 Limit subsets to at most m elements.

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return top-k subset sums in descending
# order
#
# Given a list of integers (possibly negative), return the
# k largest subset sums in descending order. The empty
# subset counts and has sum 0; different subsets with the
# same sum are listed separately.
#
# Signature:
#   def top_k_sums(nums: list, k: int) -> list
#
# Examples:
# [1, 2, 3], 4  ->  [6, 5, 4, 3]
# [3, -2, 1], 3  ->  [4, 3, 2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsets themselves, not only the sums.
# 2. k can be as large as 2^n.
# 3. Limit subsets to at most m elements.
#
# INTERVIEW NOTES
# k is small compared with 2^n, so never enumerate all
# subsets. Turn the problem into 'k smallest sums of non-
# negative numbers': subtract everything from the sum of
# the positive values. Then grow answers in increasing
# order with a heap where each state has exactly two
```

```
# successors (extend with the next element, or swap the
# last element for the next one).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def sorted_costs(nums):
    # |x| of every number, smallest first
    costs = []
    for x in nums:
        if x < 0:
            costs.append(-x)
        else:
            costs.append(x)
    costs.sort()
    return costs


# ---- main question: main function ----
# Return the k biggest sums of subsets of nums (the empty subset counts).
# Idea: the best subset takes every positive number. Any other subset
# is the best one with some numbers "flipped" (a positive one left out or a
# negative one added), and flipping x costs |x|. So the k best subsets are
# the k CHEAPEST sets of flips. Generate the flip sets in order of cost with a
# heap over the sorted costs: from a set whose last flipped item is i, the
# two successors are (a) add item i + 1, (b) replace item i by item i + 1.
# Neither is ever cheaper than the set itself, so the heap yields them in order.
# Complexity: Time O(n log n + k log k), Space O(n + k)
def top_k_sums(nums, k):
    base = 0
    for x in nums:
        if x > 0:
            base += x
    costs = sorted_costs(nums)
    flip_costs = [0]                                    # the best subset costs 0 flips
    heap = []
    if costs:
        heap.append((costs[0], 0))                      # (total cost, index of the last flipped item)
    while heap and len(flip_costs) < k:
        cost, i = heapq.heappop(heap)
        flip_costs.append(cost)
        if i + 1 < len(costs):
            heapq.heappush(heap, (cost + costs[i + 1], i + 1))                  # add the next item
            heapq.heappush(heap, (cost - costs[i] + costs[i + 1], i + 1))       # move the last item up by
 one
    sums = []
    for cost in flip_costs[:k]:
        sums.append(base - cost)
    return sums


# ---- main question: tests ----
def test_main_question():
```

```
    assert top_k_sums([3, -1, 2, -4], 4) == [5, 4, 3, 2]
```

```
    assert top_k_sums([], 3) == [0]
    assert top_k_sums([5, -2, 7, 1], 3) == [13, 12, 11]


# ========================================================
# FOLLOW-UP 1: Return the subsets themselves, not only the sums
# ========================================================
# ---- follow-up 1: function ----
# Answer: do not copy lists into the heap. Store each state as a link
# (last flipped item, the state it came from) and rebuild a subset by
# walking the links only for the k answers. Every push stays O(1).
# Complexity: Time O(n log n + k * n) (each subset is rebuilt in O(n)), Space O(k * n)
def top_k_subsets(nums, k):
    order = []                                          # item numbers from the cheapest to the most expensi
ve
    for i in range(len(nums)):
        order.append(i)
    order.sort(key=lambda i: abs(nums[i]))              # (the sort key is the size of the number)
    costs = []
    for i in order:
        costs.append(abs(nums[i]))
    best = set()
    base = 0
    for i in range(len(nums)):
        if nums[i] > 0:
            best.add(i)
            base += nums[i]

    def subset_of(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        chosen = []
        for i in range(len(nums)):
            if (i in best) != (i in flipped):           # the best subset with the flipped items changed
                chosen.append(nums[i])
        return sorted(chosen)

    results = [(base, subset_of(None))]
    heap = []
    if costs:
        heap.append((costs[0], 0, (0, None)))           # (cost, last item, the chain of flipped items)
    while heap and len(results) < k:
        cost, i, chain = heapq.heappop(heap)
        results.append((base - cost, subset_of(chain)))
```

```
        if i + 1 < len(costs):
            heapq.heappush(heap, (cost + costs[i + 1], i + 1, (i + 1, chain)))
            heapq.heappush(heap, (cost - costs[i] + costs[i + 1], i + 1, (i + 1, chain[1])))
    return results[:k]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    results = top_k_subsets([3, -1, 2, -4], 4)
    totals = []
    for total, subset in results:
        assert sum(subset) == total                     # the subsets add up to the sums
        totals.append(total)
    assert totals == [5, 4, 3, 2]
    assert results[0][1] == [2, 3]
```

```
# ============================================================
# FOLLOW-UP 2: k can be as large as 2^n
# ============================================================
# Answer: the heap holds at most k entries and costs O(k log k), which is
# fine for k up to millions but not for 2^n. For huge k, binary-search the
# sum: count how many subsets have a sum at least s (meet in the middle),
# and find the k-th value directly.


# ============================================================
# FOLLOW-UP 3: Limit subsets to at most m elements
# ============================================================
# Answer: keep the number of elements in each heap state and skip a
# successor that would use more than m elements.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(w: list[int], k: int) -> list[int]:
```

### Sample call

```
solution([3, 2, 1], 4)
```

### Test cases

| Input | Expected |
|---|---|
| ([3, 2, 1], 4) | [6, 5, 4, 3] |
| ([5], 2) | [5, 0] |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 26 OF 30

# Random pick without repeats: swap the chosen item with the last one and pop

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Random pick without repeats: swap the chosen item with the last one and pop | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 44. Shuffle array using random integer API PracHub 2025-12-08

CODE TEMPLATE | GROUP 26 OF 30 | 1 QUESTIONS, ONE PATTERN

# Random pick without repeats: swap the chosen item with the last one and pop

## The one idea these questions share

To pick a random item and never pick it again in O(1), keep the items in an array. Choose a random index, swap that item with the last one, pop the last, and fix its position in the index dict. The same trick gives O(1) delete and O(1) random access.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import random


class PickPool:
    def __init__(self, items, seed=None):
        self.a = list(items)
        self.pos = {x: i for i, x in enumerate(self.a)}   # CHANGE 1: lookup by value (for delete)
        self.rng = random.Random(seed)

    def take_random(self):
        i = self.rng.randrange(len(self.a))
        x, last = self.a[i], self.a[-1]
        self.a[i] = last                                  # the last item fills the hole
        self.pos[last] = i
        self.a.pop()
        del self.pos[x]
        return x                                          # CHANGE 2: what you hand back
```

## 2. Helper functions that usually get written next to it

```
def fisher_yates(nums, rand_int):             # rand_int(lo, hi) is inclusive on both ends
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **44** | Shuffle array using random integer API | Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j] |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 44 of 53 | topic 26 of 30: Random pick without repeats: swap the chosen item with the last one and pop | 1 of 1**

Template fit: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j] | asked 316 days ago

HARD | PracHub | Technical Screen | 44 of 53

# Shuffle array using random integer API

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 184/288

## ABOUT THIS QUESTION

**Topic:** Random pick without repeats: swap the chosen item with the last one and pop

**Optimal answer from:** the interview site's own solution

**LeetCode match:** LC 384 Shuffle an Array

Posted on PracHub: 2025-12-08 (updated 2026-06-25)

**Original:** https://prachub.com/coding-questions/shuffle-array-using-random-integer-api

## QUESTION

You have an array nums and no built-in shuffle, only rand_int(l, r), which returns a uniformly random integer in [l, r] (both ends included). Shuffle nums in place so that every one of the n! orders is equally likely.

**Function signature**

def shuffle(nums: list, rand_int) -> list

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `rand_int = lambda l, r: l # a fake 'random' source that always returns the lowest allowed index`<br>`nums = [1, 2, 3, 4]`<br>`shuffle(nums, rand_int)` |
| **Output** | `shuffle(nums, rand_int) -> [2, 3, 4, 1]` |
| **Why** | `Fisher-Yates walks i from the end and swaps nums[i] with nums[rand_int(0, i)]. With the fake source the swaps are (3,0), (2,0), (1,0), giving a predictable result; with a real source each of the 24 orders has probability 1/24.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def shuffle(nums):
    keyed = []
    for x in nums:
        keyed.append((rand_int(0, 10 ** 9), x))
    keyed.sort()
    return [x for _, x in keyed]
```

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

WHY IT IS SLOW

O(n log n) time and O(n) extra space, and equal random keys make the result very slightly non-uniform. Fisher-Yates (swap position i with a random position in 0..i) is O(n), in place and exactly uniform.

## OPTIMAL SOLUTION · Fisher-Yates

| TIME | SPACE |
|---|---|
| **O(n)** | **O(1)** extra |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(k) after copying, still uniform.

## FOLLOW-UP 2 Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

## FOLLOW-UP 3 Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shuffle array using random integer API
#
# You have an array nums and no built-in shuffle, only
# rand_int(l, r), which returns a uniformly random integer
# in [l, r] (both ends included). Shuffle nums in place so
# that every one of the n! orders is equally likely.
#
# Signature:
#   def shuffle(nums: list, rand_int) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only k random elements.
# 2. Only a coin-flip API exists, no rand_int.
# 3. Why is the biased version biased?
#
# INTERVIEW NOTES
# 'Shuffle' means every one of the n! orders must be
# equally likely. Count outcomes: a procedure with n^n
# equally likely random choices cannot give n! equal
# buckets, so it is biased. Fisher-Yates makes n choices
# with sizes n, n-1, ..., 1, which multiply to exactly n!.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (Fisher-Yates): go from the LAST position down to position 1. For
# each position i pick j uniformly from 0..i (both included) and swap
# nums[i] with nums[j]. Every one of the n! orders is equally likely,
# because each step picks uniformly among i + 1 choices.
# rand_int(low, high) gives a random whole number from low to high.
# Complexity: Time O(n), Space O(1)
def shuffle(nums, rand_int):
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# ---- main question: tests ----
def always_low(low, high):
    return low


def always_high(low, high):
    return high


def real_rand_int(low, high):
    return random.randint(low, high)


def test_main_question():
    assert shuffle([1, 2, 3, 4], always_low) == [2, 3, 4, 1]       # j is always 0
    assert shuffle([1, 2, 3, 4], always_high) == [1, 2, 3, 4]      # j is always i: nothing moves
    assert shuffle([], real_rand_int) == []
    random.seed(37)
    seen = set()
    for round_number in range(600):
        result = shuffle([1, 2, 3], real_rand_int)
        assert sorted(result) == [1, 2, 3]                          # always a rearrangement
        seen.add(tuple(result))
    assert len(seen) == 6                                           # all 3! orders occur


# ========================================================
# FOLLOW-UP 1: Return only k random elements
# ========================================================
# ---- follow-up 1: function ----
# Answer: run only k steps of Fisher-Yates from the FRONT: swap position i
# with a random position from i to the end, and return the first k.
# O(k) after copying, and still uniform.
# Complexity: Time O(n + k) (it copies the list), Space O(n)
def sample_k(nums, k, rand_int):
    items = list(nums)
    for i in range(min(k, len(items))):
        j = rand_int(i, len(items) - 1)
        items[i], items[j] = items[j], items[i]
    return items[:k]
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert sample_k([1, 2, 3, 4], 2, always_low) == [1, 2]
    assert sample_k([1, 2, 3, 4], 2, always_high) == [4, 1]
```

```
    assert sample_k([1, 2, 3], 5, always_low) == [1, 2, 3]
    random.seed(38)
    for round_number in range(200):
        picked = sample_k([1, 2, 3, 4, 5], 3, real_rand_int)
        assert len(set(picked)) == 3                                # no element twice


# =====================================================
# FOLLOW-UP 2: Only a coin-flip API exists, no rand_int
# =====================================================
# ---- follow-up 2: function ----
# Answer: build a uniform number in [0, m) by REJECTION SAMPLING: draw
# enough random bits to cover m values, and try again if the number is
# m or more. Every value stays equally likely, unlike taking a
# remainder (which would favour small numbers).
# Complexity: Time O(log m) bits per attempt and fewer than 2 attempts on average, Space O(1)
def rand_below(m, flip):
    bits = 1
    while (1 << bits) < m:
        bits += 1                                   # the fewest bits that can count up to m
    while True:
        x = 0
        for i in range(bits):
            x = x * 2 + flip()
        if x < m:
            return x


# ---- follow-up 2: tests ----
def coin():
    return random.randint(0, 1)


def test_follow_up_2():
    random.seed(39)
    counts = [0] * 5
    for round_number in range(5000):
        counts[rand_below(5, coin)] += 1
    for count in counts:
        assert 800 < count < 1200                   # about 1000 each: no value is favoured
```

```
    assert rand_below(1, coin) == 0


# =====================================================
# FOLLOW-UP 3: Why is the biased version biased?
# =====================================================
# ---- follow-up 3: function ----
# Answer: the common WRONG version swaps every position with a random
# position from the WHOLE array. That makes n choices of n options,
# n^n equally likely paths, and n! does not divide n^n for n >= 3, so
# some orders must appear more often than others.
# Complexity: Time O(n), Space O(1)
def biased_shuffle(nums, rand_int):
    for i in range(len(nums)):
        j = rand_int(0, len(nums) - 1)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# ---- follow-up 3: tests ----
def test_follow_up_3():
    counts = {}
    for a in range(3):                              # try all 27 possible sequences of random choices
        for b in range(3):
```

```
            for c in range(3):
                choices = [a, b, c]
                position = [0]

                def scripted(low, high):
                    value = choices[position[0]]
                    position[0] += 1
                    return value

                result = tuple(biased_shuffle([1, 2, 3], scripted))
                counts[result] = counts.get(result, 0) + 1
    assert sum(counts.values()) == 27
    assert len(set(counts.values())) > 1            # the 6 orders do NOT get equal shares of 27


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(nums: list[int], random_indices: list[int]) -> list[int]:
```

### Sample call

```
solution([1, 2, 3, 4], [1, 1, 0])
```

### Test cases

| Input | Expected |
|---|---|
| ([1, 2, 3, 4], [1, 1, 0]) | [3, 1, 4, 2] |
| ([], []) | [] |

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 27 OF 30

# Build and serialize: write a structure out, read it back with the same walk

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Build and serialize: write a structure out, read it back with the same walk | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**


H 45. Implement a decode and encode function of a general N-ary tree. **Exponent no date**

H 46. Serialize and Deserialize a Graph **ColorStack 2025-09-24**

CODE TEMPLATE | GROUP 27 OF 30 | 2 QUESTIONS, ONE PATTERN

# Build and serialize: write a structure out, read it back with the same walk

## The one idea these questions share

Encoding is one walk that writes tokens; decoding is the same walk reading tokens back (an iterator or an index). Put a length or a child count before the data so the reader always knows how much to take.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def encode(node, out):                      # CHANGE 1: what you write per node
    if node is None:
        out.append("#")
        return
    out.append(str(node.val))
    encode(node.left, out)
    encode(node.right, out)


def decode(tokens):
    it = iter(tokens)

    def build():
        tok = next(it)
        if tok == "#":
            return None
        node = Node(int(tok))               # CHANGE 2: build the node from what you read
        node.left = build()
        node.right = build()
        return node

    return build()
```

## 2. Helper functions that usually get written next to it

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def read_tokens(text):                      # the reader side of a length-prefixed format
    i, out = 0, []
    while i < len(text):
        j = text.index("#", i)
        n = int(text[i:j])
        out.append(text[j + 1:j + 1 + n])
        i = j + 1 + n
    return out
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **45** | Implement a decode and encode function of a general N-ary tree. | write value and child count for each node (preorder); read the count to know how many children to build |
| **46** | Serialize and Deserialize a Graph | BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 45 of 53 | topic 27 of 30: Build and serialize: write a structure out, read it back with the same walk | 1 of 2**

Template fit: write value and child count for each node (preorder); read the count to know how many children to build | no date

HARD | Exponent | 45 of 53

# Implement a decode and encode function of a general N-ary tree.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 173/288

## ABOUT THIS QUESTION

**Topic:** Build and serialize: write a structure out, read it back with the same walk

**Optimal answer from:** written and tested here for this exact question

**LeetCode match:** LC 428 Serialize and Deserialize N-ary Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/4516/decode-encode-function-general-tree

## QUESTION

A general tree has a value in every node and any number of children. Write encode(root), which turns the tree into a string, and decode(text), which rebuilds the identical tree from that string.

**Function signature**

def encode(root: Node) -> str

def decode(text: str) -> Node

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `root = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])`<br>`text = encode(root)`<br>`text`<br>`encode(decode(text)) == text` |
| **Output** | `text -> '[1, 3, 3, 2, 5, 0, 6, 0, 2, 0, 4, 0]'`<br>`encode(decode(text)) == text -> true` |
| **Why** | `Each node is written as its value followed by its number of children, in pre-order: 1 has 3 children, 3 has 2, and so on. Decoding reads exactly that many children for each node, so encoding the decoded tree gives the same text.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def encode(node):
    parts = [encode(c) for c in node.children]
    kids = ",".join(parts)
    return f"{node.val}[{kids}]"
def decode(text):
    pos = 0
    def parse():
        nonlocal pos
        start = pos
        while text[pos] != "[":
            pos += 1
        node = Node(text[start:pos])
        pos += 1  # skip "["
        while text[pos] != "]":
            node.children.append(parse())
            if text[pos] == ",":
                pos += 1
        pos += 1  # skip "]"
        return node
    return parse()
```

| TIME | SPACE |
|---|---|
| **O(n * h)** for building strings (O(n) with a list buffer) | **O(n)** |

**WHY IT IS SLOW**

Joining child strings copies text at every level (O(n * h) in the worst case) and the recursive parser can overflow on a deep tree; values containing brackets or commas break it. Pre-order with child counts is O(n) and needs no markers.

## OPTIMAL SOLUTION · Pre-order with child counts, iterative decode

| TIME | SPACE |
|---|---|
| **O(n)** | **O(n)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The values are arbitrary strings that may contain spaces, commas or brackets.

Do not invent delimiters. Put the values in a real container: a JSON list of alternating value and child count, which handles quoting for you, or length-prefix every value (5#hello). The structure part (the counts) stays the same.

## FOLLOW-UP 2 The tree is huge and you want a compact binary encoding.

Counts and node values can be packed with variable-length integers (varint), which makes small values one byte. Keep the same pre-order count layout so decoding is still one pass. Add a version byte at the front so the format can change later.
**Cost:** still O(n) time and space; variable-length integers cost about one byte per small value.

## FOLLOW-UP 3 Encode in level order instead.

Write each node's value and child count in BFS order and decode with a queue: pop a parent, read its count, attach that many new nodes, and push them. BFS order is handy when the consumer wants to stream levels.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a decode and encode function of
# a general N-ary tree.
#
# A general tree has a value in every node and any number
# of children. Write encode(root), which turns the tree
# into a string, and decode(text), which rebuilds the
# identical tree from that string.
#
# Signature:
#   def encode(root: Node) -> str
#   def decode(text: str) -> Node
#
# FOLLOW-UPS (answered below, in this order):
# 1. The values are arbitrary strings that may contain
# spaces, commas or brackets.
# 2. The tree is huge and you want a compact binary
# encoding.
# 3. Encode in level order instead.
#
# INTERVIEW NOTES
# Serialization questions are about a format that can be
# read back unambiguously. Ask what a node holds and
# whether values can contain your delimiters. A node can
# have any number of children, so the decoder must be told
# where the children end: either with a closing marker or
# by writing the child count next to each value. The count
# form needs no closing markers and can be decoded with an
# explicit stack, so deep trees are safe.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import json


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
```

```
    def __init__(self, val, children=None):
        self.val = val
        self.children = []
        if children:
            self.children = children


def preorder_pairs(root):
    # (value, number of children) for every node in pre-order. A stack
    # instead of recursion, so a very deep tree cannot overflow the call stack.
    pairs = []
    stack = []
    if root:
```

```
        stack.append(root)
    while stack:
        node = stack.pop()
        pairs.append((node.val, len(node.children)))
        for i in range(len(node.children) - 1, -1, -1):     # push the LAST child first so the first is pop
ped first
            stack.append(node.children[i])
    return pairs


def build_preorder(pairs):
    # the reverse: rebuild the tree from the same (value, child count) pairs
    if not pairs:
        return None
    root = Node(pairs[0][0])
    stack = [[root, pairs[0][1]]]              # [a node, how many children it still has to receive]
    position = 1
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()                        # this node is complete
            continue
        top[1] -= 1
        value, count = pairs[position]
        position += 1
        child = Node(value)
        top[0].children.append(child)
        stack.append([child, count])
    return root


def same(a, b):
    # do two trees have the same shape and values?
```

```
    return preorder_pairs(a) == preorder_pairs(b)


# ---- main question: main functions ----
# Idea: write every node as "value child_count" in PRE-ORDER (a node
# first, then its children in order). The counts say where each node's
# children end, so no brackets are needed. Values here are words without
# spaces (follow-up 1 handles any string).
# Complexity: Time O(n), Space O(n)
def encode(root):
    parts = []
    for value, count in preorder_pairs(root):
        parts.append(str(value))
        parts.append(str(count))
    return " ".join(parts)


# Complexity: Time O(n), Space O(n)
def decode(text):
    if text == "":
        return None
    tokens = text.split(" ")
    pairs = []
    for i in range(0, len(tokens), 2):
        pairs.append((tokens[i], int(tokens[i + 1])))
    return build_preorder(pairs)


# ---- main question: tests ----
def example_tree():
    return Node("a", [Node("b", [Node("c")]), Node("d"), Node("e", [Node("f"), Node("g")])])
```

```
def test_main_question():
    tree = example_tree()
    assert encode(tree) == "a 3 b 1 c 0 d 0 e 2 f 0 g 0"
    assert same(decode(encode(tree)), tree)
    assert encode(None) == "" and decode("") is None


# ========================================================
# FOLLOW-UP 1: The values are arbitrary strings that may contain spaces, commas or brackets
# ========================================================
# ---- follow-up 1: function ----
# Answer: do not invent delimiters a value could contain. Put the values
```

```
# in a real container: a JSON list of alternating value and child count
# (JSON does the quoting), or write the length before every value
# ("5#hello"). The counts, which carry the structure, stay the same.
# Complexity: Time O(n), Space O(n)
def encode_any(root):
    flat = []
    for value, count in preorder_pairs(root):
        flat.append(value)
        flat.append(count)
    return json.dumps(flat)


# Complexity: Time O(n), Space O(n)
def decode_any(text):
    flat = json.loads(text)
    pairs = []
    for i in range(0, len(flat), 2):
        pairs.append((flat[i], flat[i + 1]))
    return build_preorder(pairs)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = Node("a, b", [Node("[x]", [Node("")]), Node("\u00e9 \u4e16\u754c"), Node('q"uote', [Node("1"), N
ode("2")])])
    assert same(decode_any(encode_any(tree)), tree)
    assert encode_any(None) == "[]" and decode_any("[]") is None
    chain = Node("0")                                       # a chain of 5000 nodes: nothing is recursive
    last = chain
    for i in range(1, 5000):
        last.children.append(Node(str(i)))
        last = last.children[0]
    assert same(decode_any(encode_any(chain)), chain)


# ========================================================
# FOLLOW-UP 2: The tree is huge and you want a compact binary encoding
# ========================================================
# ---- follow-up 2: helper functions ----
def put_varint(n, out):
    # a whole number in 7-bit pieces: the top bit of a byte says "another byte follows"
    while n >= 128:
        out.append(n % 128 + 128)
        n = n // 128
    out.append(n)
```

```
def get_varint(data, position):
    # read one varint starting at `position`; returns (number, position after it)
    n = 0
```

```
    shift = 0
    while data[position] >= 128:
        n += (data[position] - 128) << shift
        shift += 7
        position += 1
    n += data[position] << shift
    return n, position + 1


# ---- follow-up 2: function ----
# Answer: pack the lengths and counts as variable-length integers
# (small numbers take one byte). Keep the same pre-order "value, count"
# layout, so decoding is still one pass.
# Complexity: Time O(n), Space O(n)
def encode_binary(root):
    out = bytearray([1])                          # a version byte first, so the format can change later
    for value, count in preorder_pairs(root):
        raw = value.encode("utf-8")
        put_varint(len(raw), out)
        out += raw
        put_varint(count, out)
    return bytes(out)


# Complexity: Time O(n), Space O(n)
def decode_binary(data):
    if data[0] != 1:
        raise ValueError("unknown version")
    pairs = []
    position = 1
    while position < len(data):
        size, position = get_varint(data, position)
        value = data[position:position + size].decode("utf-8")
        count, position = get_varint(data, position + size)
        pairs.append((value, count))
    return build_preorder(pairs)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    tree = example_tree()
```

```
    assert same(decode_binary(encode_binary(tree)), tree)
    assert decode_binary(encode_binary(None)) is None
    wide = Node("w", [Node(str(i)) for i in range(300)])       # 300 children need a two byte count
    assert same(decode_binary(encode_binary(wide)), wide)
    assert len(preorder_pairs(wide)) == 301


# ========================================================
# FOLLOW-UP 3: Encode in level order instead
# ========================================================
# ---- follow-up 3: function ----
# Answer: write each node's value and child count in level (BFS) order
# and decode with a queue: take a parent out, read its count, attach
# that many new nodes and put them in the queue. Handy when the reader
# wants to stream the tree level by level.
# Complexity: Time O(n), Space O(n)
def level_pairs(root):
    pairs = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        node = queue.popleft()
```

```
        pairs.append((node.val, len(node.children)))
        for child in node.children:
            queue.append(child)
    return pairs


# Complexity: Time O(n), Space O(n)
def build_levels(pairs):
    if not pairs:
        return None
    root = Node(pairs[0][0])
    queue = deque([(root, pairs[0][1])])
    position = 1
    while queue:
        parent, count = queue.popleft()
        for i in range(count):
            value, kids = pairs[position]
            position += 1
            child = Node(value)
            parent.children.append(child)
            queue.append((child, kids))
    return root
```

```


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = example_tree()
    assert level_pairs(tree)[:4] == [("a", 3), ("b", 1), ("d", 0), ("e", 2)]
    assert same(build_levels(level_pairs(tree)), tree)
    assert build_levels(level_pairs(None)) is None


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**OFFICIAL PAGE TEXT**

Log in or sign up to answer this question.

**STUDY PATH: question 46 of 53 | topic 27 of 30: Build and serialize: write a structure out, read it back with the same walk | 2 of 2**

Template fit: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking | asked 391 days ago | frequency Single report

**HARD | ColorStack | technical interview | 46 of 53**

# Serialize and Deserialize a Graph

ASKED 2025-09-24 | 391 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 203/288

## ABOUT THIS QUESTION

**Topic:** Build and serialize: write a structure out, read it back with the same walk

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #133 Clone Graph, LeetCode #297 Serialize and Deserialize Binary Tree

**LeetCode match:** LC 133 Clone Graph / LC 297 Serialize and Deserialize Binary Tree (related)

Frequency: Single report | Last asked: 2025-09-24 | Roles: SWE Intern

## QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

**Function signature**

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `adjList = [[]] # a single node with no neighbors` |
| **Output** | `A single node with val 1 and an empty neighbor list.` |

| Example 2 | |
|---|---|
| **Input** | `node = None # empty graph` |
| **Output** | `codec.deserialize(codec.serialize(None)) is None` |

CONSTRAINTS

## QUESTION

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def brute_edges(node):
    # the edge list found by a plain traversal with a visited set
    seen, edges, stack = {node.val}, [], [node]
    while stack:
        cur = stack.pop()
        for nb in cur.neighbors:
            edges.append((cur.val, nb.val))
            if nb.val not in seen:
                seen.add(nb.val)
                stack.append(nb)
    return sorted(edges)
```

| TIME | SPACE |
|---|---|
| does not finish on a cycle | - |

### WHY IT IS SLOW

Without remembering visited nodes a cycle is expanded forever; a visited set is the minimum fix.

## OPTIMAL SOLUTION · BFS to a text, two passes back

| TIME | SPACE |
|---|---|
| O(V + E) | O(V + E) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Support directed graphs, where the adjacency is not symmetric.

The format already stores each node's own neighbour list, so a directed edge u -> v is kept as given and nothing assumes the reverse edge exists. Only nodes reachable from the starting node are encoded, which is the same as for the undirected case.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Serialize and Deserialize a Graph
#
# Design a codec that can serialize a connected,
# undirected graph to a string and deserialize that string
# back into an identical graph. Each node has a unique
# integer val and a list of neighbors . The graph may
# contain cycles. After a round trip, the reconstructed
# graph must have the same node values and the same
# adjacency relationships as the original. You are given
# the reference node class below. Implement serialize
# (node -> string) and deserialize (string -> node).
# serialize(None) must round-trip to None via deserialize
```

```
# . Function signature: class Node: def __init__(self,
# val: int = 0, neighbors: list["Node"] | None = None):
# self.val = val self.neighbors = neighbors if neighbors
# is not None else [] class Codec: def serialize(self,
# node: "Node | None") -> str: """Encode the whole graph
# reachable from node into a string.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support directed graphs, where the adjacency is not
# symmetric.
#
# INTERVIEW NOTES
# A graph can contain cycles, so the encoding must name
# nodes by value and list their neighbours instead of
# nesting them like a tree. Walk the graph with BFS from
# the given node once, writing one entry per node
# ('val:n1,n2'); deserialize creates all nodes first, then
# links the neighbours, which also handles cycles and
# directed edges.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = []
        if neighbors is not None:
            self.neighbors = neighbors


def build_graph(adj):
    # adj[i] lists the neighbours of node i + 1, e.g. [[2, 4], [1, 3], [2, 4], [1, 3]]
    nodes = []
    for i in range(len(adj)):
        nodes.append(Node(i + 1))
    for i in range(len(adj)):
        for j in adj[i]:
            nodes[i].neighbors.append(nodes[j - 1])
    if nodes:
        return nodes[0]
    return None


# ---- main question: main function ----
# Idea: one entry "value:neighbour,neighbour" per node, found with a
# BFS from the given node, separated by ";". To read it back, create
# every node FIRST and link the neighbours afterwards, so a neighbour
# always exists before it is linked (a graph can have cycles).
# Complexity: Time encode and decode are O(V + E), Space O(V + E)
class Codec:
    def serialize(self, node):
```

```
        if node is None:
            return ""
        seen = {node.val}
        queue = deque([node])
        parts = []
        while queue:
            current = queue.popleft()
            names = []
            for neighbour in current.neighbors:
                names.append(str(neighbour.val))
                if neighbour.val not in seen:
                    seen.add(neighbour.val)
                    queue.append(neighbour)
            parts.append(str(current.val) + ":" + ",".join(names))
```

```
        return ";".join(parts)

    def deserialize(self, data):
        if data == "":
            return None
        rows = []
        for part in data.split(";"):
            rows.append(part.split(":"))
        nodes = {}
        for value, neighbour_text in rows:            # pass 1: create every node
            nodes[int(value)] = Node(int(value))
        for value, neighbour_text in rows:            # pass 2: link the neighbours
            if neighbour_text != "":
                for name in neighbour_text.split(","):
                    nodes[int(value)].neighbors.append(nodes[int(name)])
        return nodes[int(rows[0][0])]


# ---- main question: tests ----
def test_main_question():
    codec = Codec()
    assert codec.serialize(None) == "" and codec.deserialize("") is None
    square = build_graph([[2, 4], [1, 3], [2, 4], [1, 3]])       # a cycle of four nodes
    text = codec.serialize(square)
    assert text == "1:2,4;2:1,3;4:1,3;3:2,4"
    assert codec.serialize(codec.deserialize(text)) == text
    single = codec.deserialize(codec.serialize(build_graph([[]])))
    assert single.val == 1 and single.neighbors == []


# =========================================================
# FOLLOW-UP 1: Support directed graphs, where the adjacency is not symmetric
# =========================================================
# Answer: nothing to change: every node stores ITS OWN neighbour list, so
# a directed edge u -> v is saved as given and nothing assumes the
# reverse edge exists. (Reachability from the start node is what gets saved.)
# ---- follow-up 1: tests ----
def test_follow_up_1():
    codec = Codec()
    random.seed(25)
    for round_number in range(200):               # random graphs, edges in one direction only
        n = random.randint(1, 6)
        adj = []
        for i in range(n):
            adj.append(sorted(random.sample(range(1, n + 1), random.randint(0, min(3, n)))))
```

```
        start = build_graph(adj)
        text = codec.serialize(start)
        assert codec.serialize(codec.deserialize(text)) == text
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 28 OF 30

# Games: win or lose with a cache, and guessing with feedback

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**4**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Games: win or lose with a cache, and guessing with feedback | 4 questions | tap a title to jump to it**

**READ FIRST: the code template for these 4 questions (tap here) >**


| | | | |
|---|---|---|---|
| H | 47. Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | 2025-12-15 |
| H | 48. Determine winner in stack-merging game | PracHub | 2026-01-06 |
| H | 49. Find secret word with match-count feedback | PracHub | 2026-02-11 |
| H | 50. Babylon Tile-Stack Merge Game | 1Point3Acres | 2025-12-29 |

CODE TEMPLATE | GROUP 28 OF 30 | 4 QUESTIONS, ONE PATTERN

# Games: win or lose with a cache, and guessing with feedback

## The one idea these questions share

For two-player games, a state is winning if some move leads to a state where the opponent loses; cache every state. For guessing games, pick the guess that shrinks the worst remaining group of candidates the most, then keep only the candidates that agree with the feedback.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def first_player_wins(state, moves):
    @lru_cache(maxsize=None)
    def wins(s):
        # CHANGE: moves(s) lists the states you can move to; you win if some move leaves the opponent losing
        return any(not wins(t) for t in moves(s))

    return wins(state)
```

## 2. Helper functions that usually get written next to it

```
from collections import Counter


def match_count(a, b):                          # feedback: how many positions agree
    return sum(x == y for x, y in zip(a, b))


def best_guess(candidates):                     # minimise the largest group left after the feedback
    return min(candidates, key=lambda g: (max(Counter(match_count(g, w) for w in candidates).values()), g))
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **47** | Wordle-Style 5-Letter Minimum-Guess Strategy | ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest) |
| **48** | Determine winner in stack-merging game | state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap |
| **49** | Find secret word with match-count feedback | guess the word that minimises the size of the biggest feedback group; keep the words with the same match count |
| **50** | Babylon Tile-Stack Merge Game | state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 47 of 53 | topic 28 of 30: Games: win or lose with a cache, and guessing with feedback | 1 of 4**

Template fit: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest) | asked 309 days ago | frequency Single report

**HARD | 1Point3Acres | phone-screen, onsite-coding | 47 of 53**

# Wordle-Style 5-Letter Minimum-Guess Strategy

ASKED 2025-12-15 | 309 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 98/288

## ABOUT THIS QUESTION

**Topic:** Games: win or lose with a cache, and guessing with feedback

**Optimal answer from:** written and tested here for this exact question

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #843 Guess the Word

**LeetCode match:** LC 843 Guess the Word (related)

Frequency: Single report | Last asked: 2025-12-15 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

## QUESTION

A hidden word has 5 lowercase letters. The only thing you can ask is the oracle query(pos, letter), which says whether the letter is at that position (positions 0 to 4). Find the word with as few oracle calls as possible. by_frequency tries letters in order of English frequency; with_dictionary also knows a list of candidate words and always asks the question that splits the candidates most evenly. Both return (word, number of calls).

**Function signature**

def by_frequency(query) -> (str, int)

def with_dictionary(query, dictionary: list) -> (str, int)

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `secret = 'crane'`<br>`query = lambda pos, letter: secret[pos] == letter`<br>`by_frequency(query)`<br>`with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape'])` |
| **Output** | `by_frequency(query) -> ('crane', 31)`<br>`with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape']) -> ('crane', 2)` |
| **Why** | `Without a dictionary every position is searched letter by letter; with five candidate words a handful of well-chosen questions are enough.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
word = ""
for i in range(5):
    for c in "abcdefghijklmnopqrstuvwxyz":
        if query(i, c):
            word += c
            break
```

| TIME | SPACE |
|---|---|
| 130 calls | O(1) |

## BRUTE FORCE

WHY IT IS SLOW

Always 130 calls. Asking likely letters first, skipping the implied last letter, or splitting a dictionary of candidates in half needs far fewer.

## OPTIMAL SOLUTION · Frequency order, or entropy split when a dictionary is known

| TIME | SPACE |
|---|---|
| **at most 5 x 25 calls with...**<br>at most 5 x 25 calls without a dictionary; about log2(words) with one | **O(words)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 What is the lower bound and when is it reached?

Every call returns at most one bit, so at least ceil(log2(candidates)) calls are needed. The entropy split reaches it only when a question can cut the candidates exactly in half; real dictionaries usually need a few more. A greedy scoring rule (closest to half) is enough to state; the interviewer mostly wants the entropy idea.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Wordle-Style 5-Letter Minimum-Guess
# Strategy
#
# A hidden word has 5 lowercase letters. The only thing
# you can ask is the oracle query(pos, letter), which says
# whether the letter is at that position (positions 0 to
# 4). Find the word with as few oracle calls as possible.
# by_frequency tries letters in order of English
# frequency; with_dictionary also knows a list of
# candidate words and always asks the question that splits
# the candidates most evenly. Both return (word, number of
# calls).
#
# Signature:
#   def by_frequency(query) -> (str, int)
#   def with_dictionary(query, dictionary: list) -> (str, int)
#
# FOLLOW-UPS (answered below, in this order):
# 1. What is the lower bound and when is it reached?
#
# INTERVIEW NOTES
# Each call returns one bit, so you can never beat
# ceil(log2(number of candidates)) calls. Without a
# dictionary the best you can do is order the questions by
# likelihood and stop at the first yes for each position
# (and skip the 26th letter, because it is implied). With
# a dictionary, keep the list of words still possible and
# ask the (position, letter) question whose yes/no answer
# splits that list most evenly.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
# letters from the most to the least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"
```

```
def split_score(words, i, c):
    # how evenly does the question "is letter c at position i?" split the words?
    # The bigger the smaller side, the better: a perfect split is half and half.
    yes = 0
    for word in words:
        if word[i] == c:
            yes += 1
    no = len(words) - yes
    if yes < no:
        return yes
    return no


# ---- main question: main function ----
# query(i, c) is the black box: is letter c at position i of the secret?
# Idea (no dictionary): for each position ask the letters from the most
# to the least common and stop at the first "yes"; if 25 letters said
# "no" the 26th letter is implied and needs no call.
# Complexity: Time O(1) (at most 5 * 25 = 125 calls), Space O(1)
def by_frequency(query):
    calls = 0
    word = []
    for i in range(5):
        for k in range(26):
            c = BY_FREQUENCY[k]
            if k == 25:
                word.append(c)              # the last letter left: no call needed
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls


# Idea (a dictionary is known): keep the words that still fit every
# answer, and always ask the question that cuts the list closest to
# half, so each call removes about half of the words.
# Complexity: Time O(rounds * n^2) for n dictionary words (about log2 n rounds / calls), Space O(n)
def with_dictionary(query, dictionary):
    words = list(dictionary)
    calls = 0
    while len(words) > 1:
        options = set()
        for word in words:
```

```
            for i in range(5):
                options.add((i, word[i]))
        best_option = None
        best_value = -1
        for option in sorted(options):
            value = split_score(words, option[0], option[1])
            if value > best_value:
                best_value = value
                best_option = option
```

```
        i, c = best_option
        calls += 1
        answer = query(i, c)
        remaining = []
        for word in words:
            if (word[i] == c) == answer:
                remaining.append(word)
        words = remaining
    return words[0], calls


# ---- main question: tests ----
def make_oracle(word):
    def query(i, c):
        return word[i] == c
    return query


def test_main_question():
    word, calls = by_frequency(make_oracle("grape"))
    assert word == "grape" and calls <= 125
    words = ["grape", "grade", "trade", "crane", "plane", "slate", "stare", "share"]
    fewest = 100
    for target in words:
        found, used = with_dictionary(make_oracle(target), words)
        assert found == target
        assert used <= 6                        # about log2(8) = 3 questions suffice; allow a few more
        if used < fewest:
            fewest = used
    assert fewest >= 3                          # one bit per call: at least ceil(log2(8)) = 3 calls


# ============================================================
# FOLLOW-UP 1: What is the lower bound and when is it reached?
# ============================================================
# Answer: every call returns at most one bit (yes or no), so at least
```

```
# ceil(log2(number of candidates)) calls are needed. The half-and-half
# question (split_score) reaches that bound only when a question can
# cut the candidates exactly in half every time; real dictionaries
# usually need a few more calls.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

**STUDY PATH: question 48 of 53 | topic 28 of 30: Games: win or lose with a cache, and guessing with feedback | 2 of 4**

Template fit: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap | asked 287 days ago

**HARD | PracHub | Technical Screen | 48 of 53**

# Determine winner in stack-merging game

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 31.2 | 30 DAYS 0 | PRIORITY 141/288

## ABOUT THIS QUESTION

**Topic:** Games: win or lose with a cache, and guessing with feedback

**Optimal answer from:** the interview site's own solution

**Related in your app (a neighbouring problem, not this exact question):** LeetCode #877 Stone Game

**LeetCode match:** Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-14)

**Original:** https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

## QUESTION

A game starts with stacks of the given sizes. A move merges two stacks into one if their combined size is at most cap. The player who cannot move loses (in the misere variant, the player who cannot move wins). Return whether the first player wins with best play.

### Function signature

def first_player_wins(stacks: list, cap: int, misere=False) -> bool

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[1, 1], 2` |
| Output | `true` |
| Why | `Merge the two stacks of 1 into 2; the opponent has a single stack and cannot move.` |

| Example 2 | |
|---|---|
| Input | `[1, 1, 1], 2` |
| Output | `true` |
| Why | `Any merge makes a stack of 2 next to a stack of 1, whose sum 3 exceeds the cap: the opponent is stuck.` |

| Example 3 | |
|---|---|
| Input | `[3], 5` |
| Output | `false` |
| Why | `A single stack: the first player cannot move and loses.` |

| Example 4 | |
|---|---|
| Input | `[3], 5, misere = true` |
| Output | `true` |
| Why | `The same single stack in the misere variant: being unable to move wins.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(state):
    for nxt in all_merges(state):
        if not wins(nxt):
            return True  # opponent loses
    return False  # no move wins
```

| TIME | SPACE |
| --- | --- |
| **exponential in the number...**<br>exponential in the number of stacks | **O(depth)** |

WHY IT IS SLOW

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

## OPTIMAL SOLUTION · Memoize on the sorted state

| TIME | SPACE |
| --- | --- |
| **O(number of distinct sorted states * k^2)** | **O(number of states)** |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

## FOLLOW-UP 2 Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

## FOLLOW-UP 3 Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine winner in stack-merging game
#
# A game starts with stacks of the given sizes. A move
# merges two stacks into one if their combined size is at
# most cap. The player who cannot move loses (in the
# misere variant, the player who cannot move wins). Return
# whether the first player wins with best play.
#
# Signature:
#   def first_player_wins(stacks: list, cap: int, misere=False) -> bool
#
```

```
# Examples:
# [1, 1], 2  ->  true
# [1, 1, 1], 2  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. The game is a sum of independent games (several
# separate boards or piles).
# 2. Misere play (the player who cannot move wins).
# 3. Count the number of winning first moves.
#
# INTERVIEW NOTES
# Two-player game with perfect play and no draws: define
# win(state) = some move leads to a state where the
# opponent loses. A state with no moves is a loss for
# whoever must move. If a move does not care about stack
# order, sort the state so equal positions share one cache
# entry. If the game splits into independent parts, use
# Grundy numbers and XOR them.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def moves(state, cap):
    # every position one move away: merge two stacks if the new stack fits under the cap.
    # `state` is a sorted tuple of stack sizes; equal results count as one move.
    results = []
```

```
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] > cap:
                continue
            rest = []
            for k in range(len(state)):
                if k != i and k != j:
                    rest.append(state[k])
            rest.append(state[i] + state[j])
            new_state = tuple(sorted(rest))
            if new_state not in results:
                results.append(new_state)
    return results


# ---- main question: main function ----
# Idea: merging does not depend on the order, so a sorted tuple
# identifies a position. Solve each position once and remember it
# (a dictionary), because many move orders reach the same position.
# A position is WON for the player to move when some move leaves the
# opponent in a LOST position. The player who cannot move loses
# (misere play, follow-up 2: the player who cannot move wins).
# Complexity: Time O(S * M) for S reachable states and M moves from each (S can grow exponentially with the
 number of stacks), Space O(S)
def first_player_wins(stacks, cap, misere=False):
    memo = {}

    def wins(state):
        if state in memo:
            return memo[state]
```

```
        options = moves(state, cap)
        if not options:
            answer = misere                  # no move: loses (or wins, in misere play)
        else:
            answer = False
            for next_state in options:
                if not wins(next_state):     # a winning move leaves the opponent lost
                    answer = True
                    break
        memo[state] = answer
        return answer

    return wins(tuple(sorted(stacks)))


# ---- main question: tests ----
```

```
def test_main_question():
    assert not first_player_wins([3], 5)              # one stack: no move, the first player loses
    assert first_player_wins([1, 1], 5)               # one merge ends the game, we win
    assert first_player_wins([1, 1, 1], 2) is True


# =====================================================
# FOLLOW-UP 1: The game is a sum of independent games (several separate boards or piles)
# =====================================================
# ---- follow-up 1: function ----
# Answer: compute each board's Grundy number (the smallest number that
# is not the Grundy number of any position one move away, the "mex")
# and XOR them. The first player wins exactly when the XOR is not zero.
# Complexity: Time O(S * M), Space O(S)
def grundy(stacks, cap):
    memo = {}

    def number(state):
        if state in memo:
            return memo[state]
        reachable = set()
        for next_state in moves(state, cap):
            reachable.add(number(next_state))
        value = 0
        while value in reachable:
            value += 1
        memo[state] = value
        return value

    return number(tuple(sorted(stacks)))


# Complexity: Time O(sum of the Grundy computations) plus O(boards), Space O(S)
def sum_of_games_wins(boards, cap):
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)          # normal play: wins exactly when Grundy is not 0
```

```
    assert sum_of_games_wins([[1, 1], [1, 1]], 5) is False   # two equal boards cancel out
```

```
# ============================================================
# FOLLOW-UP 2: Misere play (the player who cannot move wins)
# ============================================================
# ---- follow-up 2: tests ----
# Answer: flip the base case of the search (no move means the player
# to move WINS): the `misere` parameter above. Grundy numbers no longer
# combine by XOR in general, so search the whole position.
def test_follow_up_2():
    assert first_player_wins([3], 5, misere=True)        # a single stack: no move, so the first player win
s
    assert not first_player_wins([1, 1], 5, misere=True)


# ============================================================
# FOLLOW-UP 3: Count the number of winning first moves
# ============================================================
# ---- follow-up 3: function ----
# Answer: the same win / lose search; count the moves whose resulting
# position is lost for the opponent.
# Complexity: Time O(M * S * M), Space O(S)
def winning_first_moves(stacks, cap):
    count = 0
    for next_state in moves(tuple(sorted(stacks)), cap):
        if not first_player_wins(list(next_state), cap):
            count += 1
    return count


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert winning_first_moves([1, 1, 1], 2) >= 1
    assert winning_first_moves([3], 5) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

### Test cases

| Input | Expected |
|---|---|
| `['Y']` | `False` |

| Input | Expected |
| --- | --- |
| `['Y', 'W']` | `True` |

**ALSO REPORTED · 1 more**

**Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report**

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API.
Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".
Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

**STUDY PATH: question 49 of 53 | topic 28 of 30: Games: win or lose with a cache, and guessing with feedback | 3 of 4**

Template fit: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count | asked 251 days ago

**HARD | IN APP LC 843 | PracHub | Technical Screen | 49 of 53**

# Find secret word with match-count feedback

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 42.2 30 DAYS 0 PRIORITY 97/288

## ABOUT THIS QUESTION

**Topic:** Games: win or lose with a cache, and guessing with feedback

**Optimal answer from:** the interview site's own solution

**Also in your app:** LeetCode #843 Guess the Word

**LeetCode match:** LC 843 Guess the Word

Posted on PracHub: 2026-02-11 (updated 2026-04-30)

**Original:** https://prachub.com/coding-questions/find-secret-word-with-match-count-feedback

## QUESTION

### Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- **`matchCount(guess) -> int`**: when you submit a word **`guess`** from the list, the API returns the number of positions **`i`** such that **`guess[i] == secret[i]`** (i.e., exact character match at the same index). You are allowed to call **`matchCount`**

**at most 10 times**.

### Task

Design/implement a function (e.g., **`findSecretWord(words)`**) that uses the API to identify the secret word within the guess limit.

### Notes / Constraints

- **`words`** contains between 1 and 100 words.
- All words have equal length **`L`** (commonly **`L = 6`**).
- Every guess must be a word from **`words`**.
- After each API response, you may adapt your next guess.

### What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once **`matchCount(guess) == L`** (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length **`L`**. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- **`matchCount(guess) -> int`**
- It returns how many positions **`i`** satisfy **`guess[i] == secret[i]`** You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement **`solution(words, secret)`** and simulate the API inside your function. The parameter **`secret`** is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string **`""`**.

## QUESTION

Constraints`1 <= len(words) <= 1001 <= len(words[i]) <= 10`All words are unique, lowercase, and have the same length `Lsecret` is guaranteed to be one of the words in `words`Your algorithm may use at most 10 simulated `matchCount` callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until `acckzz` is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

**Hints**

After a guess returns **k**, any valid remaining candidate must also have exactly **k** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    candidates = list(words)
    while True:
        guess = candidates[0]
        m = matchCount(guess)
        if m == len(guess):
            return guess
        candidates = [w for w in candidates
                      if matches(w, guess) == m]
```

| TIME | SPACE |
|---|---|
| O(guesses * W * L) | O(W) |

WHY IT IS SLOW

A poor guess may remove very few candidates, so there is no guarantee of finishing in 10 guesses. Guessing the word whose largest match-count group is smallest (minimax) keeps the list shrinking fast.

## OPTIMAL SOLUTION · Guess the word that minimises the size of its largest match-count group

| TIME | SPACE |
|---|---|
| O(guesses * W^2 * L) | O(W) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

**FOLLOW-UP 2 Guess at random instead of the minimax choice.**

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

**FOLLOW-UP 3 The word list is much larger.**

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def match(a, b):
    # the number of positions that hold the same letter
    count = 0
    for i in range(len(a)):
        if a[i] == b[i]:
            count += 1
    return count


def make_words(alphabet, length):
    # every word of the given length over the alphabet
    words = [""]
    for position in range(length):
        longer = []
        for word in words:
            for letter in alphabet:
                longer.append(word + letter)
        words = longer
    return words


# ---- main question: main function ----
# Idea: the secret agrees with every earlier guess in exactly as many
# positions as the API reported, so after each guess only the words
# with that same match count against the guess can still be the secret.
# Guess the word that makes the BIGGEST remaining group as small as
# possible (minimax), so the candidates shrink fast.
# Complexity: Time O(n^2 * L) for n candidates of length L, Space O(L)
def best_guess(candidates):
    best_word = None
    best_worst = None
    for guess in candidates:
        group_sizes = {}
        for word in candidates:
```

```
            m = match(guess, word)
            group_sizes[m] = group_sizes.get(m, 0) + 1
        worst = 0
        for size in group_sizes.values():
            if size > worst:
                worst = size
        if best_worst is None or worst < best_worst or (worst == best_worst and guess < best_word):
            best_worst = worst
            best_word = guess
    return best_word


# Complexity: Time O(guesses * n^2 * L), Space O(n)
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API: how many positions match the secret
    candidates = list(words)
    guesses = 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        remaining = []
        for word in candidates:
            if match(guess, word) == score:
                remaining.append(word)
        candidates = remaining
    return None, guesses


# ---- main question: tests ----
def test_main_question():
    words = make_words("abc", 4)
    worst_used = 0
    for secret in words:
        calls = []

        def api(guess, secret=secret):
            calls.append(guess)
            return match(guess, secret)

        found, used = find_secret(words, api)
        assert found == secret and used == len(calls)
        if used > worst_used:
            worst_used = used
    assert worst_used <= 10                      # all 81 words are found inside the limit of 10

    def always_right(guess):
        return 4

    assert find_secret(["only"], always_right) == ("only", 1)


# ======================================================
# FOLLOW-UP 1: Show how the candidate list shrinks
# ======================================================
# ---- follow-up 1: function ----
# Answer: after a guess whose answer is m, only words with match count
# m against that guess survive. Printing the sizes shows the list
# shrinking round by round.
# Complexity: Time O(guesses * n^2 * L), Space O(n)
def shrink_report(words, secret):
    sizes = [len(words)]
    candidates = list(words)
```

```
    while len(candidates) > 1:
        guess = best_guess(candidates)
        score = match(guess, secret)
        if score == len(secret):
            break
        remaining = []
        for word in candidates:
            if match(guess, word) == score:
                remaining.append(word)
        candidates = remaining
        sizes.append(len(candidates))
    return sizes


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sizes = shrink_report(make_words("abc", 4), "abca")
    assert sizes[0] == 81
    for i in range(len(sizes) - 1):
        assert sizes[i] > sizes[i + 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solution(words: list[str], secret: str) -> str:
```

### Sample call

```
solution(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

### Test cases

| Input | Expected |
|---|---|
| (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz") | 'acckzz' |
| (["secret"], "secret") | 'secret' |

**STUDY PATH: question 50 of 53 | topic 28 of 30: Games: win or lose with a cache, and guessing with feedback | 4 of 4**

Template fit: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply | asked 295 days ago | frequency Single report

**HARD | 1Point3Acres | onsite-coding | 50 of 53**

# Babylon Tile-Stack Merge Game

ASKED 2025-12-29 | 295 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 251/288

## ABOUT THIS QUESTION

**Topic:** Games: win or lose with a cache, and guessing with feedback

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2025-12-29 | Roles: SWE

**Original:** https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

## QUESTION

A two-player game on stacks of tiles. Each stack has a height and a colour (the colour of its top tile). A move places one stack on top of another when the two stacks have the same height or the same colour; the new stack has the combined height and the colour of the stack that was placed on top. The player who cannot move loses. Given the stacks as (height, colour) pairs, return whether the player to move wins with best play.

### Function signature

def babylon_wins(stacks: list) -> bool

EXAMPLES

| Example 1 | |
|---|---|
| Input | `[(1, 'Y'), (1, 'W')]` |
| Output | `true` |
| Why | `Two stacks of height 1: the mover merges them, and the opponent has nothing left to move.` |

| Example 2 | |
|---|---|
| Input | `[(1, 'Y')]` |
| Output | `false` |
| Why | `A single stack: the mover has no move and loses.` |

| Example 3 | |
|---|---|
| Input | `[(1, 'Y'), (2, 'W')]` |
| Output | `false` |
| Why | `Heights differ and colours differ: no move is possible, so the mover loses.` |

| Example 4 | |
|---|---|
| Input | `[(1, 'Y'), (1, 'Y'), (1, 'W'), (1, 'W')]` |
| Output | `true` |
| Why | `Four height-1 stacks of two colours: the mover has a winning plan.` |

## BRUTE FORCE

### BRUTE FORCE · PSEUDO-CODE

```
def wins(position):
    for i, j in legal_moves(position):
        nxt = merge(position, i, j)
        if not wins(nxt):
            return True  # opponent loses
    return False
```

| TIME | SPACE |
|---|---|
| exponential | O(depth) |

### WHY IT IS SLOW

Explores the same positions many times, exponential. Sorting the stacks and caching each position's result visits every distinct position once.

## OPTIMAL SOLUTION · Negamax with memoisation on the sorted stack tuple

| TIME | SPACE |
|---|---|
| O(states * moves^2) | O(states) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 How big is the real state space?

With 12 starting stacks the first position has at most 66 unordered pairs and the number of reachable multisets of (height, colour) stacks shrinks quickly because every merge removes a stack, so memoised search is feasible. In the interview build the clean state class, legal moves and play(move) first and add the solver only if there is time.

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Babylon Tile-Stack Merge Game
#
# A two-player game on stacks of tiles. Each stack has a
# height and a colour (the colour of its top tile). A move
# places one stack on top of another when the two stacks
# have the same height or the same colour; the new stack
# has the combined height and the colour of the stack that
# was placed on top. The player who cannot move loses.
# Given the stacks as (height, colour) pairs, return
# whether the player to move wins with best play.
#
# Signature:
#   def babylon_wins(stacks: list) -> bool
#
# Examples:
# [(1, 'Y'), (1, 'W')]  ->  true
# [(1, 'Y')]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. How big is the real state space?
#
# INTERVIEW NOTES
```

```
# Model a state as the list of stacks, each as (height,
# top colour). A legal move puts stack i on stack j when
# the heights are equal or the top colours are equal; the
# new stack has the summed height and the colour of the
# stack that was placed on top. The player with no legal
# move loses, so a state is winning when some move leads
# to a losing state. The order of stacks does not matter,
# so sort the list to share states in a cache.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Babylon:
    # a stack is (height, colour of its top tile); players take turns putting
    # one stack on top of another, allowed when the two stacks have the same
    # height or the same colour on top. A player with no legal move loses.
    def __init__(self, stacks=None):
        if stacks is None:
            stacks = []
            for colour in "YWGB" * 3:               # the real start: 12 tiles, three of each colour
                stacks.append((1, colour))
        self.stacks = list(stacks)

    def legal_moves(self):
        # (i, j) means: put stack i on top of stack j
        moves = []
        for i in range(len(self.stacks)):
            for j in range(len(self.stacks)):
                if i == j:
                    continue
                height_i, colour_i = self.stacks[i]
                height_j, colour_j = self.stacks[j]
                if height_i == height_j or colour_i == colour_j:
                    moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move; the game itself is not changed
        height_i, colour_i = self.stacks[i]
        height_j, colour_j = self.stacks[j]
        rest = []
        for k in range(len(self.stacks)):
            if k != i and k != j:
                rest.append(self.stacks[k])
        rest.append((height_i + height_j, colour_i))      # heights add; the moved stack's colour is on top
        return rest

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        return not self.legal_moves()
```

```
# ---- main question: main function ----
# Idea: a position is WON for the player to move when some legal move
# leaves the opponent in a LOST position (and a position with no move is
# lost). Remember every solved position; sort the stacks first so equal
```

```
# positions share one memory entry.
# Complexity: Time O(S * M) for S reachable states and M moves from each, Space O(S)
def babylon_wins(stacks):
    memo = {}

    def solve(state):
        if state in memo:
            return memo[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            next_state = tuple(sorted(game.after(i, j)))
            if not solve(next_state):                    # a move to a position the opponent loses
                result = True
                break
        memo[state] = result
        return result

    return solve(tuple(sorted(stacks)))


# Complexity: Time O(M * S * M), Space O(S)
def best_move(game):
    # a move that leaves the opponent lost, if there is one
    for i, j in game.legal_moves():
        if not babylon_wins(game.after(i, j)):
            return i, j
    moves = game.legal_moves()
    if moves:
        return moves[0]
    return None


# ---- main question: tests ----
def test_main_question():
    assert Babylon([(1, "Y")]).is_over()
    assert not babylon_wins([(1, "Y")])                  # one stack: no move, the player to move loses
    assert babylon_wins([(1, "Y"), (1, "W")])            # one move ends the game, we win
    game = Babylon([(1, "Y"), (1, "Y"), (1, "W")])
    assert (0, 1) in game.legal_moves()
    while not game.is_over():                             # play a whole game with the solver
        game.play(*best_move(game))
    assert len(game.stacks) >= 1
    try:
        Babylon([(1, "Y"), (2, "W")]).play(0, 1)
```

```
        raise AssertionError("move should be illegal")
    except ValueError:
        pass
    assert len(Babylon().legal_moves()) > 0


# ======================================================
# FOLLOW-UP 1: How big is the real state space?
# ======================================================
# Answer: the real start has 12 single tiles, 12 * 11 = 132 ordered moves,
# and a position is a multiset of (height, colour) stacks, so the number
# of different positions is far smaller than the number of move orders:
# the memory (sorted tuple as the key) is what makes the search finish.
```

```
# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 29 OF 30

# Sliding window: grow on the right, shrink on the left while it is too bad

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**1**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Sliding window: grow on the right, shrink on the left while it is too bad | 1 questions | tap a title to jump to it**

**READ FIRST: the code template for these 1 questions (tap here) >**

H 51. Necklace Cut into Two Halves with Equal D/R Counts 1Point3Acres 2026-05-03

CODE TEMPLATE | GROUP 29 OF 30 | 1 QUESTIONS, ONE PATTERN

# Sliding window: grow on the right, shrink on the left while it is too bad

## The one idea these questions share

Move the right end one step at a time and add that item to your state. While the window breaks the rule, remove items from the left end. After each step the window is valid, so record its length (or the shortest valid one).

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_window(items, bad, allowed):
    left = used = best = 0
    for right, x in enumerate(items):
        used += bad(x)                          # CHANGE 1: add the right item to the state
        while used > allowed:                   # CHANGE 2: when the window is invalid
            used -= bad(items[left])            # CHANGE 3: remove the left item from the state
            left += 1
        best = max(best, right - left + 1)      # record the window
    return best
```

## 2. Helper functions that usually get written next to it

```
from collections import Counter, deque


def fixed_window_sums(nums, size):
    total = sum(nums[:size])
    out = [total]
    for i in range(size, len(nums)):
        total += nums[i] - nums[i - size]       # add the new item, drop the old one
        out.append(total)
    return out


def window_max(nums, size):                     # monotonic deque: the front is always the maximum
    q, out = deque(), []
    for i, x in enumerate(nums):
        while q and nums[q[-1]] <= x:
            q.pop()
        q.append(i)
        if q[0] <= i - size:
            q.popleft()
        if i >= size - 1:
            out.append(nums[q[0]])
    return out
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **51** | Necklace Cut into Two Halves with Equal D/R Counts | a fixed-size window of half the length slides around the doubled string, keeping a count of D |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

**STUDY PATH: question 51 of 53 | topic 29 of 30: Sliding window: grow on the right, shrink on the left while it is too bad | 1 of 1**

Template fit: a fixed-size window of half the length slides around the doubled string, keeping a count of D | asked 170 days ago | frequency Single report

**HARD | 1Point3Acres | onsite-coding | 51 of 53**

# Necklace Cut into Two Halves with Equal D/R Counts

**ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS**

**STUDY: TALK** **NO LEETCODE FREQUENCY** **PRIORITY 248/288**

## ABOUT THIS QUESTION

**Topic:** Sliding window: grow on the right, shrink on the left while it is too bad

**Optimal answer from:** written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

**Original:** https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

## QUESTION

A necklace is a circular string of beads D and R. Cut it in two places into two arcs of equal length that contain the same number of D beads (and so the same number of R beads). Return the two cut positions and the two arcs, or None if no such cut exists.

**Function signature**

def necklace_two_cuts(s: str) -> tuple | None

**(cut1, cut2, arc1, arc2)**

EXAMPLES

| Example 1 | |
|---|---|
| **Input** | `'DRDR'` |
| **Output** | `(0, 2, 'DR', 'DR')` |
| **Why** | `Cut at 0 and 2: both arcs are DR.` |

| Example 2 | |
|---|---|
| **Input** | `'DDRR'` |
| **Output** | `(1, 3, 'DR', 'RD')` |
| **Why** | `Linear halves DD and RR are unbalanced, but rotating the cuts by one gives DR and RD.` |

| Example 3 | |
|---|---|
| **Input** | `'DDR'` |
| **Output** | `null` |
| **Why** | `An odd number of beads cannot be split into equal arcs.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

**WHY IT IS SLOW**

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

## OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

## FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

## FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

## FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Necklace Cut into Two Halves with Equal
# D/R Counts
#
# A necklace is a circular string of beads D and R. Cut it
# in two places into two arcs of equal length that contain
# the same number of D beads (and so the same number of R
# beads). Return the two cut positions and the two arcs,
# or None if no such cut exists.
#
# Signature:
#   def necklace_two_cuts(s: str) -> tuple | None      # (cut1, cut2, arc1, arc2)
#
# Examples:
# 'DRDR'  ->  (0, 2, 'DR', 'DR')
# 'DDRR'  ->  (1, 3, 'DR', 'RD')
#
# FOLLOW-UPS (answered below, in this order):
# 1. When is a single cut enough?
# 2. What if the two pieces must be equal strings, not
# just equal counts?
# 3. The two pieces must be equal strings, not just equal
# counts.
# 4. The two pieces must be equal strings, not just have
# equal counts.
#
# INTERVIEW NOTES
# Two arcs of a circle with the same letter counts must
# each have exactly half the D's and half the R's, so the
# total counts must be even and each arc has length n / 2.
# Slide a window of length n / 2 around the circle: the
# window and its complement have D counts adding to the
# total, and the count changes by at most one per step, so
# it must hit half somewhere.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def count_letter(text, letter):
    count = 0
    for ch in text:
        if ch == letter:
            count += 1
    return count


# ---- main question: main function ----
# A circular necklace of D and R beads. Cut it in TWO places so that each of
# the two arcs holds half of the D's and half of the R's. Return
# (cut1, cut2, arc1, arc2) or None.
# Idea: if each arc has half of the D's and half of the R's, each arc has length
# n / 2 (and the total number of D's and of R's must both be even). So slide a
# window of length n / 2 once around the circle (work on s + s so the window may
# wrap) and keep its number of D's; when that equals half of all D's, the window
# and the rest of the circle are the two good arcs.
# Complexity: Time O(n), Space O(n) (the doubled string)
def necklace_two_cuts(s):
    n = len(s)
```

```
    d_total = count_letter(s, "D")
    r_total = count_letter(s, "R")
    if n % 2 == 1 or d_total % 2 == 1 or r_total % 2 == 1:
        return None                                     # each piece needs half of both letters
    half = n // 2
    target = d_total // 2
    doubled = s + s
    window = count_letter(doubled[:half], "D")          # the D's in the first window
    for i in range(n):
        if window == target:
            return i, (i + half) % n, doubled[i:i + half], doubled[i + half:i + n]
        if doubled[i + half] == "D":
            window += 1                                 # move the window one step: add the new bead ...
        if doubled[i] == "D":
            window -= 1                                 # ... and drop the old one
    return None


# the case of ONE cut (follow-up 1): the necklace is treated as a line
# Complexity: Time O(n), Space O(n)
def single_cut(s):
    if len(s) % 2 == 1:
        return None
    half = len(s) // 2
    if count_letter(s[:half], "D") * 2 == count_letter(s, "D"):
```

```
        return half                                     # a prefix with half of everything
    return None


# ---- main question: tests ----
def test_main_question():
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and count_letter(cuts[2], "D") == 1
    i, j, a, b = necklace_two_cuts("DDRR")              # DDRR needs two cuts on the circle: DR + DR
    assert sorted(a) == sorted(b) == ["D", "R"]
    assert necklace_two_cuts("DDR") is None             # odd totals have no answer
    assert necklace_two_cuts("DRDRDR") is None


# ========================================================
# FOLLOW-UP 1: When is a single cut enough?
# ========================================================
# Answer: on a LINE one cut must leave a prefix with half of the D's and
# half of the R's: single_cut checks that. DRDR works with one cut between the
# halves, DDRR does not (it needs two cuts on the circle).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert single_cut("DRDR") == 2
    assert single_cut("DDRR") is None


# ========================================================
# FOLLOW-UP 2: What if the two pieces must be equal strings, not just equal counts?
# ========================================================


# ========================================================
# FOLLOW-UP 3: The two pieces must be equal strings, not just equal counts
# ========================================================


# ========================================================
# FOLLOW-UP 4: The two pieces must be equal strings, not just have equal counts
# ========================================================
```

```
# ---- follow-up 2: function ----
# Answer: a different question: some ROTATION of the necklace must be of the
# form t + t. Try every rotation and compare its two halves (O(n^2)); or O(n)
# by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2.
# Complexity: Time O(n^2) (a half-length comparison at every start), Space O(n)
def equal_string_halves(s):
```

```
    n = len(s)
    if n % 2 == 1:
        return None
    half = n // 2
    doubled = s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert equal_string_halves("DRDR") is not None            # DRDR is DR + DR ...
    assert equal_string_halves("DDRR") is None                # ... DDRR is not


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**PREPARATION**

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 30 OF 30

# Split into pieces, fix each piece, join (and pack words into lines)

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

**2**
**HARD**

**How to use**

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**Split into pieces, fix each piece, join (and pack words into lines) | 2 questions | tap a title to jump to it**

**READ FIRST: the code template for these 2 questions (tap here) >**

H 52. Implement a function to justify a list of words within a given width. **Exponent no date**

H 53. Convert Valid Snake Case to Lower Camel Case **PracHub 2026-04-16**

CODE TEMPLATE | GROUP 30 OF 30 | 2 QUESTIONS, ONE PATTERN

# Split into pieces, fix each piece, join (and pack words into lines)

## The one idea these questions share

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

## 1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def reformat(text, sep="_"):
    head, *rest = text.split(sep)
    return head + "".join(p[:1].upper() + p[1:] for p in rest)   # CHANGE: the rule applied to each piece


def pack_lines(words, width):
    lines, cur = [], []
    for w in words:
        if cur and len(" ".join(cur + [w])) > width:             # does the next word still fit?
            lines.append(cur)
            cur = []
        cur.append(w)
    return lines + ([cur] if cur else [])
```

## 2. Helper functions that usually get written next to it

```
def spread_spaces(line_words, width):           # distribute spaces between words, extra ones on the left
    gaps = len(line_words) - 1
    if gaps == 0:
        return line_words[0].ljust(width)
    spaces, extra = divmod(width - sum(map(len, line_words)), gaps)
    out = ""
    for k, w in enumerate(line_words[:-1]):
        out += w + " " * (spaces + (1 if k < extra else 0))
    return out + line_words[-1]
```

## 3. What you change for each question in this group (the numbers match the contents)

| # | Question | What you change in the template |
|---|---|---|
| **52** | Implement a function to justify a list of words within a given width. | pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned |
| **53** | Convert Valid Snake Case to Lower Camel Case | reformat(): split on '_', keep the first piece, capitalise the others |

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

**STUDY PATH: question 52 of 53 | topic 30 of 30: Split into pieces, fix each piece, join (and pack words into lines) | 1 of 2**

Template fit: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned | no date

**HARD | IN APP LC 68 | Exponent | 52 of 53**

**★ TOP 75: rank 56 in the Medium and Hard priority list**

# ★ Implement a function to justify a list of words within a given width.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 44.7 | 30 DAYS 0 | PRIORITY 92/288

## ABOUT THIS QUESTION

**Topic:** Split into pieces, fix each piece, join (and pack words into lines)

**Optimal answer from:** your app's interview approach (LeetCode #68)

**LeetCode match:** LC 68 Text Justification (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

**Original:** https://www.aced.io/questions/4903/function-justify-words-within-width

## QUESTION

Given a list of words and a line width, pack the words into lines of exactly that width (full justification): put as many words on a line as fit (with at least one space between them), then spread the extra spaces between the words as evenly as possible, giving the leftmost gaps the extra ones. The last line, and any line with a single word, is left-justified and padded with spaces on the right. Return the list of lines.

### Function signature

def fullJustify(words: list, max_width: int) -> list

EXAMPLES

**Example 1**

| | |
|---|---|
| **Input** | `['This', 'is', 'an', 'example', 'of', 'text', 'justification.'], 16` |
| **Output** | `['This is an', 'example of text', 'justification. ']` |
| **Why** | Three lines of width 16: the first two are spread out, the last is left-aligned and padded. |

**Example 2**

| | |
|---|---|
| **Input** | `['a', 'b'], 5` |
| **Output** | `['a b ']` |
| **Why** | Both words fit on one line, which is the last line: left-aligned with single spaces and padded on the right. |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def justify(words, width):
    lines, i = [], 0
    while i < len(words):
        j = i + 1
        while j < len(words):
            if not fits(words[i:j + 1], width):
                break
            j += 1
        last = j == len(words)
        row = format_line(words[i:j],
                          width, last)
        lines.append(row)
        i = j
    return lines
```

| TIME | SPACE |
|---|---|
| O(total characters * words per line) | O(width) |

**WHY IT IS SLOW**

Every attempt to add a word re-joins the whole line to measure it, so each line costs about (words per line) times its length. Tracking only a running length makes it linear in the total characters.

## OPTIMAL SOLUTION · Greedy line packing

| TIME | SPACE |
|---|---|
| O(total characters) | O(total characters) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

### FOLLOW-UP 1 How would you change it to fully justify the last line too, or to center the text?

Only the final-line branch changes. For full justification drop the special case for the last line (it would be spread like every other line). For centering, split the leftover spaces between the two sides, giving the extra space to the right. The greedy packing step stays identical.

### FOLLOW-UP 2 What if the words contain wide characters or different pixel widths?

Replace len(word) by a measure(word) function (for example font metrics or East Asian width) and compare against the pixel width. Packing and distribution logic stay the same; spaces are measured in the same unit, so the spare width may not divide evenly and the remainder must be assigned to gaps in whole space units.

### FOLLOW-UP 3 Fully justify the last line too.

Drop the special case for the last line (a line with more than one word is spread like the others). Single-word lines stay left-aligned.

### FOLLOW-UP 4 Words have pixel widths (a proportional font).

Replace len(word) with a measure function and compare against the pixel width; the extra space is distributed in whole space units, so the remainder rule stays the same.

**FOLLOW-UP 5 Minimise raggedness instead of packing greedily.**

That is the Knuth-Plass style DP: cost of a line = (spare space)^2, minimise the total over all ways to break the text. O(n * words per line).

**FULL INTERVIEW CODE · clean layout: main question, then each follow-up**

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a function to justify a list of
# words within a given width.
#
# Given a list of words and a line width, pack the words
# into lines of exactly that width (full justification):
# put as many words on a line as fit (with at least one
# space between them), then spread the extra spaces
# between the words as evenly as possible, giving the
# leftmost gaps the extra ones. The last line, and any
# line with a single word, is left-justified and padded
# with spaces on the right. Return the list of lines.
#
# Signature:
#   def fullJustify(words: list, max_width: int) -> list
#
# Examples:
# ['This', 'is', 'an', 'example', 'of', 'text',
#     'justification.'], 16  ->  ['This    is    an',
#     'example  of text', 'justification.  ']
# ['a', 'b'], 5  ->  ['a b  ']
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you change it to fully justify the last
# line too, or to center the text?
# 2. What if the words contain wide characters or
# different pixel widths?
# 3. Fully justify the last line too.
# 4. Words have pixel widths (a proportional font).
# 5. Minimise raggedness instead of packing greedily.
#
# INTERVIEW NOTES
# Two decisions, kept separate: which words go on each
# line (pack greedily while the words plus one space
# between them fit), and how to spread the leftover spaces
# (an equal share in each gap, the extra spaces to the
# leftmost gaps). The last line and one-word lines are the
# special cases to name before coding.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# `measure(text)` is how wide a piece of text is. It is len() for plain
```

```
# characters; follow-ups 2 and 4 pass another function (wide characters,
# pixel widths) and nothing else has to change.
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line (one space between words)
    lines = []
    line = []
    used = 0                                    # width of the line so far, spaces included
    for word in words:
        size = measure(word)
        if line and used + 1 + size > width:
            lines.append(line)                  # the word does not fit: close the line
            line = []
            used = 0
        if line:
            used += 1
        used += size
        line.append(word)
    if line:
        lines.append(line)
    return lines


def spread(line, width, measure=len):
    # fully justified: the spare space is shared over the gaps, and the EXTRA
    # spaces (when it does not divide evenly) go to the LEFT gaps first.
    # A line with one word is left-aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    letters = 0
    for word in line:
        letters += measure(word)
    spaces = width - letters
    gaps = len(line) - 1
    each = spaces // gaps
    extra = spaces % gaps
    text = ""
    for i in range(gaps):
        text += line[i] + " " * each
        if i < extra:
            text += " "
    return text + line[-1]


def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # the spare spaces are split on both sides, the odd one goes right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# ---- main question: main function ----
# Idea: greedy line packing. Put as many words as fit on a line (counting
# one space between words). Spread the spare spaces over the gaps, giving
# the extra ones to the leftmost gaps. The LAST line is left-aligned.
# Complexity: Time O(total characters), Space O(total characters)
def fullJustify(words, max_width):
    lines = pack_greedy(words, max_width)
    result = []
```

```
    for i in range(len(lines)):
        if i < len(lines) - 1:
            result.append(spread(lines[i], max_width))
        else:
            result.append(left_align(lines[i], max_width))
    return result


# ---- main question: tests ----
def test_main_question():
    text = "This is an example of text justification.".split()
    assert fullJustify(text, 16) == ["This    is    an", "example  of text", "justification.  "]
    words = "What must be acknowledgment shall be".split()
    got = fullJustify(words, 16)
    assert got == ["What   must   be", "acknowledgment  ", "shall be        "]
    for line in got:
        assert len(line) == 16
    assert fullJustify(["a"], 3) == ["a  "]
```

```
    assert fullJustify([], 5) == []


# ========================================================
# FOLLOW-UP 1: How would you change it to fully justify the last line too, or to center the text?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Fully justify the last line too
# ========================================================
# ---- follow-up 1: function ----
# Answer: only the branch for the last line changes. To justify it too, drop
# the special case (a line with several words is spread like the others;
# a one-word line stays left-aligned). To center it, use center().
# Complexity: Time O(total characters), Space O(total characters)
def justify_options(words, width, last="left", measure=len):
    lines = pack_greedy(words, width, measure)
    result = []
    for i in range(len(lines)):
        if i < len(lines) - 1 or last == "full":
            result.append(spread(lines[i], width, measure))
        elif last == "center":
            result.append(center(lines[i], width, measure))
        else:
            result.append(left_align(lines[i], width, measure))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = "What must be acknowledgment shall be".split()
    assert justify_options(words, 16, last="full")[2] == "shall         be"
    assert justify_options(words, 16, last="center")[2] == "    shall be    "
    assert justify_options(words, 16) == fullJustify(words, 16)


# ========================================================
# FOLLOW-UP 2: What if the words contain wide characters or different pixel widths?
# ========================================================


# ========================================================
# FOLLOW-UP 4: Words have pixel widths (a proportional font)
# ========================================================
```

```
# ---- follow-up 2: tests ----
# Answer: replace len(word) by a measure(word) function (font metrics, or
# East Asian character width) and compare against the pixel width. Extra
# space is still handed out in whole space units, so the remainder rule
# stays the same. The helpers above already take a `measure` argument.
def double_width(text):
    return 2 * len(text)                    # pretend every character is 2 units wide


def test_follow_up_2():
    lines = justify_options(["ab", "cd", "ef"], 12, measure=double_width)
    assert lines == ["ab    cd", "ef" + " " * 8]


# ========================================================
# FOLLOW-UP 5: Minimise raggedness instead of packing greedily
# ========================================================
# ---- follow-up 5: function ----
# Answer: this is the Knuth-Plass idea, a DP: the cost of a line is
# (spare space)^2 (the last line is free), and best[i] is the cheapest
# way to set the first i words. Try every possible start j of the last
# line. O(n * words per line). Greedy packing is not optimal for this goal.
# Complexity: Time O(n * k) for k words per line, Space O(n)
def pack_optimal(words, width):
    n = len(words)
    best = [0] + [float("inf")] * n
    previous = [0] * (n + 1)
    for i in range(1, n + 1):
        size = -1
        for j in range(i - 1, -1, -1):            # a line holds words j .. i - 1; try the longest first
            size += len(words[j]) + 1
            if size > width:
                break
            cost = 0
            if i < n:
                cost = (width - size) ** 2
            if best[j] + cost < best[i]:
                best[i] = best[j] + cost
                previous[i] = j
    lines = []
    i = n
    while i > 0:
        lines.append(words[previous[i]:i])
        i = previous[i]
    lines.reverse()
    return lines


# ---- follow-up 5: tests ----
def raggedness(lines, width):
    # the sum of the squares of the unused space (the last line is free)
    cost = 0
    for line in lines[:-1]:
        cost += (width - len(" ".join(line))) ** 2
    return cost


def test_follow_up_5():
    random.seed(34)
    for round_number in range(200):
        words = []
        for i in range(random.randint(1, 9)):
            words.append("x" * random.randint(1, 6))
        optimal = pack_optimal(words, 8)
        flat = []
```

```
        for line in optimal:
            flat += line
        assert flat == words
        for line in optimal:
            assert len(" ".join(line)) <= 8
        assert raggedness(optimal, 8) <= raggedness(pack_greedy(words, 8), 8)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

**STUDY PATH: question 53 of 53 | topic 30 of 30: Split into pieces, fix each piece, join (and pack words into lines) | 2 of 2**

Template fit: reformat(): split on '_', keep the first piece, capitalise the others | asked 187 days ago

**HARD | PracHub | Technical Screen | 53 of 53**

# Convert Valid Snake Case to Lower Camel Case

POSTED 2026-04-16 | 187 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 244/288

## ABOUT THIS QUESTION

**Topic:** Split into pieces, fix each piece, join (and pack words into lines)

**Optimal answer from:** the interview site's own solution

**LeetCode match:** No LC match (easy string manipulation)

Posted on PracHub: 2026-04-16 (updated 2026-07-18)

**Original:** https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

## QUESTION

Given a valid snake_case identifier (one or more lowercase words separated by single underscores, with no leading, trailing or doubled underscores), convert it to lowerCamelCase: keep the first word as it is, remove the underscores and capitalise the first letter of every later word.

### Function signature

def snake_to_camel(s: str) -> str

EXAMPLES

| Example 1 | |
|---|---|
| Input | `'get_http_response_code'` |
| Output | `'getHttpResponseCode'` |
| Why | `Words are joined and every word after the first is capitalised.` |

| Example 2 | |
|---|---|
| Input | `'user_id'` |
| Output | `'userId'` |
| Why | `Two words.` |

| Example 3 | |
|---|---|
| Input | `'name'` |
| Output | `'name'` |
| Why | `A single word has nothing to change.` |

## BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

## BRUTE FORCE

```
def to_camel(s):
    out, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            out += ch.upper() if up else ch
            up = False
    return out
```

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**WHY IT IS SLOW**

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

## OPTIMAL SOLUTION · Split on underscores and capitalize

| TIME | SPACE |
|---|---|
| O(n) | O(n) |

**Code:** see FULL INTERVIEW CODE below (the optimal solution is its main function).

## FOLLOW-UP 1 The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

## FOLLOW-UP 2 Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

## FULL INTERVIEW CODE · clean layout: main question, then each follow-up

**One clean program for the whole question:** the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Convert Valid Snake Case to Lower Camel
# Case
#
# Given a valid snake_case identifier (one or more
# lowercase words separated by single underscores, with no
# leading, trailing or doubled underscores), convert it to
# lowerCamelCase: keep the first word as it is, remove the
# underscores and capitalise the first letter of every
# later word.
#
# Signature:
#   def snake_to_camel(s: str) -> str
#
# Examples:
# 'get_http_response_code'  ->  'getHttpResponseCode'
# 'user_id'  ->  'userId'
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# underscores, or mixed case.
# 2. Convert back from camel case to snake case.
#
# INTERVIEW NOTES
# The statement says the input is valid (single
# underscores, no edge underscores), so rely on that and
# say what you would add if it were not. The pattern is
# 'first word unchanged, capitalize the first letter of
# each later word'.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the input is valid, so there are no empty parts. Split on "_",
# keep the first word as it is, and give every other word a capital first
# letter.
# Complexity: Time O(n), Space O(n)
def snake_to_camel(s):
    parts = s.split("_")
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result


# ---- main question: tests ----
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    assert snake_to_camel("get_http_response") == "getHttpResponse"


# ============================================================
# FOLLOW-UP 1: The input may have leading, trailing or repeated underscores, or mixed case
# ============================================================
# ---- follow-up 1: function ----
# Answer: drop the empty parts after the split, lowercase the first word
# and capitalize the rest. Say what you do with digits and other
# characters, and keep the rule in one place.
# Complexity: Time O(n), Space O(n)
def snake_to_camel_tolerant(s):
    words = []
    for part in s.split("_"):
        if part != "":
            words.append(part)
    if len(words) == 0:
        return ""
    result = words[0].lower()
    for word in words[1:]:
        result += word.capitalize()
    return result
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert snake_to_camel_tolerant("_Foo__BAR_") == "fooBar"
    assert snake_to_camel_tolerant("___") == ""


# ============================================================
# FOLLOW-UP 2: Convert back from camel case to snake case
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: put an underscore before an uppercase letter and lowercase it.
# Acronyms such as HTTPServer need a smarter rule: an underscore goes
# before an uppercase letter when the previous letter is lowercase or a
# digit, or when it is uppercase but the NEXT letter is lowercase (the
# start of a new word after an acronym).
# Complexity: Time O(n), Space O(n)
def camel_to_snake(s):
    result = ""
    for i in range(len(s)):
        ch = s[i]
        if ch.isupper() and i > 0:
            previous = s[i - 1]
            next_is_lower = i + 1 < len(s) and s[i + 1].islower()
            if previous.islower() or previous.isdigit() or (previous.isupper() and next_is_lower):
                result += "_"
        result += ch.lower()
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    assert camel_to_snake(snake_to_camel("my_var_2")) == "my_var2"       # and back again


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

## FUNCTION AND TEST CASES

### Signature

```
def solve(name) -> str:
```

### Sample call

```
solve('a')
```

### Test cases

| Input | Expected |
| --- | --- |
| `('a',)` | `'a'` |
| `('already',)` | `'already'` |

**ALSO REPORTED · 2 more**

**Define the scope and requirements for a snake_case to camelCase converter function. | Exponent**

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

**Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report**

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.